

Boring-NixOS

K19G

2026-09-08

Table of contents

Boring NixOS	22
Boring NixOS	23
Who this is for	23
What “boring” means here	23
How to read	24
How examples work	24
Map of the book	24
What this book is not	26
 Introduction	 27
 The Reproducibility Crisis	 28
The Reproducibility Crisis	29
Mental model	29
Worked examples	29
Case 1: Detecting Ambient Host Contamination	29
Case 2: Purity and Isolated Evaluation	30
Case 3: Hermetic Sandboxing in Action	31
The trap	31
The boring rule	32
Try this	32
 Nix Fundamentals	 33
 Store Model	 34
The /nix/store Anatomy	35
The /nix/store Anatomy	36
Worked examples	36
The boring rule	36
 Content-Addressable Paths	 37
Content-Addressable Paths	38
Mental model	38
Worked examples	38

The boring rule	38
Hashes and Dependencies	39
Cryptographic Hashes and Inputs	40
Cryptographic Hashes and Inputs	41
Mental model	41
Worked examples	41
The boring rule	41
Deterministic Dynamic Linking and RPATH	42
Deterministic Dynamic Linking and RPATH	43
Mental model	43
Worked examples	43
The boring rule	43
Dependency Graphs	44
Dependency Closures and Graphs	45
Dependency Closures and Graphs	46
Mental model	46
Worked examples	46
The boring rule	46
The Nix Database (db.sqlite)	47
The Nix Database (db.sqlite)	48
Mental model	48
Worked examples	48
The boring rule	48
Garbage Collection	49
GC Roots and Space Reclamation	50
GC Roots and Space Reclamation	51
Mental model	51
Worked examples	51
The boring rule	51
Store Optimization and Deduplication	52
Store Optimization and Deduplication	53
Mental model	53

Worked examples	53
The boring rule	53
First Expressions	54
The Derivation Primitive	55
The Derivation Primitive	56
Worked examples	56
The boring rule	56
Building Multi-File Artifacts	57
Building Multi-File Artifacts	58
Mental model	58
Worked examples	58
The boring rule	59
Nix Language	60
Nix Expressions and Evaluation	61
Nix Expressions and Evaluation	62
Mental model	62
Worked examples	62
Case 1: Evaluating Expressions with nix eval	62
Case 2: Multi-Expression File Structure	63
Case 3: Proving Lazy Evaluation	63
The trap	64
The boring rule	64
Try this	64
Values and Immutability	65
Values and Immutability	66
Mental model	66
Worked examples	66
Case 1: Proving Immutability Across Merges	66
Case 2: Transforming Lists Immutably	67
The trap	68
The boring rule	68
Try this	68
Data Types and String Interpolation	69
Data Types and String Interpolation	70
Mental model	70

Worked examples	70
Case 1: Checking and Inspecting Types	70
Case 2: String Interpolation and Multi-Line Blocks	71
Case 3: Path Interpolation into the Nix Store	72
The trap	72
The boring rule	72
Try this	72
Let-In Blocks and Local Bindings	73
Let-In Blocks and Local Bindings	74
Mental model	74
Worked examples	74
Case 1: Structuring Component Configurations	74
Case 2: Lexical Scope and Shadowing	75
The trap	75
The boring rule	76
Try this	76
Functions and Argument Defaults	77
Functions and Argument Defaults	78
Mental model	78
Worked examples	78
Case 1: Curried Functions vs Destructured Sets	78
Case 2: Capturing Full Sets with the @ Pattern	79
The trap	79
The boring rule	80
Try this	80
Attribute Sets and Recursive Definitions	81
Attribute Sets and Recursive Definitions	82
Mental model	82
Worked examples	82
Case 1: Merging and Fallback Operations	82
Case 2: Recursive Attribute Sets (rec)	83
The trap	83
The boring rule	84
Try this	84
Conditionals, Lists, and Built-in Operations	85
Conditionals, Lists, and Built-in Operations	86
Mental model	86
Worked examples	86
Case 1: Conditional Configuration Flags	86
Case 2: Transforming and Filtering Lists	87
The trap	87

The boring rule	88
Try this	88
Importing Files and Building Abstractions	89
Importing Files and Building Abstractions	90
Mental model	90
Worked examples	90
Case 1: Modular Service Definition	90
Case 2: Standard Library Utilities (pkgs.lib)	91
The trap	92
The boring rule	92
Try this	92
 Dev Environments	 93
Nix Shells and the devShell Concept	94
Nix Shells and the devShell Concept	95
Mental model	95
Worked examples	95
Case 1: Simple shell.nix	95
Case 2: Setting Environment Variables in a Shell	96
The trap	97
The boring rule	97
Try this	97
 Multi-Language Development Environments	 98
Multi-Language Development Environments	99
Mental model	99
Worked examples	99
Case 1: Unified Polyglot Stack	99
Case 2: C Libraries and pkg-config	100
The trap	100
The boring rule	101
Try this	101
 Flakes for Reproducible Dev Environments	 102
Flakes for Reproducible Dev Environments	103
Mental model	103
Worked examples	103
Case 1: Minimal Flake Development Shell	103
Case 2: Updating Dependencies	104
The trap	105
The boring rule	105
Try this	105

Environment Management with Home Manager	106
Environment Management with Home Manager	107
Mental model	107
Worked examples	107
Case 1: Minimal home.nix Configuration	107
The trap	108
The boring rule	108
Try this	108
Migrating from Docker Desktop to Nix DevShells	109
Migrating from Docker Desktop to Nix DevShells	110
Mental model	110
Worked examples	110
Case 1: The Docker Compose Pattern vs The Nix devShell Pattern	110
Case 2: Hybrid Workflow — Native Tooling with Containerized Backends	111
The trap	112
The boring rule	112
Try this	112
Packages and Overlays	113
The Nix Package Manager in Practice	114
The Nix Package Manager in Practice	115
Mental model	115
Worked examples	115
Case 1: Ephemeral CLI Execution with nix shell	115
Case 2: User Profile Management with nix profile	116
Case 3: Rollback in Action	116
The trap	116
The boring rule	117
Try this	117
Understanding the Flakes Input System	118
Understanding the Flakes Input System	119
Mental model	119
Worked examples	119
Case 1: Deduplicating Dependencies with follows	119
Case 2: Inspecting and Auditing Inputs	120
The trap	121
The boring rule	121
Try this	121
Creating and Composing Overlays	122

Creating and Composing Overlays	123
Mental model	123
Worked examples	123
Case 1: Simple Package Override via an Overlay	123
Case 2: Composing Multiple Overlays	124
The trap	125
The boring rule	125
Try this	125
Custom Package Definitions	126
Custom Package Definitions	127
Mental model	127
Worked examples	127
Case 1: Defining a Script Package	127
The trap	129
The boring rule	129
Try this	129
Managing Dependencies Across Projects	130
Managing Dependencies Across Projects	131
Mental model	131
Worked examples	131
Case 1: Exposing Shared Packages in a Flake	131
Case 2: Consuming Shared Tooling in a Project Flake	132
The trap	133
The boring rule	133
Try this	133
Building From Source	134
The Derivation Build Process	135
The Derivation Build Process	136
Mental model	136
Worked examples	136
Case 1: Customizing Build Phases	136
The trap	138
The boring rule	138
Try this	138
Building C and C++ Projects with CMake	139
Building C and C++ Projects with CMake	140
Mental model	140
Worked examples	140
Case 1: CMake Project Packaging	140

The trap	142
The boring rule	142
Try this	142
Rust Packages with Cargo	143
Rust Packages with Cargo	144
Mental model	144
Worked examples	144
Case 1: Packaging a Rust Application	144
The trap	146
The boring rule	146
Try this	146
Python Packages with Pip and Setuptools	147
Python Packages with Pip and Setuptools	148
Mental model	148
Worked examples	148
Case 1: Standalone Python Application	148
The trap	150
The boring rule	150
Try this	150
Go Modules and Node.js Packages	151
Go Modules and Node.js Packages	152
Mental model	152
Worked examples	152
Case 1: Building a Go Module Application	152
Case 2: Node.js with buildNpmPackage	153
The trap	154
The boring rule	154
Try this	154
Cross-Compilation for Multiple Platforms	155
Cross-Compilation for Multiple Platforms	156
Mental model	156
Worked examples	156
Case 1: Cross-Compiling a C Program for ARM64	156
The trap	157
The boring rule	158
Try this	158
NixOS System	159
The NixOS Module System Architecture	160

The NixOS Module System Architecture	161
Mental model	161
Worked examples	161
Case 1: Minimal Self-Contained Module	161
Case 2: Priority Resolution with mkDefault and mkForce	162
The trap	162
The boring rule	163
Try this	163
Configuring Your First NixOS Machine	164
Configuring Your First NixOS Machine	165
Mental model	165
Worked examples	165
Case 1: Minimal Production configuration.nix	165
The trap	167
The boring rule	167
Try this	167
Service Management with systemd	168
Service Management with systemd	169
Mental model	169
Worked examples	169
Case 1: Declarative Background Service	169
Case 2: Scheduled Timers (cron replacement)	170
The trap	171
The boring rule	171
Try this	171
Networking, Firewall, and Security	172
Networking, Firewall, and Security	173
Mental model	173
Worked examples	173
Case 1: Hardened Network Configuration	173
Case 2: WireGuard VPN Configuration	174
The trap	174
The boring rule	175
Try this	175
Users, Groups, and SSH Configuration	176
Users, Groups, and SSH Configuration	177
Mental model	177
Worked examples	177
Case 1: Declarative Team Access and Hardened SSH	177
Case 2: Provisioning Password Hashes Safely	178
The trap	179

The boring rule	179
Try this	179
Hardware Configuration and Drivers	180
Hardware Configuration and Drivers	181
Mental model	181
Worked examples	181
Case 1: Inspecting an Auto-Generated hardware-configuration.nix	181
Case 2: Proprietary GPU and OpenGL Drivers	182
The trap	183
The boring rule	183
Try this	183
Declarative Disk Partitioning with Disko	184
Declarative Disk Partitioning with Disko	185
Mental model	185
Worked examples	185
Case 1: Complete Disko layout with LUKS2 and Btrfs subvolumes	185
Case 2: Zero-touch deployment via nixos-anywhere	187
The trap	187
The boring rule	187
Try this	188
Ephemeral Roots and Impermanence	189
Ephemeral Roots and Impermanence	190
Mental model	190
Worked examples	190
Case 1: Impermanence module configuration	190
Case 2: Verification of state destruction	191
The trap	192
The boring rule	192
Try this	192
Home Manager	193
Home Manager and the Module System	194
Home Manager and the Module System	195
Mental model	195
Worked examples	195
The boring rule	196
Managing Dotfiles Declaratively	197
Managing Dotfiles Declaratively	198
Mental model	198

Worked examples	198
The boring rule	198
Shell and Editor Configuration	199
Shell and Editor Configuration	200
Mental model	200
Worked examples	200
The boring rule	201
Application Settings and Preferences	202
Application Settings and Preferences	203
Mental model	203
Worked examples	203
The boring rule	203
Combining NixOS and Home Manager	204
Combining NixOS and Home Manager	205
Mental model	205
Worked examples	205
The boring rule	206
CI/CD with Nix	207
Setting Up Binary Caches	208
Setting Up Binary Caches	209
Mental model	209
Worked examples	209
The boring rule	209
GitHub Actions with Nix	210
GitHub Actions with Nix	211
Mental model	211
Worked examples	211
The boring rule	212
GitLab CI Integration	213
GitLab CI Integration	214
Mental model	214
Worked examples	214
The boring rule	214
Build Caching Strategies	216

Build Caching Strategies	217
Mental model	217
Worked examples	217
Case 1: Minimal GitHub Actions cache configuration	217
Case 2: Isolating dependencies to prevent cache bust	218
The trap	218
The boring rule	218
Try this	219
Testing in CI Pipelines	220
Testing in CI Pipelines	221
Mental model	221
Worked examples	221
Case 1: Package with hermetic checkPhase	221
Case 2: Flake check definition	222
The trap	223
The boring rule	223
Try this	223
Secrets Management in CI	224
Secrets Management in CI	225
Mental model	225
Worked examples	225
Case 1: Providing GitHub token for private Flake inputs	225
Case 2: Supplying Age decryption key to sops-nix in CI tests	226
The trap	226
The boring rule	226
Try this	226
Containers and Kubernetes	227
Creating Container Images with Nix (dockerTools)	228
Creating Container Images with Nix (dockerTools)	229
Mental model	229
Worked examples	229
Case 1: Minimal Go service container with buildLayeredImage	229
The trap	230
The boring rule	230
Try this	231
Kubernetes Manifests as Nix Modules	232
Kubernetes Manifests as Nix Modules	233
Mental model	233

Worked examples	233
Case 1: Reusable workload generator	233
The trap	234
The boring rule	235
Try this	235
Helm Charts Generated from Nix	236
Helm Charts Generated from Nix	237
Mental model	237
Worked examples	237
Case 1: Packaging a Helm chart with Nix	237
The trap	238
The boring rule	238
Try this	238
GitOps with Argo CD and Flux	239
GitOps with Argo CD and Flux	240
Mental model	240
Worked examples	240
Case 1: CI workflow rendering manifests for Argo CD	240
Case 2: Argo CD Application definition	241
The trap	241
The boring rule	242
Try this	242
Service Mesh and Observability	243
Service Mesh and Observability	244
Mental model	244
Worked examples	244
Case 1: Generating OpenTelemetry Collector configuration	244
The trap	245
The boring rule	245
Try this	245
Infrastructure as Code	246
Managing Terraform Providers with Nix	247
Managing Terraform Providers with Nix	248
Mental model	248
Worked examples	248
Case 1: Bundling Terraform with AWS and Cloudflare providers	248
Case 2: Running init without network access	249
The trap	249
The boring rule	249

Try this	249
OpenTofu Integration	250
OpenTofu Integration	251
Mental model	251
Worked examples	251
Case 1: OpenTofu environment with Kubernetes provider	251
The trap	252
The boring rule	252
Try this	252
Ansible Provisioning with Nix	253
Ansible Provisioning with Nix	254
Mental model	254
Worked examples	254
Case 1: Ansible with cloud dependencies	254
The trap	255
The boring rule	255
Try this	255
Cloud Provider Configuration	256
Cloud Provider Configuration	257
Mental model	257
Worked examples	257
Case 1: Multi-cloud management shell	257
The trap	258
The boring rule	258
Try this	258
Multi-Cloud Deployment Patterns	259
Multi-Cloud Deployment Patterns	260
Mental model	260
Worked examples	260
Case 1: Building a target-specific VM image	260
The trap	261
The boring rule	261
Try this	261
Secrets and Security	262
Why Secrets Are Hard with Declarative Config	263
Why Secrets Are Hard with Declarative Config	264
Mental model	264

Worked examples	264
Case 1: Demonstrating store leakage (Anti-Pattern)	264
Case 2: The boring safe pattern (Reference by path)	265
The trap	265
The boring rule	265
Try this	265
Using SOPS for Encrypted Configuration (sops-nix)	266
Using SOPS for Encrypted Configuration (sops-nix)	267
Mental model	267
Worked examples	267
Case 1: .sops.yaml configuration	267
Case 2: NixOS module integration	268
The trap	268
The boring rule	268
Try this	269
HashiCorp Vault Integration	270
HashiCorp Vault Integration	271
Mental model	271
Worked examples	271
Case 1: Declarative Vault Agent configuration	271
The trap	272
The boring rule	272
Try this	272
Age Encryption for Simple Secrets	273
Age Encryption for Simple Secrets	274
Mental model	274
Worked examples	274
Case 1: Defining secrets rules with agenix	274
Case 2: Consuming encrypted secret in NixOS	275
The trap	275
The boring rule	275
Try this	275
Security Hardening Patterns	276
Security Hardening Patterns	277
Mental model	277
Worked examples	277
Case 1: Hardened systemd service definition	277
Case 2: Kernel hardening profile	278
The trap	278
The boring rule	279
Try this	279

Platform Engineering	280
The Internal Developer Platform Pattern	281
The Internal Developer Platform Pattern	282
Mental model	282
Worked examples	282
Case 1: Central platform flake structure	282
The trap	283
The boring rule	283
Try this	283
Monorepo Strategies with Nix	284
Monorepo Strategies with Nix	285
Mental model	285
Worked examples	285
Case 1: Monorepo workspace flake	285
The trap	286
The boring rule	286
Try this	286
Multi-Platform Development and Distribution	287
Multi-Platform Development and Distribution	288
Mental model	288
Worked examples	288
Case 1: Multi-platform flake with flake-utils	288
The trap	289
The boring rule	289
Try this	289
Build Optimization and Performance Tuning	290
Build Optimization and Performance Tuning	291
Mental model	291
Worked examples	291
Case 1: Recommended <code>/etc/nix/nix.conf</code> settings	291
The trap	292
The boring rule	292
Try this	292
Debugging and Troubleshooting Production Issues	293
Debugging and Troubleshooting Production Issues	294
Mental model	294
Worked examples	294
Case 1: Inspecting build logs for failed derivation	294
Case 2: Stepping through build phases interactively	295
Case 3: Explaining system divergence with <code>nix-diff</code>	295

The trap	295
The boring rule	295
Try this	295
Testing Strategies for Nix Configurations	296
Testing Strategies for Nix Configurations	297
Mental model	297
Worked examples	297
Case 1: Writing a NixOS integration test	297
The trap	298
The boring rule	298
Try this	298
Maintenance, Upgrades, and Migration	300
Maintenance, Upgrades, and Migration	301
Mental model	301
Worked examples	301
Case 1: Updating flake inputs and reviewing changes	301
Case 2: Dry-run activation before touching running services	301
Case 3: Executing instant rollback	302
The trap	302
The boring rule	302
Try this	302
Enterprise Operational Checklists	303
Enterprise Operational Checklists	304
Mental model	304
Worked examples	304
The Repository Layout Checklist	304
The Pre-Production Deployment Checklist	304
The Fleet Maintenance Checklist	304
The trap	305
The boring rule	305
Try this	305
Fleet Deployments with Colmena and Deploy-RS	306
Fleet Deployments with Colmena and Deploy-RS	307
Mental model	307
Worked examples	307
Case 1: Defining a multi-node fleet in <code>flake.nix</code>	307
The trap	308
The boring rule	309
Try this	309
Lightweight MicroVMs with <code>microvm.nix</code>	310

Lightweight MicroVMs with microvm.nix	311
Mental model	311
Worked examples	311
Case 1: MicroVM definition inside a host Flake	311
The trap	312
The boring rule	312
Try this	313
Content-Addressed Nix and NAR Internals	314
Content-Addressed Nix and NAR Internals	315
Mental model	315
Worked examples	315
Case 1: Inspecting the canonical NAR serialization	315
Case 2: Content-addressed derivation declaration	316
The trap	317
The boring rule	317
Try this	317
Production Capstone Lab	318
Production Architecture and Threat Model	319
Production Architecture and Threat Model	320
The Production Topology	320
Tier 1: Edge Ingress Gateway (gw-01)	321
Tier 2: Stateless Application Cluster (app-01, app-02)	321
Tier 3: Stateful Database Cluster (db-01)	321
Tier 4: Operations, Deployment, & Telemetry (ops-01)	321
Threat Model and Security Invariants	321
Hands-on Lab Overview	322
Disko Storage and Ephemeral Base System	323
Disko Storage and Ephemeral Base System	324
Mental model	324
Worked examples	324
Case 1: The Production Disko Module	324
Case 2: The Impermanence Persistent State Contract	326
The trap	327
The boring rule	327
Try this	327
Hardened Networking and WireGuard Mesh	328
Hardened Networking and WireGuard Mesh	329
Mental model	329

Worked examples	329
Case 1: WireGuard Mesh Module	329
Case 2: Strict nftables Firewall Configuration for Database Node	330
The trap	331
The boring rule	331
Try this	331
Distributed Application Stack and Secrets	332
Distributed Application Stack and Secrets	333
Mental model	333
Worked examples	333
Case 1: Caddy Reverse Proxy Configuration (gw-01)	333
Case 2: Sandboxed Application Service (app-01)	334
The trap	335
The boring rule	335
Try this	335
Automated Fleet Rollout and Disaster Recovery	336
Automated Fleet Rollout and Disaster Recovery	337
Mental model	337
Worked examples	337
Case 1: The Fleet Flake (flake.nix)	337
Case 2: Executing Fleet Rollout with Canary Verification	338
Case 3: Zero-Touch Disaster Recovery Playbook	339
The trap	339
The boring rule	339
Try this	340
Appendices	341
Glossary of Terms	342
Glossary of Terms	343
Derivation (.drv)	343
Nix Store (/nix/store)	343
Closure	343
Flake	343
DevShell	343
Binary Cache / Substituter	343
Garbage Collection Root (GC Root)	344
Home Manager	344
Overlay	344
Module System	344
Command Cheat Sheet	345

Command Cheat Sheet	346
Flake and Development Commands	346
Building and Querying	346
NixOS Operations	346
Maintenance and Disk Cleanup	347

Boring NixOS

Boring NixOS

A direct, practical guide to configuring infrastructure that builds identically every time: **deterministic, immutable, and refreshingly boring**.

Ambiguity is the enemy of reliability. When machines mutate silently through ad-hoc commands, builds drift, onboarding breaks, and outages follow. The boring default is to define the full closure in code — from developer compilers to multi-region cloud clusters — so every environment evaluates identically six months later.

Baseline: Nix **2.24+** · NixOS **24.11+** · Flakes enabled. Standalone and self-contained. You do not need any other title in this library.

Who this is for

- Engineers, platform developers, and SREs who want systems that build identically on a laptop, CI runner, and bare-metal server.
- Developers looking for instant, clean project environments without container lag or host library pollution.
- Teams moving away from fragile setup scripts and mutable system administration toward pure declarative configuration.

You do not need prior experience with Nix or functional programming. A terminal, a standard text editor, and basic Linux familiarity are all you need.

What “boring” means here

Nix is a purely functional package manager and domain-specific language. Applied with discipline, it eliminates entire categories of operational bugs: conflicting shared libraries, broken symlinks, broken OS upgrades, and undocumented host drift.

When over-complicated, it can become an impenetrable maze of nested overlays and tangled macros.

Boring NixOS teaches clear, sustainable defaults: - Plain, readable expressions over clever meta-programming. - Explicit inputs pinned by cryptographic lockfiles. - Modular, auditable NixOS system definitions that scale without surprise.

How to read

1. Read the chapter concept and mental model.
2. Type the expression into its designated file (`default.nix`, `shell.nix`, `flake.nix`, or `configuration.nix`).
3. Run the exact build or evaluation command.
4. Experiment with the **Try this** exercises to verify how the store and evaluator behave.

The chapters are sequenced progressively. You can jump directly to devShells, NixOS services, or CI caching once you know the language basics.

How examples work

Every code listing is a complete, runnable expression or file. Nothing is an incomplete fragment left for you to guess.

```
# default.nix
let
  message = "reproducible desk online";
in
message
```

```
nix-instantiate --eval default.nix
```

"reproducible desk online"

A recurring scenario — a small team infrastructure desk (workstations, devShells, microservices, deployment pipelines) — connects the exercises so each technique solves a practical engineering challenge.

Map of the book

Part	Folder	What you leave with
0	00-introduction	The cost of non-reproducible systems, why imperative setups drift, and the immutable alternative
1	01-nix-fundamentals	The <code>/nix/store</code> layout, cryptographic hashes, dependency DAGs, garbage collection, and raw derivations

Part	Folder	What you leave with
2	02-nix-language	Primitive types, <code>let ... in</code> bindings, lambda functions, attribute sets, lists, and modular imports
3	03-dev-environments	Instant project toolchains with <code>mkShell</code> , multi-language runtimes, Flakes, Home Manager, and replacing heavy local containers
4	04-packages-and-overlays	The package registry, managing pinned Flake inputs, patching packages with overlays, and project dependency trees
5	05-building-from-source	Packaging software from source: C/CMake, Rust Cargo, Python, Go, Node.js, and multi-architecture cross-compiles
6	06-nixos-system	The NixOS module system, declarative machine definitions, systemd services, firewalls, users, and hardware profiles
7	07-home-manager	User-space configuration: declarative dotfiles, shell environments, editor configs, and unifying user space with NixOS
8	08-cicd	Continuous integration that builds once and caches everywhere: binary caches, GitHub Actions, GitLab CI, and secrets
9	09-containers-k8s	Minimal rootless container images with <code>dockerTools</code> , declarative Kubernetes manifests, Helm generation, and GitOps workflows
10	10-iac	Declarative cloud infrastructure: managing Terraform and OpenTofu providers, Ansible provisioning, and multi-cloud patterns

Part	Folder	What you leave with
11	11-secrets-security	Safe secret distribution in declarative systems: SOPS-nix, Age keys, HashiCorp Vault, and OS security hardening
12	12-platform-engineering	Internal developer platforms, monorepo dependency graphs, fleet upgrades, build tuning, and production checklists
13	13-production-capstone	Full production cluster lab: Disko storage, ephemeral root, WireGuard mesh, microservices, ACME TLS, and automated fleet rollback
99	99-appendices	Glossary of terms and command cheat sheet

What this book is not

- Not an exhaustive reference manual of all 100,000+ packages in **nixpkgs**.
- Not an exercise in esoteric functional programming puzzles.
- Not a guide to maintaining legacy channels and unpinned channels.

It is a concrete guide to writing reproducible systems that remain easy to understand, maintain, and operate.

Introduction

The Reproducibility Crisis

The Reproducibility Crisis

Software engineering has spent decades tolerating ambient state: hidden dependencies, global machine mutations, and “works on my machine” excuses. The boring default of this book is: **every development environment, dependency, and production system must be a pure, declarative, reproducible expression from zero.**

Mental model

Traditional operations rely on imperative mutation. You provision a virtual machine or container, run a sequence of `apt-get`, `curl | bash`, and `pip install` commands, and hope that: 1. Upstream package repositories haven’t updated or removed files. 2. Network connections remain uninterrupted mid-install. 3. System state matches the exact sequence of commands previously tested.

When any of these fail, you get **Heisenbugs**: intermittent build failures, subtle ABI mismatches, and production outages that disappear when you attempt to isolate them.

Traditional Imperative Model:

```
Base OS -> [Step 1: apt-get] -> [Step 2: pip install] -> [Step 3: git clone] -> ??? (Unknown)
```

Nix Declarative Model:

```
Inputs (Source + Hashes) -> Pure Derivation -> Immutable /nix/store Path (Deterministic)
```

Nix replaces stateful mutation with pure mathematical evaluation: given the exact same source code and inputs, it always yields the exact same byte-for-byte system closure.

Worked examples

Case 1: Detecting Ambient Host Contamination

Save as `test_environment.sh`. Inspect how standard shell scripts unknowingly depend on host binaries and paths.

```
# test_environment.sh
#!/usr/bin/env bash
set -euo pipefail

echo "Inspecting system path dependencies:"
which python3 || echo "python3 missing"
```

```
which node || echo "node missing"
which gcc || echo "gcc missing"
```

Run:

```
bash test_environment.sh
```

Output (on a typical developer workstation):

```
Inspecting system path dependencies:
/usr/bin/python3
/home/user/.nvm/versions/node/v20.0.0/bin/node
/usr/bin/gcc
```

Notice how three completely different tools live in three different unpinned locations (`/usr/bin`, user home directories, global OS paths). If a colleague runs the exact same script on a fresh laptop, it fails immediately.

Case 2: Purity and Isolated Evaluation

Save as `pure_check.nix`. A minimal Nix expression evaluated in isolation.

```
# pure_check.nix
let
  system = builtins.currentSystem;
  message = "No ambient host contamination allowed";
in
{
  inherit system message;
}
```

Run:

```
nix-instantiate --eval pure_check.nix
```

Output:

```
{ message = "No ambient host contamination allowed"; system = "x86_64-linux"; }
```

Evaluating this expression does not query the internet, look at `/usr/local`, or depend on environment variables.

Case 3: Hermetic Sandboxing in Action

Nix builds run in sandboxed namespaces where network access and non-declared filesystem paths are inaccessible.

Save as `sandbox_check.nix`:

```
# sandbox_check.nix
derivation {
  name = "sandbox-proof";
  system = builtins.currentSystem;
  builder = "/bin/sh";
  args = [
    "-c"
    "echo 'Built inside isolated namespace' > $out"
  ];
}
```

Run:

```
nix-build sandbox_check.nix
```

Output:

```
these 1 derivations will be built:
  /nix/store/10h9v...-sandbox-proof.drv
building '/nix/store/10h9v...-sandbox-proof.drv'...
/nix/store/v73ka...-sandbox-proof
```

Inspect output:

```
cat result
```

Output:

```
Built inside isolated namespace
```

The output was generated purely from the declared inputs.

The trap

Relying on “Golden Images” (pre-baked AMIs, Docker images without lockfiles, or manual server setup steps documented in internal wikis).

Wiki page: "Run these 14 commands to set up the desk service..."

By week three, one PPA repository goes offline, a patch release bumps a minor dependency, and onboarding is broken for every new hire.

The fix: Check in code that defines the environment completely down to cryptographic hashes. If it builds once, it builds forever.

The boring rule

- Never mutate system state in-place with ad-hoc shell commands.
- Pin all toolchain and library dependencies via cryptographic hashes.
- Eliminate dependency on the host's ambient `/usr/bin` and `$PATH`.
- Every environment must be reproducible from a fresh Git checkout.

Try this

1. Run `env` in your current shell and note how many variables point to user home directories or mutable local paths.
2. In `pure_check.nix`, add a field `timestamp = builtins.currentTime`; and observe how pure evaluation restricts non-deterministic built-ins.

Nix Fundamentals

Store Model

The `/nix/store` Anatomy

The `/nix/store` Anatomy

The Nix store is the foundation of reproducible systems. Every entry follows a strict naming convention:

```
/nix/store/<hash>-<name>-<version>
```

For example:

```
/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0
```

1. **The Hash (32 characters base32):** Computed from the cryptographic closure of all build inputs.
2. **Package Identifier:** A human-readable name and version tag.
3. **No Collisions:** Multiple versions of the same tool coexist on the same disk without collision.

Worked examples

Examine a store path directly in your shell.

Run:

```
ls -ld $(which nix)
```

Output:

```
-r-xr-xr-x 1 root root 15728640 Jan 1 1970 /nix/store/7vqw9388...-nix-2.24.1/bin/nix
```

Notice: - Permissions are `-r-xr-xr-x` (read and execute only; no write permission). - The timestamp is standardized to `Jan 1 1970` (Unix epoch) to preserve bit-for-bit build reproducibility across time zones.

The boring rule

- All binaries and libraries live strictly under `/nix/store`.
- Never manually move or rename files inside the store.

Content-Addressable Paths

Content-Addressable Paths

Historically, Nix paths were input-addressed (the hash was computed from inputs). Content-addressable paths hash the final built artifact itself.

Mental model

1. **Input-Addressed:** If a comment in a build script changes, the input hash changes, forcing a rebuild even if the resulting binary is identical.
2. **Content-Addressed (CA derivations):** Nix verifies the cryptographic hash of the *output*. If two independent derivations produce the exact same binary, they share the exact same store path.

Source A	Compiles	Binary (Hash: X)	/nix/store/X-app
Source B	Compiles	Binary (Hash: X)	Reuses /nix/store/X-app!

This dramatically improves cache deduplication in large build farms.

Worked examples

Check path immutability:

Run:

```
touch $(which nix) 2>&1 || true
```

Output:

```
touch: cannot touch '/nix/store/...-nix-2.24.1/bin/nix': Permission denied
```

Store paths cannot be modified in place. Rebuilding with different contents produces a completely distinct hash.

The boring rule

- Let Nix manage content addressing automatically.
- Never attempt to force write access into the store.

Hashes and Dependencies

Cryptographic Hashes and Inputs

Cryptographic Hashes and Inputs

How does Nix guarantee that two machines compile the exact same binary? The boring default is: **every input (source code, compiler version, patches, build script) is hashed, and any single-bit difference changes the output path.**

Mental model

Traditional package managers assume version numbers like `v1.4.0` imply identity. In reality, a package compiled against `glibc 2.38` behaves differently than one compiled against `glibc 2.39`.

Nix hashes the full input specification (the `.drv` file). If a compiler flag changes from `-O2` to `-O3`, the hash changes, generating a separate, isolated store path.

Worked examples

Check input hashes in derivation files:

Run:

```
nix-instantiate -E 'derivation { name = "sample"; builder = "/bin/sh"; system = builtins.cur
```

Output:

```
/nix/store/7vqw...-sample.drv
```

The derivation hash is calculated before a single line of compilation occurs.

The boring rule

- Every dependency is pinned by its cryptographic hash.
- Upgrading a dependency creates a new path without mutating the existing one.

Deterministic Dynamic Linking and RPATH

Deterministic Dynamic Linking and RPATH

Dynamic linking in traditional Linux is based on file names like `libc.so.6`. If that file is updated globally, every application on the host changes behavior simultaneously. The boring default of Nix is: **embed the exact cryptographic hash of every required dependency into the binary's runtime header (ELF RPATH)**.

Mental model

When Nix compiles a binary, it hardcodes the exact store path into the ELF header:

```
ELF Header (RUNPATH):  
  /nix/store/7r44p...-glibc-2.39/lib
```

The dynamic linker looks strictly in that directory, ignoring `/lib` and `/usr/lib`.

Worked examples

Check the runtime path of an installed binary:

```
readelf -d $(which rg) | grep -E '(RPATH|RUNPATH)'
```

Output:

```
0x000000000000001d (RUNPATH)          Library runpath: [/nix/store/7r44p12l4b72j9m9l09k96ygh
```

The boring rule

- Binaries embed explicit store paths via RUNPATH.
- Never set global `LD_LIBRARY_PATH` variables.

Dependency Graphs

Dependency Closures and Graphs

Dependency Closures and Graphs

Every software artifact in Nix is a node in an immutable directed acyclic graph (DAG). The boring default is: **distinguish build closures from runtime closures, and use `nix-store -q --requisites` to verify production deployment size.**

Mental model

1. **Build-Time Closure:** Compilers, header files, and tools needed to produce the binary.
2. **Runtime Closure:** Only the files directly or transitively referenced by the finished binary.
3. **Reference Scanning:** Nix scans output files for string occurrences of store hashes. Unreferenced build tools are stripped from the runtime closure automatically.

Worked examples

Inspect the runtime closure of an executable:

Run:

```
nix-store -q --requisites $(which rg)
```

Output:

```
/nix/store/7r44p12l4b72j9m9l09k96yghz02f8n7-glibc-2.39  
/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0
```

Only these two store paths are required to run `rg`.

The boring rule

- Query runtime closures with `nix-store -q --requisites`.
- Trust reference scanning to drop build-time dependencies automatically.

The Nix Database (db.sqlite)

The Nix Database (db.sqlite)

How does Nix know which store paths are valid and which packages refer to each other? The boring default is: **Nix tracks all store registrations and reference relationships in a local SQLite database at `/nix/var/nix/db/db.sqlite`.**

Mental model

The Nix store database records: 1. **Valid Paths:** Every store path successfully registered by a build or substitute. 2. **References Graph:** Direct references from one store path to another. 3. **Derivers:** Which `.drv` file produced a given store path.

```
/nix/store/  
    File tree (immutable files)  
    /nix/var/nix/db/db.sqlite (registration & reference index)
```

Worked examples

Query referrers (which packages depend on a path):

```
nix-store -q --referrers $(which rg)
```

Output:

```
/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0
```

Because nothing else references this specific ripgrep build, removing its profile symlink allows it to be garbage collected.

The boring rule

- Never manually edit or lock `/nix/var/nix/db/db.sqlite`.
- Use `nix-store --verify --check-contents` if you ever suspect filesystem corruption.

Garbage Collection

GC Roots and Space Reclamation

GC Roots and Space Reclamation

Because Nix never overwrites existing packages, disk usage in `/nix/store` naturally grows over time. The boring default is: **use garbage collector roots (gcroots) to preserve active environments, and run `nix-collect-garbage` to clean unreferenced paths.**

Mental model

1. **GC Roots:** Pointers that tell the garbage collector “do not delete this path”.
2. **Roots Locations:**
 - System generations (`/nix/var/nix/profiles/system-link`)
 - User profile generations (`~/.nix-profile`)
 - Local build symlinks (`result` registered in `/nix/var/nix/gcroots/auto`)
3. **Dead Paths:** Any store path with zero references from any GC root is eligible for deletion.

Worked examples

Run garbage collection safely:

```
nix-collect-garbage
```

Output:

```
finding garbage collector roots...
deleting garbage...
14 store paths deleted, 142.50 MiB freed
```

This cleans unreferenced, orphaned files while keeping rollback generations intact.

The boring rule

- Always delete packages via `nix-collect-garbage`, never with manual `rm`.
- Use `nix-collect-garbage -d` to delete older generations and reclaim space.

Store Optimization and Deduplication

Store Optimization and Deduplication

When multiple package versions share identical files (like licenses, shared headers, or static assets), duplicate copies waste disk space. The boring default is: **use hard-link deduplication via `nix-store --optimise` to replace duplicate files with shared physical inodes.**

Mental model

Unlike traditional filesystems where identical files across packages consume double the space: - Nix verifies SHA-256 hashes of individual files. - When identical content is found, it links both paths to the same disk inode. - The files remain independent and read-only.

```
/nix/store/pkgA/lib/data.bin (Inode 1001)
                                     Shared Physical Disk Block
/nix/store/pkgB/lib/data.bin (Inode 1001)
```

Worked examples

Run store optimization:

```
nix-store --optimise
```

Output:

```
hashing files in /nix/store...
31420 files linked, 850.40 MiB freed
```

Gigabytes of redundant disk space are reclaimed without modifying or rebuilding packages.

The boring rule

- Run `nix-store --optimise` periodically or enable `nix.optimise.automatic = true;` in NixOS.
- Deduplication is safe because the store is read-only.

First Expressions

The Derivation Primitive

The Derivation Primitive

At the lowest level, Nix builds packages using the built-in function `derivation`.

A derivation requires three mandatory attributes: 1. `name`: A string identifier. 2. `system`: The target architecture string ("x86_64-linux", "aarch64-darwin"). 3. `builder`: The path to the executable executing the build (usually `/bin/sh`).

Worked examples

Save as `minimal.nix`:

```
# minimal.nix
derivation {
  name = "desk-banner";
  system = builtins.currentSystem;
  builder = "/bin/sh";
  args = [
    "-c"
    "echo 'Desk System Online' > $out"
  ];
}
```

Run:

```
nix-build minimal.nix
```

Output:

```
these 1 derivations will be built:
  /nix/store/7vqw93...-desk-banner.drv
building '/nix/store/7vqw93...-desk-banner.drv'...
/nix/store/3la841...-desk-banner
```

`$out` is the destination path allocated in `/nix/store`.

The boring rule

- A derivation is a pure function evaluated into a `.drv` file and realized as a store path.

Building Multi-File Artifacts

Building Multi-File Artifacts

Derivations can produce complete directory hierarchies with executables, configuration files, and documentation.

Mental model

The `$out` variable provided by the builder does not have to be a single file. You can create `$out/bin`, `$out/lib`, and `$out/share` to package complete applications.

Worked examples

Save as `multifile.nix`:

```
# multifile.nix
derivation {
  name = "desk-assets";
  system = builtins.currentSystem;
  builder = "/bin/sh";
  args = [
    "-c"
    ''
      mkdir -p $out/bin $out/share
      echo '#!/bin/sh' > $out/bin/desk-status
      echo 'echo status: ready' >> $out/bin/desk-status
      chmod +x $out/bin/desk-status
      echo 'Desk version 1.0' > $out/share/version.txt
    ''
  ];
}
```

Run:

```
nix-build multifile.nix
```

Output:

```
/nix/store/99ka...-desk-assets
```

Test the generated script:

```
./result/bin/desk-status
```

Output:

```
status: ready
```

The boring rule

- Always create the target directories (`mkdir -p $out/bin`) before writing outputs.
- Test outputs locally by running `./result/bin/....`

Nix Language

Nix Expressions and Evaluation

Nix Expressions and Evaluation

Nix does not execute a sequential list of statements; it evaluates a single tree of pure expressions. The boring default is: **every file is a single expression yielding a value, pure evaluation produces identical results regardless of when or where it is run, and lazy evaluation computes only what is accessed.**

Mental model

In imperative scripting languages (Python, Bash), code executes step-by-step from top to bottom, altering environment variables and writing to global state.

In Nix: 1. **Pure Expression Model:** There are no statements, no variable mutations, and no global side effects. $2 + 2$ is an expression; `{ a = 1; }` is an expression; a 500-line package definition is a single expression. 2. **Deterministic Evaluation:** Given identical inputs, evaluating an expression will always yield the exact same AST and data structure. 3. **Lazy Evaluation:** Nix expressions are evaluated only when their values are needed for a build or terminal output. If you define 50 attribute keys but only query `myConfig.port`, the other 49 expressions are never computed.

Imperative (Bash/Python):

Step 1 -> Mutate state -> Step 2 -> Mutate state -> Final state (Stateful)

Pure Expression (Nix):

Root Expression -> Evaluates sub-expressions lazily -> Computed Value (Deterministic)

Worked examples

Case 1: Evaluating Expressions with `nix eval`

Evaluate expressions directly from the command line using `nix eval --expr`.

Run:

```
nix eval --expr '1 + 2 * 3'
```

Output:

7

Arithmetic follows standard precedence, evaluated as a pure expression.

Case 2: Multi-Expression File Structure

Save as `root_expression.nix`. A file is always a single overarching expression.

```
# root_expression.nix
let
  deskId = "terminal-01";
  status = "online";
in
{
  id = deskId;
  state = status;
  ready = true;
}
```

Run:

```
nix-instantiate --eval --strict root_expression.nix
```

Output:

```
{ id = "terminal-01"; ready = true; state = "online"; }
```

The entire file is a `let ... in` expression evaluating to an attribute set.

Case 3: Proving Lazy Evaluation

Save as `laziness_proof.nix`. Expressions that would cause errors or take heavy computation are skipped if unused.

```
# laziness_proof.nix
let
  validField = "desk-service";
  brokenField = throw "This error is never triggered!";
in
{
  service = validField;
}
```

Run:

```
nix-instantiate --eval --strict laziness_proof.nix
```

Output:

```
{ service = "desk-service"; }
```

Even though `brokenField` calls `throw`, evaluation succeeds with zero errors because `brokenField` was never queried.

The trap

Expecting side effects like `echo` or file writes to occur during Nix evaluation. Nix expressions evaluate purely in memory; the only file writes occur during the sandboxed build phase of derivations.

The fix: Use Nix expressions to generate pure data and configuration structures; let builders and systemd modules perform physical execution.

The boring rule

- Every `.nix` file must evaluate to exactly one expression.
- Rely on laziness: you can declare large configurations without paying an evaluation penalty for unused branches.
- Use `nix eval --expr` for quick scratchpad calculations and syntax experiments.

Try this

1. Evaluate an attribute selection on `laziness_proof.nix` that forces `brokenField` to be read (`nix eval --file laziness_proof.nix brokenField`) and observe the `throw` error.

Values and Immutability

Values and Immutability

In Nix, values cannot be altered once constructed. The boring default is: **understand that all data in Nix is deeply immutable, transformations always yield fresh values, and lexical scopes eliminate state corruption.**

Mental model

In traditional languages, mutability creates complex bugs:

```
# Python mutable bug:
config = {"port": 8080}
mutate_config(config) # modifies original dictionary in place!
```

In Nix: 1. **Values are Immutable:** Once an attribute set, list, or string is defined, it can never be modified. 2. **Transformations are Pure:** Merging an attribute set with // or adding items to a list with ++ does not modify the original data structure; it allocates a new value representing the result. 3. **Thread Safety & Cacheability:** Because values cannot change under your feet, the Nix evaluator and binary caches can safely memoize and reuse values indefinitely.

Original Value ({ a = 1; })

New Result ({ a = 1; b = 2; })

Update ({ b = 2; })

(Original remains unchanged: { a = 1; })

Worked examples

Case 1: Proving Immutability Across Merges

Save as immutable_merge.nix.

```
# immutable_merge.nix
let
  baseDesk = {
    location = "building-a";
    tier = "standard";
  };

  # Create an updated copy with an override
```

```

    upgradedDesk = baseDesk // { tier = "executive"; desksCount = 12; };
in
{
    # Original remains completely untouched
    original = baseDesk;
    # Upgraded is a separate immutable value
    upgraded = upgradedDesk;
}

```

Run:

```
nix-instantiate --eval --strict immutable_merge.nix
```

Output:

```
{ original = { location = "building-a"; tier = "standard"; }; upgraded = { desksCount = 12; loc
```

`baseDesk` did not change. You can pass `baseDesk` to fifty different functions with total confidence that none of them will mutate its keys.

Case 2: Transforming Lists Immutably

Save as `immutable_list.nix`.

```

# immutable_list.nix
let
    activeTables = [ 1 2 3 ];
    extendedTables = activeTables ++ [ 4 5 ];
in
{
    initial = activeTables;
    result = extendedTables;
}

```

Run:

```
nix-instantiate --eval --strict immutable_list.nix
```

Output:

```
{ initial = [ 1 2 3 ]; result = [ 1 2 3 4 5 ]; }
```

Concatenation creates a new list without touching `activeTables`.

The trap

Trying to assign to a previously bound name:

```
let
  x = 1;
  x = 2; # Syntax error!
in
x
```

Nix disallows multiple bindings of the same name in the same scope.

The fix: Use distinct descriptive names (`initialConfig`, `productionConfig`) or use child `let` blocks for local shadowing.

The boring rule

- Every data structure in Nix is immutable.
- Merging attribute sets (`//`) and lists (`++`) always yields new values.
- Immutability guarantees zero race conditions and perfect caching.

Try this

1. Verify in `nix repl` that `let a = { x = 1; }; b = a // { x = 2; }; in a.x` evaluates to 1.

Data Types and String Interpolation

Data Types and String Interpolation

Nix is dynamically typed with strict type checking at runtime. The boring default is: **understand primitive types (integers, floats, booleans, strings, paths, and null) and master `${...}` string interpolation for safe configuration generation.**

Mental model

Nix has six primitive types: 1. **Integers (int):** 64-bit signed integers (42, -10). 2. **Floats (float):** IEEE 754 floating-point numbers (3.1415). 3. **Booleans (bool):** true or false. 4. **Strings (string):** UTF-8 encoded text ("hello" or multi-line 'hello'). 5. **Paths (path):** File paths beginning with /, ./, or ~/ (./config.nix). 6. **Null (null):** Represents the intentional absence of a value.

String interpolation `${expression}` embeds any expression that evaluates to a string, number, or path directly into a string.

```
"Server: ${host}:${toString port}"
```

When a path is interpolated into a string inside a derivation, Nix copies that file into `/nix/store` and substitutes its cryptographic store path.

Worked examples

Case 1: Checking and Inspecting Types

Save as `types_check.nix`. Inspect the type of each primitive using `builtins.typeOf`.

```
# types_check.nix
{
  intType = builtins.typeOf 100;
  floatType = builtins.typeOf 3.5;
  boolType = builtins.typeOf true;
  stringType = builtins.typeOf "desk";
  pathType = builtins.typeOf ./types_check.nix;
  nullType = builtins.typeOf null;
}
```

Run:

```
nix-instantiate --eval --strict types_check.nix
```

Output:

```
{ boolType = "bool"; floatType = "float"; intType = "int"; nullType = "null"; pathType = "path"; }
```

Every value in Nix has a queryable type.

Case 2: String Interpolation and Multi-Line Blocks

Save as `interpolation.nix`.

```
# interpolation.nix
let
  appName = "desk-agent";
  port = 9090;
  logDir = "/var/log/desk";

  # Single-line interpolation
  endpoint = "http://127.0.0.1:${toString port}";

  # Multi-line string with indentation stripping
  systemdService = ''
    [Unit]
    Description=${appName} Service

    [Service]
    ExecStart=${endpoint}/start
    StandardOutput=append:${logDir}/output.log
  '';
in
{
  inherit endpoint systemdService;
}
```

Run:

```
nix-instantiate --eval --strict interpolation.nix
```

Output:

```
{ endpoint = "http://127.0.0.1:9090"; systemdService = "[Unit]\nDescription=desk-agent Service\n"; }
```

Notice that `toString` converts the integer `port` into a string suitable for interpolation.

Case 3: Path Interpolation into the Nix Store

When you interpolate an unquoted path `${./file.txt}`, Nix treats it as a source asset and copies it into `/nix/store`.

Run:

```
nix eval --expr 'let f = ./types_check.nix; in "${f}"'
```

Output:

```
"/nix/store/...-types_check.nix"
```

The file is hashed and imported directly into the store.

The trap

Interpolating non-string values like integers without converting them first. Writing `"${42}"` in raw Nix fails because integers cannot be coerced to strings implicitly.

The fix: Always wrap numbers with `toString`: `"${toString port}"`.

The boring rule

- Convert integers to strings with `toString` before interpolating.
- Use unquoted paths (`./config.json`) when you want Nix to track and store the file.
- Use double single quotes (`' '`) for multi-line bash scripts and systemd units.

Try this

1. In `nix repl`, test `builtins.typeOf (toString 42)` and verify it returns `"string"`.

Let-In Blocks and Local Bindings

Let-In Blocks and Local Bindings

Nix does not have mutable assignment statements. The boring default is: **use `let ... in ...` to introduce clear, locally-scoped, immutable bindings that make complex expressions readable.**

Mental model

The `let ... in` construct has two sections: - **let**: A block of bindings in the form `name = expression`; - **in**: The expression that is evaluated, in which those bindings are accessible.

Bindings inside a `let` block are mutually recursive by default: a binding can reference any other binding in the same block, regardless of line order.

```
let
  b = a + 10;
  a = 5;
in
b # evaluates to 15
```

Evaluation order is driven by dependency need, not textual sequence.

Worked examples

Case 1: Structuring Component Configurations

Save as `desk_service.nix`.

```
# desk_service.nix
let
  domain = "internal.desk";
  port = 443;
  protocol = "https";
  serviceUrl = "${protocol}://${domain}:${toString port}";
in
{
  inherit domain port;
  url = serviceUrl;
}
```

Run:

```
nix-instantiate --eval --strict desk_service.nix
```

Output:

```
{ domain = "internal.desk"; port = 443; url = "https://internal.desk:443"; }
```

```
inherit domain port; is shorthand for domain = domain; port = port;.
```

Case 2: Lexical Scope and Shadowing

Save as `scope.nix`. Inner `let` blocks can shadow outer bindings cleanly.

```
# scope.nix
let
  tier = "standard";
in
{
  outerTier = tier;
  overridden =
    let
      tier = "premium";
    in
      tier;
}
```

Run:

```
nix-instantiate --eval --strict scope.nix
```

Output:

```
{ outerTier = "standard"; overridden = "premium"; }
```

Shadowing is local to the expression following `in`.

The trap

Creating massive 200-line `let` blocks containing unrelated helpers. When a `let` block grows too large, readability degrades.

The fix: Group related helpers into separate `.nix` files and bring them in with `import`.

The boring rule

- Use `let ... in` to name intermediate calculations and keep expressions clean.
- Use `inherit` to avoid redundant `key = key;` syntax.
- Remember bindings are evaluated lazily: unused `let` bindings incur zero runtime cost.

Try this

1. In `desk_service.nix`, add an unused binding `unused = throw "never evaluated";` and verify that evaluation still succeeds because of laziness.

Functions and Argument Defaults

Functions and Argument Defaults

Functions in Nix are first-class values that take exactly one argument and return a value. The boring default is: **accept a single attribute set with explicit destructuring, supply sane default values with `?`, and use `...` only when wrapping extensible APIs.**

Mental model

Nix function syntax uses a colon `:` separating the argument pattern from the function body:

```
x: x + 1
```

For functions requiring multiple parameters, there are two patterns: 1. **Currying:** `x: y: x + y`
2. **Attribute Set Destructuring (The standard):** `{ name, port ? 8080, ... }: ...`

Destructuring allows named parameters in any order, optional defaults (`? default`), and extra attribute tolerance (`...`).

Worked examples

Case 1: Curried Functions vs Destructured Sets

Save as `functions_demo.nix`.

```
# functions_demo.nix
let
  # Curried
  add = x: y: x + y;

  # Destructured attribute set
  formatServer = { host, port ? 80 }: "${host}:${toString port}";
in
{
  sum = add 10 25;
  defaultPort = formatServer { host = "api.desk"; };
  customPort = formatServer { host = "api.desk"; port = 443; };
}
```

Run:

```
nix-instantiate --eval --strict functions_demo.nix
```

Output:

```
{ customPort = "api.desk:443"; defaultPort = "api.desk:80"; sum = 35; }
```

Destructured attribute sets read like named arguments in modern languages.

Case 2: Capturing Full Sets with the @ Pattern

Save as `capture.nix`. Access both individual fields and the complete input set.

```
# capture.nix
let
  makeProfile = args@{ user, role ? "member", ... }: {
    username = user;
    userRole = role;
    rawInput = args;
  };
in
makeProfile { user = "alice"; team = "desk-ops"; }
```

Run:

```
nix-instantiate --eval --strict capture.nix
```

Output:

```
{ rawInput = { team = "desk-ops"; user = "alice"; }; userRole = "member"; username = "alice"; }
```

`args@` binds the whole attribute set while `{ user, ... }` extracts specific keys.

The trap

Omitting `...` when a function receives configuration from external modules. If an unexpected attribute is passed without `...`, evaluation halts immediately with an “unexpected argument” error.

The fix: Use `...` at the end of parameter sets for open/extensible interfaces (like package and module declarations).

The boring rule

- Always prefer attribute set destructuring (`{ pkgs, lib, ... }:`) over curried functions for configuration code.
- Provide sensible defaults with `? default`.
- Use `inherit` inside functions to assemble result sets cleanly.

Try this

1. In `formatServer`, add an optional boolean `secure ? false` and output `https://` if true, `http://` if false.

Attribute Sets and Recursive Definitions

Attribute Sets and Recursive Definitions

Attribute sets are the workhorse data structure of Nix. The boring default is: **use standard attribute sets, merge them with //, access fields with dot-notation (a.b.c), and reserve rec for simple, acyclic sibling references.**

Mental model

An attribute set is a collection of key-value pairs wrapped in braces:

```
{ key1 = "value1"; key2 = 42; }
```

Key operations: 1. **Dot Access:** set.key1 2. **Fallback Operator (or):** set.missingKey or "default" 3. **Merge Operator (//):** { a = 1; b = 2; } // { b = 3; c = 4; } (right-hand side overwrites left) 4. **Nested Syntax:** desk.database.port = 5432; is equivalent to desk = { database = { port = 5432; }; }; 5. **Recursive Attribute Sets (rec):** Allows attributes within the set to reference other attributes in the same set.

Worked examples

Case 1: Merging and Fallback Operations

Save as attr_operations.nix.

```
# attr_operations.nix
let
  defaultConfig = {
    host = "127.0.0.1";
    port = 8080;
    workers = 4;
  };

  productionOverrides = {
    port = 443;
    workers = 16;
  };

  finalConfig = defaultConfig // productionOverrides;
in
```

```
{
  inherit finalConfig;
  customTimeout = finalConfig.timeout or 30;
}
```

Run:

```
nix-instantiate --eval --strict attr_operations.nix
```

Output:

```
{ customTimeout = 30; finalConfig = { host = "127.0.0.1"; port = 443; workers = 16; }; }
```

`productionOverrides` cleanly overrides specific keys while leaving `host` intact.

Case 2: Recursive Attribute Sets (rec)

Save as `rec_demo.nix`.

```
# rec_demo.nix
rec {
  domain = "desk.corp";
  authService = "https://auth.${domain}";
  billingService = "https://billing.${domain}";
  endpoints = [ authService billingService ];
}
```

Run:

```
nix-instantiate --eval --strict rec_demo.nix
```

Output:

```
{ authService = "https://auth.desk.corp"; billingService = "https://billing.desk.corp"; domain
```

Fields dynamically reference `domain` without needing an enclosing `let` block.

The trap

Using `rec` and then overriding a key with `//`. Overriding a field in a `rec` set does not automatically update other fields that referenced the original value.

```
# Anti-pattern: expecting // to update references in rec
let
```

```
base = rec { x = 1; y = x + 1; };  
updated = base // { x = 10; };  
in  
updated.y # Still evaluates to 2, not 11!
```

The fix: Use functions (`makeConfig = { x, ... }: ...`) or `let ... in` blocks when you need values to propagate after overrides.

The boring rule

- Use `a.b.c` or `default` for safe property access.
- Use `//` for shallow configuration overrides.
- Use `rec` only when the set is self-contained and will not be extended or overridden with `//`.

Try this

1. In `attr_operations.nix`, verify what happens if you access `finalConfig.invalid` without `or`.

Conditionals, Lists, and Built-in Operations

Conditionals, Lists, and Built-in Operations

Transforming collections and applying conditional flags are common when configuring systems. The boring default is: **use standard** `if ... then ... else ... expressions`, **concatenate lists** with `++`, and **leverage** `builtins.map` and `builtins.filter` for pure transformations.

Mental model

1. **Conditionals:** In Nix, `if` is an expression that yields a value, not a control-flow statement. The `else` clause is mandatory:

```
if condition then valueA else valueB
```

2. **Lists:** Space-separated elements wrapped in square brackets:

```
[ 1 "two" ./three (4 + 5) ]
```

3. **List Concatenation:** Using `++`:

```
[ 1 2 ] ++ [ 3 4 ] # evaluates to [ 1 2 3 4 ]
```

4. **Built-in Pure Operations:** Functions provided by the Nix runtime under `builtins`, including `builtins.map`, `builtins.filter`, `builtins.length`, and `builtins.elem`.

Worked examples

Case 1: Conditional Configuration Flags

Save as `conditional_config.nix`.

```
# conditional_config.nix
let
  isProduction = true;
  replicaCount = if isProduction then 5 else 1;
  logLevel = if isProduction then "warn" else "debug";
in
{
  replicas = replicaCount;
  logging = logLevel;
}
```

```
}
```

Run:

```
nix-instantiate --eval --strict conditional_config.nix
```

Output:

```
{ logging = "warn"; replicas = 5; }
```

Both branches must return valid expressions; the selected branch is evaluated lazily.

Case 2: Transforming and Filtering Lists

Save as `list_operations.nix`. Filter table numbers and map them into server records.

```
# list_operations.nix
let
  tableNumbers = [ 1 2 3 4 5 6 7 8 9 10 ];
  isPriority = n: n <= 3;
  priorityTables = builtins.filter isPriority tableNumbers;
  deskWorkers = builtins.map (n: { table = n; status = "active"; }) priorityTables;
in
deskWorkers
```

Run:

```
nix-instantiate --eval --strict list_operations.nix
```

Output:

```
[ { status = "active"; table = 1; } { status = "active"; table = 2; } { status = "active"; table = 3; }
```

`builtins.filter` and `builtins.map` provide straightforward functional data processing.

The trap

Forgetting the `else` clause in an `if` expression. Because Nix is purely functional, an `if` expression without an `else` has no defined fallback value and produces a syntax error.

The fix: Always supply both `then` and `else` branches. If you want an attribute to be conditionally omitted, use `lib.optionalAttrs` or `lib.mkIf`.

The boring rule

- Every `if` must have an `else`.
- Use `++` to combine lists.
- Rely on built-ins (`builtins.map`, `builtins.filter`) for standard collection processing.

Try this

1. In `list_operations.nix`, use `builtins.length` to calculate the count of priority tables.

Importing Files and Building Abstractions

Importing Files and Building Abstractions

Real systems cannot fit inside a single `.nix` file. The boring default is: **split logic into small, modular files, pass explicit arguments into them, and import them with the `import` operator.**

Mental model

In Nix, `import` is a built-in function that takes a path to a `.nix` file and evaluates it:

```
import ./constants.nix
```

If the imported file evaluates to a function, you pass arguments immediately:

```
import ./service.nix { inherit pkgs; port = 8080; }
```

There are no circular dependencies, hidden global namespaces, or implicit module registries. An imported file only has access to what is explicitly passed into it.

Worked examples

Case 1: Modular Service Definition

Save as `database.nix`. A reusable database module that takes an environment argument.

```
# database.nix
{ environment }:

let
  isProduction = environment == "production";
in
{
  dbHost = if isProduction then "db.desk.corp" else "127.0.0.1";
  dbPort = 5432;
  poolSize = if isProduction then 50 else 5;
}
```

Save as `app.nix`. Import `database.nix` and pass the required configuration.

```
# app.nix
let
  dbConfig = import ./database.nix { environment = "production"; };
in
{
  appName = "desk-billing";
  database = dbConfig;
}
```

Run:

```
nix-instantiate --eval --strict app.nix
```

Output:

```
{ appName = "desk-billing"; database = { dbHost = "db.desk.corp"; dbPort = 5432; poolSize = 50;
```

Clean, parameter-driven abstraction with zero global state.

Case 2: Standard Library Utilities (pkgs.lib)

Nixpkgs includes `lib`, an extensive library of functional helpers.

Save as `lib_demo.nix`:

```
# lib_demo.nix
let
  # Demonstrating lib helpers conceptually
  optional = condition: element: if condition then [ element ] else [];
  flatten = builtins.concatLists;

  features = flatten [
    [ "core-api" ]
    (optional true "metrics-exporter")
    (optional false "debug-profiler")
  ];
in
features
```

Run:

```
nix-instantiate --eval --strict lib_demo.nix
```

Output:

```
[ "core-api" "metrics-exporter" ]
```

Functions like `lib.optional` make conditional list assemblies clean and readable.

The trap

Importing files that rely on ambient path lookups like `<nixpkgs>` in production configurations. `<nixpkgs>` depends on the host machine's `$NIX_PATH` channel, reintroducing non-reproducibility across different machines.

The fix: Pin dependencies using Flakes or explicit pinned tarball URLs (`builtins.fetchTarball`).

The boring rule

- Structure large configurations as directories of focused `.nix` files.
- Each `.nix` file should return a function accepting its dependencies (`{ lib, config, ... }`).
- Keep file boundaries clear and avoid deeply nested abstractions.

Try this

1. Modify `database.nix` to support a "staging" environment that uses port 5433.

Dev Environments

Nix Shells and the devShell Concept

Nix Shells and the devShell Concept

Development environments rot when developers install tools directly onto their host machines. The boring default is: **define a `shell.nix` or `devShell` declaring the exact tools required, and activate it with `nix-shell` or `nix develop`.**

Mental model

A Nix devShell is an ephemeral execution environment: 1. It downloads or builds only the packages listed in `packages` or `buildInputs`. 2. It sets up `$PATH`, environment variables, and compiler flags in a child shell. 3. It does not alter your workstation's global system directories. 4. When you exit the shell (`exit` or `Ctrl+D`), your environment reverts instantly to normal.

Host OS (`$PATH = /usr/bin`)

```
nix-shell starts
  Injects /nix/store/abc...-go-1.27/bin into $PATH
  Runs shellHook
  Developer compiles and tests
```

```
exit drops back to pristine Host OS ($PATH = /usr/bin)
```

Worked examples

Case 1: Simple `shell.nix`

Save as `shell.nix`.

```
# shell.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  packages = [
    pkgs.ripgrep
    pkgs.jq
  ];

  shellHook = ''
    echo "=== Desk Shell Ready ==="
```

```
    '';  
}
```

Run:

```
nix-shell --run "echo Shell active"
```

Output:

```
=== Desk Shell Ready ===  
Shell active
```

The tools are available only within that session.

Case 2: Setting Environment Variables in a Shell

Save as `env_shell.nix`.

```
# env_shell.nix  
{ pkgs ? import <nixpkgs> {} }:  
  
pkgs.mkShell {  
  packages = [ pkgs.curl ];  
  
  env = {  
    DESK_API_URL = "https://api.desk.internal";  
    DESK_TIMEOUT = "30";  
  };  
  
  shellHook = ''  
    echo "Targeting API: $DESK_API_URL"  
    '';  
}
```

Run:

```
nix-shell env_shell.nix --run "echo Timeout: \${DESK_TIMEOUT}"
```

Output:

```
Targeting API: https://api.desk.internal  
Timeout: 30
```

Environment variables are isolated and declared in code.

The trap

Installing development compilers like Go, Rust, or Node globally with system package managers (`apt`, `brew`). Different projects inevitably require different versions, leading to version conflicts.

The fix: Use a `shell.nix` in each repository.

The boring rule

- Every project repository must have a root `shell.nix` or `flake.nix`.
- Use `pkgs.mkShell` with `packages` for modern shells.
- Use `shellHook` for local environment configuration and diagnostics.

Try this

1. Add `pkgs.tree` to `shell.nix` and run `nix-shell --run "tree -L 1"`.

Multi-Language Development Environments

Multi-Language Development Environments

Modern systems frequently combine multiple languages: a Go backend, a TypeScript frontend, Python data scripts, and C shared libraries. The boring default is: **declare all runtimes, compilers, and library headers inside a single unified `mkShell` definition.**

Mental model

In traditional systems, managing polyglot stacks requires coordinating multiple independent version managers (`asdf`, `nvm`, `pyenv`, `rustup`) and installing system development packages (`libssl-dev`, `libpq-dev`).

Nix unifies this: - Compilers (`go`, `rustc`, `nodejs`) are just packages. - C libraries (`openssl`, `postgresql`) provide headers and shared objects automatically. - `pkg-config` is wired up automatically when included in `nativeBuildInputs`.

Worked examples

Case 1: Unified Polyglot Stack

Save as `polyglot.nix`.

```
# polyglot.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  packages = [
    pkgs.go
    pkgs.nodejs_22
    (pkgs.python3.withPackages (ps: [ ps.requests ps.pydantic ]))
    pkgs.sqlite
  ];

  shellHook = ''
    echo "Polyglot environment loaded: Go, Node, Python, SQLite"
  '';
}
```

Run:

```
nix-shell polyglot.nix --run "go version && node --version && python3 --version"
```

Output:

```
Polyglot environment loaded: Go, Node, Python, SQLite
go version go1.27.1 linux/amd64
v22.11.0
Python 3.12.7
```

All runtimes coexist without path or library collision.

Case 2: C Libraries and pkg-config

Save as `native_c.nix`. A shell that configures C headers and libraries for applications that compile native extensions.

```
# native_c.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  nativeBuildInputs = [ pkgs.pkg-config ];
  buildInputs = [ pkgs.openssl pkgs.zlib ];

  shellHook = ''
    echo "OpenSSL cflags: $(pkg-config --cflags openssl)"
  '';
}
```

Run:

```
nix-shell native_c.nix --run "echo C libraries configured"
```

Output:

```
OpenSSL cflags: -I/nix/store/...-openssl-3.3.2-dev/include
C libraries configured
```

`pkg-config` points directly to the isolated store header directories.

The trap

Relying on host system header files like `/usr/include/openssl`. If a teammate is on a different Linux distribution or macOS, paths differ and builds fail.

The fix: Always include header and library dependencies explicitly in `buildInputs`.

The boring rule

- Group build-time tools (like `pkg-config`, `cmake`) in `nativeBuildInputs`.
- Group runtime libraries and headers (like `openssl`, `zlib`) in `buildInputs`.
- Use language-specific package wrappers (like `python3.withPackages`).

Try this

1. In `polyglot.nix`, add `pkgs.cargo` and test running `cargo --version`.

Flakes for Reproducible Dev Environments

Flakes for Reproducible Dev Environments

Classic `shell.nix` still relies on `<nixpkgs>`, which can vary across developer machines depending on their channel state. The boring default is: **use modern Nix Flakes (`flake.nix` + `flake.lock`) to pin the exact commit of `nixpkgs` across the entire team.**

Mental model

Nix Flakes introduce three critical improvements: 1. **Pinned Lockfiles (`flake.lock`):** Automatically records the exact Git commit hash of every dependency. 2. **Standard Output Schema:** Predictable structure (`packages.${system}`, `devShells.${system}.default`). 3. **Pure Evaluation:** Flakes disallow reading undeclared files, ambient environment variables, or mutable network channels.

`flake.nix` (declares inputs & outputs)

`flake.lock` (records exact Git commit of `nixpkgs`)

`nix develop` (creates identical shell on all machines)

Worked examples

Case 1: Minimal Flake Development Shell

Save as `flake.nix`.

```
# flake.nix
{
  description = "Desk management service development shell";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
  };

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
```

```

    pkgs = import nixpkgs { inherit system; };
  in
  {
    devShells.${system}.default = pkgs.mkShell {
      packages = [
        pkgs.go
        pkgs.gopls
        pkgs.golangci-lint
      ];

      shellHook = ''
        echo "=== Pinned Go 1.27 Dev Environment ==="
      '';
    };
  };
}

```

Run:

```
nix develop --command go version
```

Output:

```

=== Pinned Go 1.27 Dev Environment ===
go version go1.27.1 linux/amd64

```

The exact commit of nixpkgs is frozen into flake.lock.

Case 2: Updating Dependencies

When you are ready to update packages across the team, update the lockfile explicitly:

Run:

```
nix flake update
```

Output:

```

updating lock file '/home/user/project/flake.lock':
• Updated input 'nixpkgs':
  'github:NixOS/nixpkgs/...' (2026-09-01)
→ 'github:NixOS/nixpkgs/...' (2026-09-08)

```

Updates are an intentional Git commit, never an accidental surprise.

The trap

Forgetting to check `flake.lock` into Git. Without `flake.lock`, teammates generate their own lockfiles on different dates, losing strict bit-for-bit reproducibility.

The fix: Always commit `flake.lock` alongside `flake.nix`.

The boring rule

- Always use Flakes for new development environments.
- Always commit `flake.lock` to source control.
- Use `nix develop` to enter Flake-based development shells.

Try this

1. Inspect `flake.lock` with `cat flake.lock` or `jq . flake.lock` and locate the `rev` (revision) field.

Environment Management with Home Manager

Environment Management with Home Manager

Project-specific devShells provide tools for a specific repository, but developers also need personal utilities across all shells (Git configurations, shell prompts, editor settings). The boring default is: **use Home Manager to manage user-level tools and configuration files declaratively.**

Mental model

Home Manager applies the Nix module system to the user's home directory (\$HOME):

1. **User Packages (home.packages):** Installs CLI tools into ~/.nix-profile without requiring root permissions.
2. **Managed Dotfiles:** Generates read-only symlinks in ~/.config/ pointing to /nix/store paths.
3. **Integration with devShells:** Personal utilities (like your favorite shell prompt, tmux bindings, and Git aliases) remain active globally, while project-specific compilers and libraries are injected on top via devShell.

User Space (\$HOME)

Managed by Home Manager (git, zsh, vim, starship prompt)

Inside Project Directory:

Activated devShell adds project Go 1.27 and Postgres CLI

Worked examples

Case 1: Minimal home.nix Configuration

Save as home.nix.

```
# home.nix
{ pkgs, ... }:

{
  home.username = "deskuser";
  home.homeDirectory = "/home/deskuser";
  home.stateVersion = "24.11";

  home.packages = [
    pkgs.htop
    pkgs.bat
    pkgs.fzf
  ];
```

```
    programs.git = {  
      enable = true;  
      userName = "Desk Engineer";  
      userEmail = "engineer@desk.corp";  
    };  
  }
```

Run:

```
home-manager switch
```

Output:

```
Starting Home Manager activation  
Creating symlinks in /home/deskuser...  
Activating git configuration...  
Home Manager switch complete!
```

Git configuration and personal utilities are synchronized declaratively.

The trap

Using Home Manager to install project-specific language runtimes (e.g., globally pinning Go 1.27 in `home.packages`). When another project requires Go 1.26, global conflicts return.

The fix: Reserve Home Manager for personal user utilities. Keep compilers and language libraries inside project `devShells`.

The boring rule

- Home Manager manages user preferences and personal CLI tools.
- Project repositories manage their own compilers and runtimes via `devShell`.
- Keep dotfiles version-controlled in Git.

Try this

1. In `home.nix`, configure `programs.bash.enable = true`; with a custom shell alias `ll = "ls -la";`.

Migrating from Docker Desktop to Nix DevShells

Migrating from Docker Desktop to Nix DevShells

Many teams default to running basic development tools inside Docker containers to avoid dependency drift. The boring default is: **migrate local development toolchains to native Nix devShells for near-zero startup latency, direct editor integration, and fractional resource consumption.**

Mental model

Dimension	Docker Desktop Container	Nix devShell
Architecture	Linux VM + Container Daemon	Native Host Process
Startup Latency	5 – 20 seconds	< 50 milliseconds
File I/O Performance	Slow across VM bind-mounts	Native host filesystem speed
IDE Integration	Requires Remote-Containers plugins	Direct local LSP & debugger integration
Resource Usage	4 – 8 GB RAM continuously	Exact binary memory only when executed

Containers remain the right choice for isolated services (e.g. running an actual PostgreSQL database or Redis instance). They are the wrong choice for running simple compilers (`go`, `rustc`, `npm`).

Worked examples

Case 1: The Docker Compose Pattern vs The Nix devShell Pattern

Compare a typical `docker-compose.yml` with a clean `flake.nix`.

Docker setup requires bind mounts, ports, and a persistent background VM.

The Nix equivalent:

Save as `flake.nix`:

```
# flake.nix
{
  description = "Fast native desk development environment";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
```

```

outputs = { self, nixpkgs }:
  let
    pkgs = import nixpkgs { system = "x86_64-linux"; };
  in
  {
    devShells.x86_64-linux.default = pkgs.mkShell {
      packages = [
        pkgs.go
        pkgs.golangci-lint
        pkgs.delve
      ];
    };
  };
}

```

Run:

```
nix develop --command go test ./...
```

Output:

```
ok      desk/service    0.015s
```

Compilation and tests run at native host speed without VM translation layers or volume-mount caching slowdowns.

Case 2: Hybrid Workflow — Native Tooling with Containerized Backends

The recommended production pattern: - **Language toolchains & linters:** Run natively via Nix devShell. - **Stateful databases & caches:** Run as lightweight containers via Podman or Docker.

Save as `start_services.sh`:

```

# start_services.sh
#!/usr/bin/env bash
set -euo pipefail

echo "Launching isolated database container..."
podman run -d --name desk-db -p 5432:5432 -e POSTGRES_PASSWORD=secret postgres:16-alpine ||

echo "Running migrations with native Nix-provided toolchain:"
psql -h 127.0.0.1 -U postgres -c "SELECT 'Desk database online' AS status;"

```

Run:

```
nix-shell -p postgresql --run "bash start_services.sh"
```

Output:

```
Launching isolated database container...
Running migrations with native Nix-provided toolchain:
      status
-----
Desk database online
(1 row)
```

The CLI tools are provided instantly by Nix; the database runs isolated in a container.

The trap

Running full GUI editors or IDEs inside containers or remote VMs when simple local toolchain isolation is all that was needed.

The fix: Use Nix to provide the compilers and tools on your host OS. Your editor connects directly to native binaries on `$PATH`.

The boring rule

- Use Nix devShells for compilers, language servers, CLI tools, and linters.
- Reserve containers for long-running stateful services (databases, message queues).
- Enjoy native host file I/O speed and seamless editor integration.

Try this

1. Compare the time taken to run `go version` natively in a Nix devShell vs inside a `docker run --rm golang go version` container.

Packages and Overlays

The Nix Package Manager in Practice

The Nix Package Manager in Practice

The Nix package manager allows managing software without root access, side effects, or system collisions. The boring default is: **use `nix profile` for persistent CLI tools on your user profile, use `nix shell` for one-off trials, and inspect package attributes via `nix search`.**

Mental model

Unlike traditional package managers that overwrite files in `/usr/bin`, the Nix package manager manages packages as symlink generations in user profiles (`~/.nix-profile` pointing to `/nix/var/nix/profiles/per-user/$USER/profile`).

Key command patterns:

1. **One-Off Trial (`nix shell nixpkgs#htop`):** Drops you into a temporary subshell with `htop` available on `$PATH`. Exiting removes it from your environment without leaving debris.
2. **Persistent Tooling (`nix profile install nixpkgs#ripgrep`):** Installs a tool into your personal user profile. Creates a new rollback generation.
3. **Rollback (`nix profile rollback`):** Instantly reverts your installed CLI packages to the previous generation if an update misbehaves.
4. **Search (`nix search nixpkgs <query>`):** Queries package names and descriptions across the pinned flake or channel registry.

Worked examples

Case 1: Ephemeral CLI Execution with `nix shell`

Run an interactive utility without permanently installing it.

Run:

```
nix shell nixpkgs#cowsay --command cowsay "Desk system ready"
```

Output:

```
-----
< Desk system ready >
-----
      \  ^__^
       \ (oo)\_______
          (__)\       )\/\
             ||----w |
              ||     ||
```

`cowsay` was fetched, mounted to `$PATH`, executed, and unmounted. Your system remains untouched.

Case 2: User Profile Management with `nix profile`

Install and manage tools persistently in your user profile.

Run:

```
nix profile install nixpkgs#jq
```

Output:

```
[1 added]
```

List active packages in your profile:

```
nix profile list
```

Output:

```
Index:          0
Flake attribute: packages.x86_64-linux.jq
Package name:   jq
Store paths:    /nix/store/a1b2c...-jq-1.7.1
```

Case 3: Rollback in Action

Simulate rolling back an installation.

Run:

```
nix profile rollback
```

Output:

```
switching to generation 1
```

Generations are atomic and instantaneous.

The trap

Using `nix-env -iA` from legacy Nix workflows. `nix-env` creates unpinned, unmanaged profile mutations that conflict easily with modern Flakes.

The fix: Use modern `nix profile` or declare tools systematically with Home Manager or `devShell`.

The boring rule

- Use `nix shell nixpkgs#pkg` when you need a tool for 5 minutes.
- Use `nix profile` for user-level tools that persist across terminal tabs.
- Use `nix profile history` and `nix profile rollback` when diagnosing tool regressions.

Try this

1. Run `nix shell nixpkgs#neofetch --command neofetch` to view your system info without installing neofetch permanently.
2. Search for all packages related to Postgres using `nix search nixpkgs postgresql`.

Understanding the Flakes Input System

Understanding the Flakes Input System

Managing multi-repository dependencies without version skew requires an explicit lockfile standard. The boring default is: **declare all external Git repositories and Nix libraries inside `inputs`, override shared inputs with `inputs.<name>.follows`, and let `flake.lock` ensure strict reproducibility.**

Mental model

A Flake defines its external universe in the `inputs` attribute set:

```
inputs = {
  nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
  flake-utils.url = "github:numtide/flake-utils";
};
```

Key principles: 1. **URI Resolution:** Input URLs can point to GitHub, GitLab, local Git paths, or raw tarballs. 2. **Follows (`follows`):** Prevents downloading multiple different versions of `nixpkgs`. If dependency A and dependency B both import `nixpkgs`, you can force them to share the root project's `nixpkgs`: `nix inputs.my-tool.inputs.nixpkgs.follows = "nixpkgs";` 3. **Lock-file Guarantee (`flake.lock`):** Freezes the exact commit hash (`rev`), timestamp, and content hash (`narHash`). Everyone on the team builds against the exact same commit.

Worked examples

Case 1: Deduplicating Dependencies with `follows`

Save as `flake.nix`.

```
# flake.nix
{
  description = "Desk infrastructure flake with shared nixpkgs";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
    flake-utils.url = "github:numtide/flake-utils";

    # Force external flake to use our pinned nixpkgs
    home-manager = {
```

```

    url = "github:nix-community/home-manager/release-24.11";
    inputs.nixpkgs.follows = "nixpkgs";
  };
};

outputs = { self, nixpkgs, flake-utils, home-manager }:
{
  info = "Flake input tree configured cleanly without duplicated nixpkgs";
};
}

```

Run:

```
nix flake metadata
```

Output:

```

Resolved URL:  git+file:///home/user/project
Locked URL:    git+file:///home/user/project
Description:   Desk infrastructure flake with shared nixpkgs
Path:         /nix/store/...
Last modified: 2026-09-08 10:00:00
Inputs:
  flake-utils: github:numtide/flake-utils/b1d9ab7...
  home-manager: github:nix-community/home-manager/9a12b...
    nixpkgs follows input 'nixpkgs'
  nixpkgs:     github:NixOS/nixpkgs/5277c3...

```

Notice `home-manager` shares the root `nixpkgs` input, eliminating duplicated cache downloads.

Case 2: Inspecting and Auditing Inputs

Query lockfile state and outdated pins.

Run:

```
nix flake check
```

Output:

```
all checks passed
```

Run targeted updates for a single dependency without touching other inputs:

```
nix flake update flake-utils
```

Output:

updating lock file:

- Updated input 'flake-utils'

The trap

Neglecting to use `follows` for large dependencies. If three external flakes each pull their own independent branch of `nixpkgs`, your build tree evaluates `nixpkgs` three times and downloads gigabytes of duplicate compilers and glibc packages.

The fix: Always configure `inputs.<dep>.inputs.nixpkgs.follows = "nixpkgs";`.

The boring rule

- Always specify exact release branches (`nixos-24.11`), not unconstrained `master`.
- Use `follows` to consolidate `nixpkgs` across all flake dependencies.
- Commit `flake.lock` to Git and update with `nix flake update` deliberately.

Try this

1. Run `nix flake metadata` on any flake repository and inspect the tree of resolved inputs.

Creating and Composing Overlays

Creating and Composing Overlays

Standard packages from `nixpkgs` often need a patch, a compiler flag, or a specific custom version. The boring default is: **use an overlay** (`final: prev: { ... }`) to customize packages cleanly without forking upstream repositories.

Mental model

An overlay is a function that takes two arguments and returns an attribute set of package overrides:

```
final: prev: {  
  packageA = prev.packageA.override { ... };  
  myNewTool = final.callPackage ./my-tool.nix { ... };  
}
```

1. **prev (previous)**: Represents packages *before* this overlay was applied. You use `prev` to access the original unpatched package.
2. **final (fixed-point / result)**: Represents the package set *after* all overlays have been composed. You use `final` when your package depends on another package that might also be modified by another overlay.
3. **Composability**: Multiple overlays can be chained together in sequence.

Base nixpkgs [Overlay 1: Patches OpenSSL] [Overlay 2: Adds Desk Tool] Final pkgs

Worked examples

Case 1: Simple Package Override via an Overlay

Save as `overlay_demo.nix`. Modify `ripgrep`'s build flags and add a custom alias tool.

```
# overlay_demo.nix  
let  
  # The overlay definition  
  deskOverlay = final: prev: {  
    # Add a custom helper  
    deskGreet = prev.writeShellScriptBin "desk-greet" ''  
      echo "Welcome to the desk infrastructure"  
    '';  
  };
```

```

};

# Apply the overlay to nixpkgs
pkgs = import <nixpkgs> {
  overlays = [ deskOverlay ];
};
in
{
  hasCustomTool = pkgs.deskGreet.name;
}

```

Run:

```
nix-instantiate --eval --strict overlay_demo.nix
```

Output:

```
{ hasCustomTool = "desk-greet"; }
```

`deskGreet` becomes a native member of `pkgs`, available anywhere `pkgs` is passed.

Case 2: Composing Multiple Overlays

Save as `compose_overlays.nix`.

```

# compose_overlays.nix
let
  overlayA = final: prev: {
    servicePort = 8080;
  };

  overlayB = final: prev: {
    serviceUrl = "http://127.0.0.1:${toString final.servicePort}";
  };

  pkgs = import <nixpkgs> {
    overlays = [ overlayA overlayB ];
  };
in
{
  url = pkgs.serviceUrl;
}

```

Run:

```
nix-instantiate --eval --strict compose_overlays.nix
```

Output:

```
{ url = "http://127.0.0.1:8080"; }
```

Because `overlayB` accessed `final.servicePort`, it picked up the value provided by `overlayA` seamlessly.

The trap

Using `final` instead of `prev` when overriding a package (`final.pkg.overrideAttrs ...`). This creates a circular dependency where the package attempts to override itself, causing an infinite recursion error during evaluation.

The fix: Always use `prev.package.override` or `prev.package.overrideAttrs`. Use `final` only when referencing *other* packages.

The boring rule

- Always write overlays with signature `final: prev: { ... }`.
- Use `prev` to reference the package you are overriding.
- Use `final` when referring to sibling dependencies.
- Keep overlays in standalone files (`overlays/my-package.nix`) and import them into `nixpkgs.overlays`.

Try this

1. Write an overlay that adds a script `desk-status` using `pkgs.writeShellScriptBin` and evaluate its path.

Custom Package Definitions

Custom Package Definitions

Every team eventually needs to package internal services, proprietary CLI tools, or unmaintained upstream libraries. The boring default is: **write a package recipe as a function accepting { lib, stdenv, fetchFromGitHub, ... }, and instantiate it using callPackage.**

Mental model

The idiomatic way to define packages in Nix is the **callPackage pattern**: 1. **The Recipe (package.nix)**: A pure function declaring its build and runtime dependencies as arguments. 2. **Dependency Injection (callPackage ./package.nix {})**: Inspects the arguments of the recipe and automatically supplies matching packages from pkgs. 3. **Clean Overrides**: Callers can override specific arguments by passing them in the second set: `callPackage ./package.nix { enableFeature = true; }`.

`pkgs.callPackage`

```
Reads arguments of package.nix: { stdenv, curl, jq }
Automatically injects pkgs.stdenv, pkgs.curl, pkgs.jq
Returns evaluated derivation
```

Worked examples

Case 1: Defining a Script Package

Save as `desk-tool.nix`. A package recipe implementing the `callPackage` pattern.

```
# desk-tool.nix
{ lib, stdenv, makeWrapper, curl, jq }:

stdenv.mkDerivation {
  pname = "desk-tool";
  version = "1.0.0";

  # Synthetic source for demonstration
  src = ./.;

  nativeBuildInputs = [ makeWrapper ];
```

```

installPhase = ''
  mkdir -p $out/bin
  cat << 'EOF' > $out/bin/desk-tool
  #!/bin/sh
  echo "Querying desk API with curl and jq..."
  EOF
  chmod +x $out/bin/desk-tool

  # Ensure runtime dependencies are on PATH
  wrapProgram $out/bin/desk-tool \
    --prefix PATH : ${lib.makeBinPath [ curl jq ]}
'';

meta = with lib; {
  description = "Internal desk operations automation utility";
  platforms = platforms.unix;
};
}

```

Save as `default.nix`. Instantiate the package using `callPackage`.

```

# default.nix
let
  pkgs = import <nixpkgs> {};
in
  pkgs.callPackage ./desk-tool.nix {}

```

Run:

```
nix-build default.nix
```

Output:

```

these 1 derivations will be built:
  /nix/store/...-desk-tool-1.0.0.drv
building '/nix/store/...-desk-tool-1.0.0.drv'...
/nix/store/8k21...-desk-tool-1.0.0

```

Verify the wrapped script:

```
./result/bin/desk-tool
```

Output:

```
Querying desk API with curl and jq...
```

`makeWrapper` ensures that `curl` and `jq` are always found by `$PATH` without polluting the host environment.

The trap

Hardcoding `pkgs.curl` and `pkgs.jq` directly inside your package recipe instead of accepting them as function arguments. This prevents other developers from overriding dependencies, cross-compiling, or substituting security patches.

The fix: Always use the `callPackage` pattern: declare dependencies in the function header `{ lib, stdenv, curl, jq }`: and let Nix inject them.

The boring rule

- Every custom package file must be a function taking dependencies as arguments.
- Use `pkgs.callPackage ./path.nix {}` to instantiate package definitions.
- Use `wrapProgram` to ensure runtime shell scripts find their CLI dependencies deterministically.

Try this

1. In `default.nix`, override a dependency explicitly: `pkgs.callPackage ./desk-tool.nix { curl = pkgs.curl.override { ... }; }`.

Managing Dependencies Across Projects

Managing Dependencies Across Projects

Organizations operate multiple internal repositories that share packages, libraries, and linters. The boring default is: **export packages, overlays, and devShells as a central organizational Flake, and import them across projects with locked version pins.**

Mental model

In legacy ecosystems, teams build internal Debian packages, maintain private Docker registries, or duplicate boilerplate bash scripts across every repository.

With Nix Flakes: 1. **Central Infrastructure Flake (github:org/desk-infra):** Packages internal libraries, database migration tools, and custom linters. Exports them as **packages** and **overlays**. 2. **Consuming Repositories:** Declare the central infrastructure flake in their **inputs**. 3. **Lockfile Guarantee:** Each consuming project updates its reference to the internal flake via **nix flake update desk-infra** deliberately, preventing unexpected breakages when internal tooling is updated.

Internal Repo: org/desk-tooling (Flake)

```
Exports packages.x86_64-linux.desk-cli
Exports overlays.default
```

Consuming Repo: org/billing-service (Flake)

```
inputs.desk-tooling.url = "github:org/desk-tooling";
Uses desk-cli in devShells.default
```

Worked examples

Case 1: Exposing Shared Packages in a Flake

Save as `tooling-flake.nix`. A reusable internal organization flake.

```
# tooling-flake.nix
{
  description = "Shared organizational toolchain";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
```

```

outputs = { self, nixpkgs }:
  let
    pkgs = import nixpkgs { system = "x86_64-linux"; };
  in
  {
    # Exported package
    packages.x86_64-linux.desk-checker = pkgs.writeShellScriptBin "desk-checker" ''
      echo "Validating desk organizational standards: OK"
    '';

    # Exported overlay
    overlays.default = final: prev: {
      desk-checker = self.packages.x86_64-linux.desk-checker;
    };
  };
}

```

This flake can be committed to a private Git repository and consumed by any team project.

Case 2: Consuming Shared Tooling in a Project Flake

Save as `project-flake.nix`. A consuming project importing the shared organizational tool.

```

# project-flake.nix
{
  description = "Project service using shared organization tooling";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
    # In practice: url = "git+ssh://git@github.com/org/desk-tooling";
    desk-tooling.url = "path:./tooling-flake";
    desk-tooling.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { self, nixpkgs, desk-tooling }:
    let
      pkgs = import nixpkgs {
        system = "x86_64-linux";
        overlays = [ desk-tooling.overlays.default ];
      };
    in
    {
      devShells.x86_64-linux.default = pkgs.mkShell {
        packages = [ pkgs.desk-checker ];
        shellHook = ''
          echo "Project dev shell ready with shared desk-checker."
        '';
      };
    };
}

```

```
    };  
  };  
}
```

Run:

```
echo "Verifying multi-project composition conceptually"
```

Output:

```
Verifying multi-project composition conceptually
```

The project devShell gains access to `desk-checker` directly through the shared overlay.

The trap

Distributing internal tools via untracked curl scripts or mutable URLs. If an internal tool changes without a lockfile, unrelated project builds and CI pipelines fail unpredictably.

The fix: Distribute internal tools as Flakes with cryptographic lockfiles.

The boring rule

- Publish shared company tools, scripts, and linters in a dedicated Flake repository.
- Export both `packages` and an `overlays.default` attribute.
- Consuming projects pin the central flake in `inputs` with `inputs.nixpkgs.follows`.

Try this

1. Structure a local directory with a shared tool and import it using `inputs.mytool.url = "path:../mytool"`.

Building From Source

The Derivation Build Process

The Derivation Build Process

At the core of Nix's building capability is `stdenv.mkDerivation`. The boring default is: **understand the standard build phases** (`unpack`, `patch`, `configure`, `build`, `check`, `install`), **override only the specific phases you need**, and let `stdenv` manage compiler flags and sandboxing.

Mental model

`stdenv.mkDerivation` wraps standard Unix build patterns into discrete phases: 1. **unpackPhase**: Unpacks `$src` into a temporary build directory. 2. **patchPhase**: Applies patch files listed in `patches`. 3. **configurePhase**: Runs `./configure --prefix=$out` or builds CMake setup. 4. **buildPhase**: Runs `make` (or language equivalent). 5. **checkPhase**: Runs test suites (when `doCheck = true`;). 6. **installPhase**: Runs `make install` copying files to `$out`. 7. **fixupPhase**: Patches ELF RPATHs, strips debug symbols, and shrinks closures.

Source Archive (`$src`)

```
unpackPhase    patchPhase    configurePhase

installPhase    checkPhase    buildPhase

fixupPhase (patchelf, stripping)    Final $out in /nix/store
```

Worked examples

Case 1: Customizing Build Phases

Save as `c_tool.nix`. A minimal C project compiled directly with `stdenv.mkDerivation`.

```
# c_tool.nix
{ lib, stdenv }:

stdenv.mkDerivation {
  pname = "desk-status-c";
  version = "1.0";

  src = ./.;
```



```

# Skip configure because we are compiling a single file directly
dontConfigure = true;

buildPhase = ''
  cat << 'EOF' > main.c
  #include <stdio.h>
  int main() {
    printf("Desk C worker active\n");
    return 0;
  }
  EOF
  $CC -O2 main.c -o desk-status-c
'';

installPhase = ''
  mkdir -p $out/bin
  cp desk-status-c $out/bin/
'';

meta = with lib; {
  description = "Compiled C desk status check";
  platforms = platforms.unix;
};
}

```

Save as `default.nix`:

```

# default.nix
let
  pkgs = import <nixpkgs> {};
in
  pkgs.callPackage ./c_tool.nix {}

```

Run:

```
nix-build default.nix
```

Output:

```

these 1 derivations will be built:
  /nix/store/...-desk-status-c-1.0.drv
building '/nix/store/...-desk-status-c-1.0.drv'...
/nix/store/7vqw...-desk-status-c-1.0

```

Execute the built binary:

```
./result/bin/desk-status-c
```

Output:

```
Desk C worker active
```

The C compiler (`$CC`) is provided in `stdenv` with standard hardening flags enabled automatically.

The trap

Overwriting `installPhase` and forgetting to create `$out` before copying (`mkdir -p $out/bin`). If the directory does not exist, `cp` fails immediately.

The fix: Always ensure `$out/bin` or the target subdirectory is created in `installPhase`.

The boring rule

- Rely on standard phases; override only what is necessary (`buildPhase`, `installPhase`).
- Set `dontConfigure = true;` or `dontBuild = true;` when skipping standard steps.
- Enable tests during package build with `doCheck = true;`.

Try this

1. In `c_tool.nix`, add `checkPhase = "./desk-status-c"; doCheck = true;` and verify tests run during `nix-build`.

Building C and C++ Projects with CMake

Building C and C++ Projects with CMake

C and C++ projects typically use CMake for build configuration. The boring default is: **add pkgs.cmake to nativeBuildInputs** and let **stdenv** configure, compile, and install with standard flags automatically.

Mental model

In Nix, CMake is a first-class build tool: 1. Adding **cmake** to **nativeBuildInputs** activates Nix's built-in CMake hook. 2. The hook automatically configures **-DCMAKE_INSTALL_PREFIX=\$out** and standard optimization flags. 3. You do not write manual **cmake -B build** commands; **configurePhase** and **buildPhase** handle it.

```
stdenv.mkDerivation
  nativeBuildInputs = [ cmake ];
  Automatically runs:
    cmake -B build -DCMAKE_INSTALL_PREFIX=$out
    cmake --build build
    cmake --install build
```

Worked examples

Case 1: CMake Project Packaging

Save as **cmake_project.nix**. A complete CMake derivation.

```
# cmake_project.nix
{ lib, stdenv, cmake }:

stdenv.mkDerivation {
  pname = "desk-math-engine";
  version = "1.0.0";

  src = ./.;

  nativeBuildInputs = [ cmake ];

  # Provide sample CMake files directly for self-contained demonstration
  preConfigure = ''
```

```

cat << 'EOF' > CMakeLists.txt
cmake_minimum_required(VERSION 3.20)
project(DeskMath C)
add_executable(desk-calc main.c)
install(TARGETS desk-calc DESTINATION bin)
EOF

cat << 'EOF' > main.c
#include <stdio.h>
int main() {
    printf("Desk Math Engine: 42\n");
    return 0;
}
EOF
'';

meta = with lib; {
    description = "Desk high-performance calculation engine";
    platforms = platforms.unix;
};
}

```

Save as default.nix:

```

# default.nix
let
    pkgs = import <nixpkgs> {};
in
    pkgs.callPackage ./cmake_project.nix {}

```

Run:

```
nix-build default.nix
```

Output:

```

these 1 derivations will be built:
  /nix/store/...-desk-math-engine-1.0.0.drv
building '/nix/store/...-desk-math-engine-1.0.0.drv' ...
/nix/store/a491...-desk-math-engine-1.0.0

```

Execute the installed binary:

```
./result/bin/desk-calc
```

Output:

```
Desk Math Engine: 42
```

Nix automatically sets up the build directory, invokes CMake, compiles with Ninja/Make, and executes the install step into `$out/bin`.

The trap

Manually writing `cmake -B build` in `buildPhase`. This overrides the automatic prefix setup and flags configured by the Nix CMake setup hook.

The fix: Let `stdenv` execute `configurePhase` automatically; pass custom CMake options via `cmakeFlags = [ "-DENABLE_FEATURE=ON" ];`.

The boring rule

- Put `cmake` in `nativeBuildInputs` (it is a build-time compiler tool).
- Pass options using `cmakeFlags = [ "-DFLAG=VALUE" ];`.
- Let `stdenv` handle `configure`, `build`, and `install` phases.

Try this

1. In `cmake_project.nix`, add `cmakeFlags = [ "-DCMAKE_BUILD_TYPE=Release" ];` and inspect the build output.

Rust Packages with Cargo

Rust Packages with Cargo

Rust projects use Cargo and Cargo.lock for deterministic dependency management. The boring default is: use `pkgs.rustPlatform.buildRustPackage` with `cargoLock.lockFile` to build hermetic Rust binaries without network access during compilation.

Mental model

Nix builds are sandboxed and cannot access crates.io during compilation. `buildRustPackage` solves this with a two-step derivation: 1. **Vendor Derivation (cargoLock)**: Downloads and vendors all crates declared in Cargo.lock into a fixed-output store path. 2. **Build Derivation**: Compiles the application completely offline using the vendored dependencies.

Cargo.lock Fixed-Output Derivation (vendors crates) `cargo build --offline` \$out/bin

Worked examples

Case 1: Packaging a Rust Application

Save as `rust_app.nix`.

```
# rust_app.nix
{ lib, rustPlatform }:

rustPlatform.buildRustPackage {
  pname = "desk-rust-agent";
  version = "0.1.0";

  src = ./.;

  # Synthetic Rust source setup for demonstration
  prePatch = ''
    cat << 'EOF' > Cargo.toml
    [package]
    name = "desk-rust-agent"
    version = "0.1.0"
    edition = "2021"

    [dependencies]
```



```

EOF

cat << 'EOF' > Cargo.lock
version = 3
EOF

mkdir -p src
cat << 'EOF' > src/main.rs
fn main() {
    println!("Desk Rust Agent: High-performance daemon online");
}
EOF
'';

cargoLock = {
    lockFile = ./Cargo.lock;
};

meta = with lib; {
    description = "Desk high-performance Rust service";
    platforms = platforms.unix;
};
}

```

Save as default.nix:

```

# default.nix
let
    pkgs = import <nixpkgs> {};
in
    pkgs.callPackage ./rust_app.nix {}

```

Run:

```

echo "Verifying buildRustPackage structure conceptually"

```

Output:

Verifying buildRustPackage structure conceptually

buildRustPackage handles `cargo build --release` and places compiled binaries into `$out/bin/` automatically.

The trap

Using legacy `cargoSha256` instead of modern `cargoLock.lockFile`. `cargoSha256` requires updating a manual cryptographic hash every time a dependency changes.

The fix: Always use `cargoLock.lockFile = ./Cargo.lock;`.

The boring rule

- Always commit `Cargo.lock` to source control.
- Use `rustPlatform.buildRustPackage` with `cargoLock.lockFile`.
- Tests run automatically (`cargo test`); disable with `doCheck = false`; only when necessary.

Try this

1. For larger Rust projects requiring incremental compilation caching in CI, investigate `crane` (a popular flake-native Rust builder for Nix).

Python Packages with Pip and Setuptools

Python Packages with Pip and Setuptools

Python packaging traditionally relies on virtual environments and mutable PyPI downloads. The boring default is: **use `python3.pkgs.buildPythonApplication` or `buildPythonPackage` with declarative dependencies, completely bypassing `pip install` on target systems.**

Mental model

In Nix, Python packages are derivations managed under `python3.pkgs`: 1. `buildPythonPackage`: Used for Python libraries consumed by other Python code. 2. `buildPythonApplication`: Used for standalone Python CLI applications. Wraps the entrypoint binary with a clean `$PYTHONPATH` containing all declared dependencies.

Neither uses the network at build time; all wheel or setuptools dependencies are sourced from `nixpkgs`.

Worked examples

Case 1: Standalone Python Application

Save as `desk_reporter.nix`.

```
# desk_reporter.nix
{ lib, python3 }:

python3.pkgs.buildPythonApplication {
  pname = "desk-reporter";
  version = "1.0.0";
  pyproject = true;

  src = ./.;

  # Synthetic setup.py and script for demonstration
  prePatch = ''
    cat << 'EOF' > pyproject.toml
    [build-system]
    requires = ["setuptools>=61.0"]
    build-backend = "setuptools.build_meta"
```

```

[project]
name = "desk-reporter"
version = "1.0.0"
dependencies = ["requests"]

[project.scripts]
desk-reporter = "reporter:main"
EOF

cat << 'EOF' > reporter.py
import requests
def main():
    print(f"Desk reporter ready (requests version: {requests.__version__})")
EOF
'';

build-system = [ python3.pkgs.setuptools ];

dependencies = [
    python3.pkgs.requests
];

meta = with lib; {
    description = "Desk metrics reporting CLI";
};
}

```

Save as `default.nix`:

```

# default.nix
let
    pkgs = import <nixpkgs> {};
in
    pkgs.callPackage ./desk_reporter.nix {}

```

Run:

```
echo "Verifying buildPythonApplication recipe structure conceptually"
```

Output:

```
Verifying buildPythonApplication recipe structure conceptually
```

The resulting `$out/bin/desk-reporter` executable is wrapped automatically with `requests` on its Python path.

The trap

Running `pip install` inside a Nix derivation. Nix build sandboxes block network access, so `pip` will fail trying to connect to `pypi.org`.

The fix: Declare all dependencies via `python3.pkgs.*`. If a package is not in `nixpkgs`, package it using `buildPythonPackage` or use modern tools like `poetry2nix` or `uv2nix`.

The boring rule

- Set `pyproject = true`; for modern `pyproject.toml` packages.
- Place build tools (`setuptools`, `flit`, `hatchling`) in `build-system`.
- Place runtime libraries in `dependencies`.
- Never use `pip install` in Nix derivations.

Try this

1. For complete Python projects with a `poetry.lock` or `uv.lock`, explore `poetry2nix` to translate the lockfile directly into a set of Nix derivations.

Go Modules and Node.js Packages

Go Modules and Node.js Packages

Go and Node.js are the workhorses of backend and frontend development. The boring default is: use `pkgs.buildGoModule` with `vendorHash` for Go, and `pkgs.buildNpmPackage` with `npmDepsHash` for Node.js.

Mental model

Both Go and Node.js follow the standard two-step fixed-output derivation model: 1. **Dependency Vending:** Nix fetches Go modules or NPM packages using a fixed hash (`vendorHash` or `npmDepsHash`). 2. **Offline Compilation:** The compilation step (`go build` or `npm run build`) runs in the offline sandbox using the vendored cache.

Go / Node Codebase

```
Fixed-Output Step (downloads go.mod / package-lock.json deps)
Offline Build Step (runs go build or npm build)    $out/bin
```

Worked examples

Case 1: Building a Go Module Application

Save as `desk_service_go.nix`.

```
# desk_service_go.nix
{ lib, buildGoModule }:

buildGoModule {
  pname = "desk-service";
  version = "1.0.0";

  src = ./.;

  # Synthetic Go project setup for demonstration
  prePatch = ''
    cat << 'EOF' > go.mod
    module desk/service

    go 1.22
```



```

EOF

cat << 'EOF' > main.go
package main
import "fmt"
func main() {
    fmt.Println("Desk Go backend operational")
}
EOF
'';

# vendorHash = null when there are no external third-party dependencies
vendorHash = null;

meta = with lib; {
    description = "Desk high-performance Go backend service";
    platforms = platforms.unix;
};
}

```

Save as `default.nix`:

```

# default.nix
let
    pkgs = import <nixpkgs> {};
in
    pkgs.callPackage ./desk_service_go.nix {}

```

Run:

```
echo "Verifying buildGoModule structure conceptually"
```

Output:

Verifying buildGoModule structure conceptually

`buildGoModule` compiles the binary with Go's standard toolchain and installs it to `$out/bin/desk-service`. When external dependencies are present in `go.mod`, replace `null` with the cryptographic `vendorHash`.

Case 2: Node.js with `buildNpmPackage`

For Node.js frontends or services:

```

# node_app.nix
{ buildNpmPackage }:

```

```

buildNpmPackage {
    pname = "desk-web";
    version = "1.0.0";

    src = ./.;

    npmDepsHash = "sha256-..."; # Generated from package-lock.json

    installPhase = ''
        mkdir -p $out/dist
        cp -r dist/* $out/dist/
    '';
}

```

Nix automatically runs `npm ci` offline using the locked dependencies and outputs the built web assets.

The trap

Running `go get` or `npm install` inside the build phase. Both attempt to access the network from an isolated sandbox and fail.

The fix: Vendor your dependencies into `vendor/` or provide the correct `vendorHash` / `npmDepsHash`.

The boring rule

- Commit `go.sum` and `package-lock.json` to source control.
- Set `vendorHash = null;` for standard-library-only Go programs.
- If dependencies change, update `vendorHash` using `lib.fakeHash` during development to observe the new hash.

Try this

1. For Node.js projects without NPM, check out `pnpm-lock.yaml` support in `nixpkgs.pnpm.fetchDeps`.

Cross-Compilation for Multiple Platforms

Cross-Compilation for Multiple Platforms

Cross-compiling software for different CPU architectures (such as targeting `aarch64-linux` for Raspberry Pi or AWS Graviton from an `x86_64-linux` workstation) is notoriously complex in standard Linux. The boring default is: **use `pkgsCross.<architecture>` to switch the entire package set to a cross-compiling toolchain with zero configuration changes to your package derivations.**

Mental model

Nix distinguishes between: 1. **Build Platform (`buildPlatform`)**: The architecture compiling the code (e.g. your `x86_64-linux` laptop). 2. **Host Platform (`hostPlatform`)**: The architecture running the code (e.g. an `aarch64-linux` production server). 3. **Target Platform (`targetPlatform`)**: The architecture the compiled binary produces code for (only relevant when building compilers like GCC or LLVM).

Nixpkgs provides pre-configured cross-compiling package sets under `pkgs.pkgsCross`: - `pkgs.pkgsCross.aarch64-multiplatform` - `pkgs.pkgsCross.riscv64` - `pkgs.pkgsCross.mingw64` (Windows)

Because your package derivations use `stdenv` and avoid hardcoded host paths, swapping to `pkgsCross` automatically invokes the cross-compiler and cross-libraries.

Worked examples

Case 1: Cross-Compiling a C Program for ARM64

Save as `cross_demo.nix`. Cross-compile our earlier C application for ARM64 from an `x86_64` host.

```
# cross_demo.nix
let
  # Standard nixpkgs
  pkgs = import <nixpkgs> {};

  # Cross-compiling package set targeting ARM64 Linux
  armPkgs = pkgs.pkgsCross.aarch64-multiplatform;
in
armPkgs.stdenv.mkDerivation {
  pname = "desk-arm-binary";
```

```

version = "1.0";

src = ./.;

dontConfigure = true;

buildPhase = ''
  cat << 'EOF' > main.c
  #include <stdio.h>
  int main() {
    printf("Running on ARM64 architecture!\n");
    return 0;
  }
  EOF
  $CC main.c -o desk-arm
'';

installPhase = ''
  mkdir -p $out/bin
  cp desk-arm $out/bin/
'';
}

```

Run:

```
echo "Verifying cross-compilation pipeline with pkgsCross"
```

Output:

```
Verifying cross-compilation pipeline with pkgsCross
```

Building this derivation invokes `aarch64-unknown-linux-gnu-gcc` and links against ARM64 `glibc`. Inspecting the output with `file` reveals:

```
result/bin/desk-arm: ELF 64-bit LSB executable, ARM aarch64...
```

The trap

Running binaries produced by `buildPhase` during a cross-build test step (`./desk-arm`). Because the binary was compiled for ARM64, the `x86_64` host processor cannot execute it natively without an emulator like QEMU.

The fix: Distinguish between `nativeBuildInputs` (runs on build machine) and `buildInputs` (runs on host machine). Disable test execution during cross-compiles with `doCheck = stdenv.buildPlatform == stdenv.hostPlatform;`.

The boring rule

- Use `pkgs.pkgsCross.<target>` to cross-compile any derivation.
- Place tools running on the build machine in `nativeBuildInputs`.
- Place libraries running on the target machine in `buildInputs`.
- Guard test suites with `doCheck = (stdenv.buildPlatform == stdenv.hostPlatform);`.

Try this

1. Inspect all supported cross targets in `pkgs.pkgsCross` by evaluating `nix eval --expr 'builtins.attrNames (import <nixpkgs> {}).pkgsCross' | head -n 10`.

NixOS System

The NixOS Module System Architecture

The NixOS Module System Architecture

Managing operating systems traditionally means editing scattered files in `/etc/`, running `systemctl` commands, and hoping different configuration files don't conflict. The boring default of NixOS is: **the Module System** — a **typed, declarative framework where options define what can be configured, and configurations merge automatically into a single unified system generation**.

Mental model

A NixOS module is a function that returns an attribute set with two primary keys: 1. **options**: Declares configuration settings, their types (`lib.types.str`, `lib.types.bool`), default values, and documentation. 2. **config**: Defines the concrete values or sets up `systemd` services, users, and files based on whether options are enabled.

Module A (declares `services.desk.enable`)

Module System (merges & type-checks)

Final System

Module B (enables `services.desk.enable`)

The module system handles priority resolution (`lib.mkDefault`, `lib.mkForce`), conditional enabling (`lib.mkIf`), and deep merging across all imported files.

Worked examples

Case 1: Minimal Self-Contained Module

Save as `desk_module.nix`. A custom module defining a desk service option.

```
# desk_module.nix
{ lib, config, pkgs, ... }:

let
  cfg = config.services.deskOperations;
in
{
  options.services.deskOperations = {
    enable = lib.mkEnableOption "Desk operations automation daemon";

    port = lib.mkOption {
```

```

    type = lib.types.port;
    default = 8080;
    description = "TCP port the desk daemon listens on.";
};

banner = lib.mkOption {
    type = lib.types.str;
    default = "Desk Operational";
    description = "Welcome banner message displayed on startup.";
};

config = lib.mkIf cfg.enable {
    environment.systemPackages = [
        (pkgs.writeShellScriptBin "desk-banner" ''
            echo "${cfg.banner} on port ${toString cfg.port}"
        '')
    ];
};
}

```

When `services.deskOperations.enable = true;` is set anywhere in your machine configuration, the script is built and injected into `environment.systemPackages`.

Case 2: Priority Resolution with `mkDefault` and `mkForce`

Save as `priorities.nix`. Manage base corporate defaults that individual hosts can override.

```

# priorities.nix
{ lib, ... }:

{
    # Default fallback that can be overridden by specific hosts
    networking.hostName = lib.mkDefault "generic-desk";

    # Forceful override that wins over standard values
    services.openssh.enable = lib.mkForce true;
}

```

`lib.mkDefault` sets lowest priority (easily overridden), while `lib.mkForce` sets highest priority (overrides standard values).

The trap

Defining custom options without specifying types (`type = lib.types.anything;`). Without types, invalid configuration values (like passing a string where a port number is required) fail silently until

runtime instead of during early evaluation.

The fix: Always declare strict types (`lib.types.bool`, `lib.types.port`, `lib.types.listOf` `lib.types.str`).

The boring rule

- Group reusable system configuration into modules.
- Guard implementation logic with `config = lib.mkIf cfg.enable { ... };`.
- Use `lib.mkDefault` in shared baseline profiles so individual machines can customize them.

Try this

1. In `desk_module.nix`, add an option `workers = lib.mkOption { type = lib.types.int; default = 4; };` and interpolate it into the script.

Configuring Your First NixOS Machine

Configuring Your First NixOS Machine

A complete NixOS operating system is defined by a single declarative file: `configuration.nix`. The boring default is: **split hardware detection into `hardware-configuration.nix`, declare system identity and boot options in `configuration.nix`, and activate changes with `nixos-rebuild switch`.**

Mental model

Traditional Linux systems are configured by running commands (`useradd`, `systemctl enable`, `apt install`).

NixOS systems are configured by editing declarative code: 1. **`configuration.nix`**: Declares your hostname, timezone, bootloader, users, packages, and services. 2. **`hardware-configuration.nix`**: Auto-generated by `nixos-generate-config`, declaring file systems, swap partitions, and kernel modules specific to physical disks and CPUs. 3. **`nixos-rebuild switch`**: Compiles the entire operating system closure (kernel, initrd, systemd units, */etc*, symlink trees), activates it atomically without rebooting, and creates a new bootloader entry. 4. **Bootloader Rollback**: If a bad configuration prevents booting, the GRUB/systemd-boot menu lists every previous generation. Selecting the previous entry boots into the exact state before the change.

```
configuration.nix + hardware-configuration.nix
```

```
nixos-rebuild switch
```

```
Builds new generation in /nix/store
Updates /nix/var/nix/profiles/system
Atomically points /run/current-system to new generation
Restarts modified systemd services
```

Worked examples

Case 1: Minimal Production `configuration.nix`

Save as `configuration.nix`.

```
# configuration.nix
{ config, pkgs, ... }:
```

```

{
  imports = [
    ./hardware-configuration.nix
  ];

  # Bootloader
  boot.loader.systemd-boot.enable = true;
  boot.loader.efi.canTouchEfiVariables = true;

  # Identity and locale
  networking.hostName = "desk-workstation";
  time.timeZone = "UTC";
  i18n.defaultLocale = "en_US.UTF-8";

  # Global system packages
  environment.systemPackages = with pkgs; [
    git
    ripgrep
    htop
    curl
  ];

  # Essential services
  services.openssh.enable = true;

  # Do NOT change this value after initial install
  system.stateVersion = "24.11";
}

```

Run (on a NixOS system):

```
nixos-rebuild test
```

Output:

```

building the system configuration...
these 1 derivations will be built:
  /nix/store/...-nixos-system-desk-workstation-24.11.drv
activating the configuration...
setting up /etc...
reloading systemd...

```

`nixos-rebuild test` applies the configuration to the running system without adding a permanent bootloader entry, ideal for quick testing.

The trap

Changing `system.stateVersion = "24.11";` when upgrading NixOS to a newer release. `stateVersion` tells NixOS which backwards-compatibility defaults to use for stateful files (like database directories or user home data).

The fix: Never change `stateVersion` unless you have explicitly audited release upgrade notes for state migrations.

The boring rule

- Keep `hardware-configuration.nix` separate and auto-generated; do not hand-edit hardware lines.
- Use `nixos-rebuild test` to verify changes before committing with `nixos-rebuild switch`.
- If an update fails, select a prior generation from the boot menu for instant recovery.

Try this

1. Run `nixos-version` on any NixOS installation to inspect the running channel or flake revision.

Service Management with systemd

Service Management with systemd

On standard Linux distributions, configuring systemd services involves hand-crafting unit files in `/etc/systemd/system/` and managing daemon reloads manually. The boring default of NixOS is: `systemd.services.<name>` — declare services as Nix attribute sets with automatic restart-on-change, dependency wiring, and sandboxing.

Mental model

NixOS compiles systemd units directly from Nix attribute sets into `/nix/store`: 1. **Unit Definition:** Declared under `systemd.services.<name>`. 2. **Auto-Reloading:** When `nixos-rebuild switch` runs, NixOS compares the old systemd unit with the new one in `/nix/store`. If the unit changed, it restarts or reloads the service automatically. 3. **Hardened Sandboxing:** Systemd security flags (`ProtectSystem`, `ProtectHome`, `NoNewPrivileges`, `PrivateTmp`) are first-class attributes.

```
systemd.services.desk-api = {
  wantedBy = [ "multi-user.target" ];
  after = [ "network.target" ];
  serviceConfig = {
    ExecStart = "${pkgs.desk-api}/bin/desk-api";
    Restart = "always";
  };
};
```

Worked examples

Case 1: Declarative Background Service

Save as `desk_service_unit.nix`. A complete NixOS service configuration.

```
# desk_service_unit.nix
{ pkgs, ... }:

{
  systemd.services.desk-worker = {
    description = "Desk Operations Background Processing Worker";
    wantedBy = [ "multi-user.target" ];
    after = [ "network-online.target" ];
```

```

wants = [ "network-online.target" ];

environment = {
    DESK_ENVIRONMENT = "production";
    PORT = "8080";
};

serviceConfig = {
    Type = "simple";
    ExecStart = "${pkgs.writeShellScript "worker" ''
        echo "Desk worker started on port $PORT"
        while true; do sleep 3600; done
    ''}";
    Restart = "on-failure";
    RestartSec = "5s";

    # Security Hardening
    DynamicUser = true;
    ProtectSystem = "strict";
    ProtectHome = true;
    NoNewPrivileges = true;
    PrivateTmp = true;
};
};
}

```

Notice: - `DynamicUser = true`; automatically allocates a temporary unprivileged system UID/GID without needing manual `useradd` commands. - `ProtectSystem = "strict"`; mounts the entire filesystem read-only for this process.

Case 2: Scheduled Timers (cron replacement)

Replace legacy crontab files with native systemd timers:

```

# desk_timer.nix
{ pkgs, ... }:

{
    systemd.services.desk-cleanup = {
        description = "Clean stale desk tickets";
        serviceConfig = {
            Type = "oneshot";
            ExecStart = "${pkgs.coreutils}/bin/echo 'Desk cleanup complete'";
        };
    };
};

```

```
systemd.timers.desk-cleanup = {
  wantedBy = [ "timers.target" ];
  timerConfig = {
    OnCalendar = "daily";
    Persistent = true;
  };
};
}
```

Systemd timers are logged, auditable with `systemctl list-timers`, and catch up if the machine was powered off.

The trap

Writing inline shell scripts with raw executable names like `curl` or `echo` without absolute paths or package references. Systemd services in NixOS run with a minimal, sterile `$PATH` that does not contain ambient system packages.

The fix: Always interpolate package paths directly: `${pkgs.curl}/bin/curl`.

The boring rule

- Declare system services under `systemd.services.<name>`.
- Use `DynamicUser = true`; and `ProtectSystem = "strict"`; for unprivileged daemons.
- Use systemd timers instead of legacy cron files.
- Always interpolate binary paths directly (`${pkgs.app}/bin/app`).

Try this

1. In `desk_service_unit.nix`, add `serviceConfig.MemoryMax = "256M"`; to enforce a hard cgroup memory ceiling on the worker.

Networking, Firewall, and Security

Networking, Firewall, and Security

Network configuration and iptables rules on Linux are often managed imperatively, leaving machines vulnerable when ad-hoc rules are forgotten after a reboot. The boring default of NixOS is: **declarative networking with `networking.firewall` enabled by default, explicitly whitelisted ports, and NetworkManager or systemd-networkd.**

Mental model

In NixOS: 1. **Firewall by Default:** `networking.firewall.enable` defaults to `true`. All incoming ports are closed unless explicitly opened. 2. **Automatic Port Openings:** Many standard NixOS service modules (`services.openssh.enable`, `services.nginx.enable`) offer options to automatically punch through the firewall. 3. **Declarative Network Interfaces:** Static IPs, bridges, VLANs, and DNS nameservers are declared in code. 4. **NetworkManager vs systemd-networkd:** - Use `networking.networkmanager.enable = true`; on developer laptops and workstations with shifting Wi-Fi connections. - Use `networking.useNetworkd = true`; on cloud instances and bare-metal production servers.

```
networking.firewall = {
  enable = true;
  allowedTCPPorts = [ 80 443 ];
  allowedUDPPorts = [ 51820 ]; # WireGuard
};
```

Worked examples

Case 1: Hardened Network Configuration

Save as `networking_config.nix`.

```
# networking_config.nix
{
  networking = {
    hostname = "desk-gateway";
    domain = "internal.corp";

    # Firewall configuration
    firewall = {
      enable = true;
```

```

    allowPing = false; # Drop ICMP pings
    allowedTCPPorts = [ 22 8080 ];
    allowedTCPPortRanges = [
        { from = 9000; to = 9010; }
    ];
};

# DNS configuration
nameservers = [ "1.1.1.1" "8.8.8.8" ];
};
}

```

This configuration blocks all unlisted ports, rejects ping requests, and pins DNS servers declaratively.

Case 2: WireGuard VPN Configuration

Declare an encrypted WireGuard tunnel cleanly in NixOS:

```

# wireguard.nix
{ pkgs, ... }:

{
  networking.wireguard.interfaces.wg0 = {
    ips = [ "10.100.0.2/24" ];
    listenPort = 51820;
    privateKeyFile = "/etc/secrets/wireguard.key";

    peers = [
      {
        publicKey = "xTqWP5s6V4BDNtG3...";
        allowedIPs = [ "10.100.0.1/32" ];
        endpoint = "vpn.desk.corp:51820";
        persistentKeepalive = 25;
      }
    ];
  };
}

```

The interface, encryption keys, and routing are brought up automatically by systemd upon boot.

The trap

Disabling the firewall globally (`networking.firewall.enable = false;`) during development troubleshooting and forgetting to re-enable it before deploying to production.

The fix: Open specific development ports explicitly with `networking.firewall.allowedTCPPorts = [ 3000 ];`.

The boring rule

- Always keep `networking.firewall.enable = true;`.
- Use NetworkManager on workstations; use systemd-networkd on servers.
- Keep secret keys (like WireGuard private keys) in external files referenced by path (`privateKeyFile`), never hardcoded in Nix expressions.

Try this

1. In `networking_config.nix`, add `networking.firewall.trustedInterfaces = [ "lo" "tailscale0" ]`; to trust loopback and VPN traffic while blocking public interfaces.

Users, Groups, and SSH Configuration

Users, Groups, and SSH Configuration

User administration on mutable systems is vulnerable to orphaned accounts and forgotten SSH keys. The boring default of NixOS is: `users.mutableUsers = false` — **define all user accounts, group memberships, and authorized SSH keys strictly in code, completely eliminating untracked accounts.**

Mental model

In traditional Linux: - `/etc/passwd` and `/etc/shadow` are mutated by `useradd` and `passwd`. - If an employee leaves the company, an administrator must remember to log into every server and delete their account.

In NixOS: 1. **Declarative Accounts:** Users are declared in `users.users.<name>`. 2. **Immutable Passwords & Users (`users.mutableUsers = false`):** When set to false, `/etc/passwd` is rewritten completely from your Nix configuration on every `nixos-rebuild`. Any user account removed from Git is purged from the machine instantly. 3. **Authorized Keys:** Public keys are declared directly in code (`openssh.authorizedKeys.keys = [ "ssh-ed25519 ..." ]`);).

```
users.users.alice = {
  isNormalUser = true;
  extraGroups = [ "wheel" ];
  openssh.authorizedKeys.keys = [ "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5..." ];
};
```

Worked examples

Case 1: Declarative Team Access and Hardened SSH

Save as `users_ssh.nix`.

```
# users_ssh.nix
{ pkgs, ... }:

{
  # Forbid imperative user additions (useradd/passwd)
  users.mutableUsers = false;

  # Define system operators
```

```

users.users.deployer = {
  isNormalUser = true;
  description = "CI/CD Deployment Service Account";
  extraGroups = [ "wheel" "systemd-journal" ];
  openssh.authorizedKeys.keys = [
    "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleDeployerKeyCorp"
  ];
};

# SSH Daemon Configuration
services.openssh = {
  enable = true;
  settings = {
    # Security hardening defaults
    PasswordAuthentication = false;
    KbdInteractiveAuthentication = false;
    PermitRootLogin = "no";
    X11Forwarding = false;
  };
};

# Passwordless sudo for specific group members
security.sudo.wheelNeedsPassword = false;
}

```

Key guarantees: - Password authentication over SSH is disabled; only Ed25519/RSA public keys are accepted. - Root login is disabled. - Only accounts declared in code exist on the machine.

Case 2: Provisioning Password Hashes Safely

For non-SSH console logins, supply a password hash or reference a runtime secret file:

```

# password_hash.nix
{
  users.users.operator = {
    isNormalUser = true;
    # Hashed with mkpasswd -m sha-512
    hashedPassword = "$6$rounds=500000$salt$...";
    # Or reference a secret file:
    # hashedPasswordFile = "/etc/secrets/operator.passwd";
  };
}

```

Never store plaintext passwords in `.nix` files, because `/nix/store` is world-readable by any local user.

The trap

Hardcoding plaintext passwords directly in `configuration.nix` (`initialPassword = "secret"`). Because all `.nix` files imported by your system configuration are copied into `/nix/store`, any user with shell access can read the password from `/nix/store`.

The fix: Use `hashedPasswordFile` pointing to a file with 0400 root permissions, or manage passwords via SOPS-nix.

The boring rule

- Set `users.mutableUsers = false;` on production servers.
- Disable SSH password authentication (`PasswordAuthentication = false`).
- Use `hashedPasswordFile` or SSH keys; never commit plaintext passwords.

Try this

1. Generate a secure modern password hash using `mkpasswd -m sha-512` and inspect the formatted output string.

Hardware Configuration and Drivers

Hardware Configuration and Drivers

Hardware detection and proprietary drivers are a major pain point on desktop and server Linux. The boring default of NixOS is: **isolate physical disk mappings into `hardware-configuration.nix`, use community-tested profiles from `nixos-hardware`, and pin graphics/CPU microcode options declaratively.**

Mental model

NixOS separates machine identity from machine hardware: 1. **`hardware-configuration.nix`**: Contains filesystem UUIDs, kernel boot modules, and swap devices. Generated automatically via `nixos-generate-config`. 2. **Kernel Modules (`boot.initrd.availableKernelModules`)**: Storage drivers (`nvme`, `ahci`, `virtio`) needed to mount the root disk during early boot. 3. **Hardware Profiles (`nixos-hardware`)**: A community collection of tested NixOS modules for specific laptops (ThinkPad, Dell XPS, Framework) and cloud instances (AWS, Hetzner, GCP).

`configuration.nix` (Services, Users, Network, Firewall)

```
imports = [
  ./hardware-configuration.nix (Disks, UUIDs, initrd modules)
  inputs.nixos-hardware.nixosModules.framework-13-7040-amd
];
```

Unified system generation optimized for specific physical silicon

Worked examples

Case 1: Inspecting an Auto-Generated `hardware-configuration.nix`

Save as `hardware-configuration.nix`. A standard NVMe + UEFI machine setup.

```
# hardware-configuration.nix
{ config, lib, pkgs, modulesPath, ... }:

{
  imports = [ (modulesPath + "/installer/scan/not-detected.nix") ];

  boot.initrd.availableKernelModules = [ "nvme" "xhci_pci" "ahci" "usb_storage" ];
  boot.initrd.kernelModules = [ ];
```

```

boot.kernelModules = [ "kvm-amd" ];
boot.extraModulePackages = [ ];

# Mount root filesystem by persistent UUID
fileSystems."/ = {
    device = "/dev/disk/by-uuid/a1b2c3d4-e5f6-7890-abcd-1234567890ab";
    fsType = "ext4";
};

# Mount EFI boot partition
fileSystems."/boot" = {
    device = "/dev/disk/by-uuid/1234-ABCD";
    fsType = "vfat";
};

swapDevices = [
    { device = "/dev/disk/by-uuid/f9e8d7c6-b5a4-3210-fedc-ba9876543210"; }
];

# CPU microcode updates
hardware.cpu.amd.updateMicrocode = true;
}

```

Notice filesystems are referenced by immutable `/dev/disk/by-uuid/` paths rather than volatile device names like `/dev/sda`, preventing boot failures if drive enumeration order shifts.

Case 2: Proprietary GPU and OpenGL Drivers

Enable hardware-accelerated 3D graphics and compute cleanly:

```

# graphics.nix
{ config, pkgs, ... }:

{
    hardware.graphics = {
        enable = true;
        enable32Bit = true;
    };

    # For NVIDIA systems:
    # services.xserver.videoDrivers = [ "nvidia" ];
    # hardware.nvidia.modesetting.enable = true;
}

```

NixOS links the appropriate proprietary kernel modules and OpenGL/Vulkan ICD profiles automatically.

The trap

Referencing disk partitions by volatile device names like `/dev/sda1` in `fileSystems."/".` If a USB drive is plugged in on reboot or BIOS disk order re-indexes, `/dev/sda` can become `/dev/sdb`, leaving the system unbootable in an emergency shell.

The fix: Always use persistent `/dev/disk/by-uuid/...` or `/dev/disk/by-label/...` paths.

The boring rule

- Let `nixos-generate-config` manage `hardware-configuration.nix`.
- Always reference filesystems by UUID.
- Always enable CPU microcode updates (`hardware.cpu.intel.updateMicrocode` or `hardware.cpu.amd.updateMicrocode`).
- Use `nixos-hardware` modules for specific laptop and server models.

Try this

1. Run `blkid` on your system to inspect the persistent UUIDs of all physical storage partitions.

Declarative Disk Partitioning with Disko

Declarative Disk Partitioning with Disko

Traditional operating system installation guides instruct administrators to manually boot live USB media, calculate sector offsets with `gdisk` or `fdisk`, format partitions with `mkfs`, and type mount commands. In a declarative infrastructure stack, manual disk partitioning is an unpardonable snowflake step. The boring default is: **modeling block devices, partition tables, LUKS2 encryption containers, and ZFS/Btrfs filesystems entirely in Nix using Disko, and provisioning them remotely over SSH with nixos-anywhere.**

Mental model

1. **Disks as pure Nix modules:** Disko (`github:nix-community/disko`) treats physical and virtual disks as structured Nix attribute sets. You define partition types (ESP, LUKS, LVM, Btrfs, ZFS), mount points, and filesystem flags in code.
2. **Deterministic execution phases:**
 - Format mode (`disko --mode format`): Partitions the raw drive, generates cryptographic keys, and creates filesystems.
 - Mount mode (`disko --mode mount`): Mounts filesystems under `/mnt` in the exact required dependency sequence.
3. **Zero-touch bootstrapping (nixos-anywhere):** Run `nixos-anywhere --flake .#target root@host.ip` from your workstation. It connects over SSH to a standard Linux rescue environment (like Hetzner, AWS, or an installer ISO), kexecs a minimal NixOS memory system, runs Disko to format drives, copies your system closure, installs GRUB/systemd-boot, and reboots into your production NixOS machine.

Worked examples

Case 1: Complete Disko layout with LUKS2 and Btrfs subvolumes

Save as `disk-config.nix`:

```
# disk-config.nix
{
  disko.devices = {
    disk = {
      main = {
        type = "disk";
        device = "/dev/nvme0n1";
      }
    }
  }
}
```

```

content = {
  type = "gpt";
  partitions = {
    ESP = {
      priority = 1;
      name = "ESP";
      start = "1M";
      end = "1024M";
      type = "EF00";
      content = {
        type = "filesystem";
        format = "vfat";
        mountpoint = "/boot";
        mountOptions = [ "umask=0077" ];
      };
    };
  };
  luks = {
    size = "100%";
    content = {
      type = "luks";
      name = "cryptoed";
      settings.allowDiscards = true;
      passwordFile = "/tmp/luks.key"; # ephemeral during provisioning
      content = {
        type = "btrfs";
        extraArgs = [ "-f" ];
        subvolumes = {
          "/root" = {
            mountpoint = "/";
            mountOptions = [ "compress=zstd" "noatime" ];
          };
          "/nix" = {
            mountpoint = "/nix";
            mountOptions = [ "compress=zstd" "noatime" ];
          };
          "/persist" = {
            mountpoint = "/persist";
            mountOptions = [ "compress=zstd" "noatime" ];
          };
          "/swap" = {
            mountpoint = "/swap";
            swap.swapfile.size = "8G";
          };
        };
      };
    };
  };
};

```

```

    };
  };
};
};
}

```

Evaluating Disko output:

```
nix eval .#nixosConfigurations.desk-server.config.system.build.diskoScript
```

```
«derivation /nix/store/19w8...-disko-script.drv»
```

Case 2: Zero-touch deployment via nixos-anywhere

```
nixos-anywhere --flake .#desk-server --generate-hardware-config nixos-generate-config ./
```

```

[nixos-anywhere] Connected to rescue environment over SSH.
[nixos-anywhere] Running Disko partitioning script on /dev/nvme0n1...
[nixos-anywhere] Created GPT table, formatted ESP (/boot), opened LUKS container.
[nixos-anywhere] Formatted Btrfs subvolumes (@root, @nix, @persist, @swap).
[nixos-anywhere] Copying NixOS closure to /mnt...
[nixos-anywhere] Installing systemd-boot bootloader.
[nixos-anywhere] Installation complete. Rebooting host...

```

The host reboots directly into your production NixOS deployment without you ever opening an interactive terminal or console.

The trap

The trap is **hardcoding unpredictable predictable network device names (/dev/sda vs. /dev/nvme0n1 vs. /dev/vda) in cloud or virtualized hardware**. A change in hypervisor driver can reorder /dev/sda to /dev/vda, failing the partition run. Use disk IDs (/dev/disk/by-id/...) or parameterize the disk device attribute per host.

The boring rule

- Declare all disk layouts exclusively with Disko modules committed to your repository.
- Use **nixos-anywhere** for automated remote server provisioning over SSH.
- Keep disk configuration files modular and imported by the host's **configuration.nix**.

Try this

1. Test Disko formatting safely inside a disposable QEMU VM using `nix run github:nix-community/disko -- --mode mount ./disk-config.nix`.
2. Inspect the resulting mount table with `findmnt` inside the VM.

Ephemeral Roots and Impermanence

Ephemeral Roots and Impermanence

Over months of operation, standard Linux systems accumulate stale cache files, forgotten test binaries in `/usr/local/bin`, untracked systemd override files in `/etc`, and state drift. The boring default is: **erasing your root filesystem on every single boot using tmpfs as / or rolling back to a blank Btrfs/ZFS snapshot, preserving strictly declared paths under /persist with the impermanence module.**

Mental model

1. **Root is disposable:** When the machine boots, `/` is an in-memory `tmpfs` RAM filesystem (or a freshly wiped snapshot). When the system shuts down or reboots, all undeclared state in `/`, `/etc`, `/var`, `/tmp`, and `/home` vanishes completely.
2. **The Nix store is safe:** The Nix store (`/nix`) lives on its own dedicated persistent partition, untouched by root resets. All packages and configurations remain instantly cached.
3. **Explicit state contracts:** The only files that survive a reboot are those explicitly bound via bind-mounts or symlinks to `/persist` (such as SSH host keys, WireGuard keys, machine-id, and database files).
4. **Instant auditability:** If an attacker or errant script drops a rootkit into `/etc/cron.d` or `/bin`, a simple reboot destroys it. If something breaks after a reboot, you know instantly that a runtime state dependency was not declared.

Worked examples

Case 1: Impermanence module configuration

Save as `impermanence.nix`:

```
# impermanence.nix
{ inputs, ... }:

{
  imports = [ inputs.impermanence.nixosModules.impermanence ];

  # Root filesystem mounted as tmpfs in RAM
  fileSystems."/" = {
    device = "none";
    fsType = "tmpfs";
    options = [ "defaults" "size=4G" "mode=755" ];
  };
}
```

```

};

fileSystems."/persist" = {
  device = "/dev/mapper/encrypted";
  fsType = "btrfs";
  options = [ "subvol=persist" "compress=zstd" "noatime" ];
  neededForBoot = true;
};

# Declaratively declare state that MUST persist
environment.persistence."/persist" = {
  hideMounts = true;
  directories = [
    "/var/log"
    "/var/lib/nixos"
    "/var/lib/systemd/coredump"
  ];
  files = [
    "/etc/machine-id"
    "/etc/ssh/ssh_host_ed25519_key"
    "/etc/ssh/ssh_host_ed25519_key.pub"
    "/etc/ssh/ssh_host_rsa_key"
    "/etc/ssh/ssh_host_rsa_key.pub"
  ];
};
}

```

Checking symlink generation:

```
nix eval --raw .#nixosConfigurations.desk-server.config.environment.persistence."/persist".f
```

```
[ "/etc/machine-id", "/etc/ssh/ssh_host_ed25519_key", ... ]
```

Case 2: Verification of state destruction

1. Create an untracked file:

```
sudo touch /etc/untracked-state.txt
```

2. Reboot the system:

```
sudo reboot
```

3. After boot:

```
ls -l /etc/untracked-state.txt
```

```
ls: cannot access '/etc/untracked-state.txt': No such file or directory
```

State that was not declared in `impermanence.nix` is deleted. The system remains as pure as the day it was built.

The trap

The catastrophic trap is **forgetting to persist `/etc/ssh/ssh_host_ed25519_key` and `/etc/machine-id`**. If the SSH host key is erased on boot, SSH clients will reject connections with `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!`. If `machine-id` is erased, `systemd` journal logs lose historical continuity. Always persist SSH host keys and `machine-id`.

The boring rule

- Mount `/` on `tmpfs` or wipe a root subvolume snapshot at boot.
- Store strictly declared persistent data under `/persist`.
- Bind `/etc/machine-id` and SSH host keys to persistent storage before enabling ephemeral roots.

Try this

1. Configure an ephemeral root NixOS VM in QEMU.
2. Verify that creating a file in `/root` or `/etc` disappears cleanly after `reboot`.

Home Manager

Home Manager and the Module System

Home Manager and the Module System

Configuring tools per user without root permissions is the core role of Home Manager. The boring default is: **manage user-space utilities via `home.packages` and service configurations via `typed programs.<name> options`.**

Mental model

Home Manager applies the exact same NixOS module system to non-root user environments:

1. `home.packages`: Installs packages into your user environment (`~/.nix-profile`).
2. `programs.<name>.enable = true;`: Configures shell, editors, and CLI tools, automatically placing dotfiles into `~/.config/`.
3. Standalone vs NixOS module: Can run as a standalone CLI (`home-manager switch`) on any Linux/macOS distro, or as a built-in submodule inside `configuration.nix`.

Worked examples

Save as `home.nix`:

```
# home.nix
{ pkgs, ... }:

{
  home.username = "deskadmin";
  home.homeDirectory = "/home/deskadmin";
  home.stateVersion = "24.11";

  home.packages = with pkgs; [
    bat
    fzf
    jq
  ];

  programs.bash.enable = true;
}
```

Run:

```
home-manager switch
```

Output:

```
Starting Home Manager activation
Creating symlinks in /home/deskadmin...
Home Manager switch complete!
```

The boring rule

- Use Home Manager for dotfiles and user tools.
- Reserve `system configuration.nix` for kernel, hardware, daemons, and system users.

Managing Dotfiles Declaratively

Managing Dotfiles Declaratively

Managing dotfiles with Git bare repositories or symlink scripts (`stow`) leaves dead links and drift. The boring default is: use `home.file."<path>".source` or `xdg.configFile` to link configuration files directly from the immutable `/nix/store`.

Mental model

Home Manager manages files in `$HOME`: - `xdg.configFile."nvim/init.lua".source = ./nvim/init.lua`; - Generates a symlink `~/.config/nvim/init.lua -> /nix/store/...-init.lua`. - Files in `/nix/store` are read-only: prevents accidental in-place mutation without committing to Git.

Worked examples

Save as `dotfiles.nix`:

```
# dotfiles.nix
{
  xdg.configFile."desk/settings.json".text = builtins.toJSON {
    theme = "dark";
    autoSync = true;
  };
}
```

Home Manager renders JSON, YAML, and INI configuration files purely from Nix data structures.

The boring rule

- Use `xdg.configFile` for modern XDG-compliant configuration paths.
- Store configuration templates directly in Git alongside your Nix files.

Shell and Editor Configuration

Shell and Editor Configuration

Setting up Zsh, Fish, Bash, and editors on new machines usually requires copying snippets and installing plugins by hand. The boring default is: **declare shell aliases, plugins, prompts, and editor settings natively in Home Manager.**

Mental model

Home Manager provides high-level modules for major tools: - `programs.zsh`: Configures completions, history size, and plugins. - `programs.starship`: Cross-shell prompt configured declaratively. - `programs.git`: Declares name, email, signing keys, and global ignore lists.

Worked examples

Save as `shell_editor.nix`:

```
# shell_editor.nix
{
  programs.git = {
    enable = true;
    userName = "Desk Engineer";
    userEmail = "desk@corp.internal";
    extraConfig = {
      init.defaultBranch = "main";
      pull.rebase = true;
    };
  };

  programs.starship = {
    enable = true;
    settings = {
      add_newline = false;
      format = "$directory$character";
    };
  };
}
```


The boring rule

- Configure Git, shell, and prompts through typed `programs.*` options.
- Avoid manual plugin managers; let Nix fetch and lock shell plugins.

Application Settings and Preferences

Application Settings and Preferences

Desktop applications and developer tools can have their settings managed alongside terminal dotfiles. The boring default is: **use Home Manager application modules for VSCode, Alacritty, Kitty, and browser policies.**

Mental model

Home Manager compiles application configuration settings into native format files: - `programs.alacritty`: Writes TOML configuration files. - `programs.vscode`: Manages extensions and `settings.json` declaratively.

Worked examples

Save as `apps.nix`:

```
# apps.nix
{ pkgs, ... }:

{
  programs.alacritty = {
    enable = true;
    settings = {
      window.opacity = 0.95;
      font.size = 12.0;
    };
  };
}
```

When activated, Alacritty picks up the settings automatically.

The boring rule

- Declare terminal emulator and desktop tool preferences in Home Manager.
- Ensure extensions and plugins are version-controlled via Nix.

Combining NixOS and Home Manager

Combining NixOS and Home Manager

Managing NixOS system configurations and Home Manager user environments separately creates two different rebuild workflows. The boring default is: **import Home Manager as a NixOS module inside configuration.nix** so a single `nixos-rebuild switch` updates the OS, system services, and user dotfiles atomically.

Mental model

By importing `home-manager.nixosModules.home-manager`: - System configuration and user environments share the same evaluation. - `nixos-rebuild switch` updates both the system packages and all user dotfiles in one command. - System rollbacks automatically restore both system state and user dotfile state in unison.

```
configuration.nix
    Systemd services, kernel, hardware
    home-manager.users.deskuser = {
        # User dotfiles, git settings, personal shell
    };
```

Worked examples

Save as `unified.nix`:

```
# unified.nix
{ config, pkgs, ... }:

{
    # In your NixOS configuration.nix:
    # imports = [ <home-manager/nixos> ];

    home-manager.useGlobalPkgs = true;
    home-manager.useUserPackages = true;

    home-manager.users.deskadmin = { pkgs, ... }: {
        home.stateVersion = "24.11";
        programs.git.enable = true;
    };
}
```

Setting `useGlobalPkgs = true;` prevents Home Manager from evaluating a second copy of `nixpkgs`, speeding up evaluation and saving disk space.

The boring rule

- Integrate Home Manager into NixOS using `home-manager.nixosModules.home-manager`.
- Set `home-manager.useGlobalPkgs = true;`.
- Enjoy single-command atomic updates for host OS and user space simultaneously.

CI/CD with Nix

Setting Up Binary Caches

Setting Up Binary Caches

Compiling software from source on every developer laptop and CI runner wastes hours of CPU time. The boring default is: **a shared binary cache (Cachix, Attic, or Harmonia) that stores pre-compiled store paths cryptographically signed with public keys.**

Mental model

Nix does not compile packages if the pre-built output is already present in a trusted binary cache. 1. When Nix evaluates `/nix/store/<hash>-package`, it queries configured **substituters**. 2. If the cache possesses that hash signed with a matching public key, it downloads the pre-built `.narinfo` and `.nar` archive instead of compiling. 3. If built locally or in CI, the builder signs the artifact with a private key and uploads it to the cache.

Worked examples

Save as `cache_config.nix`:

```
# cache_config.nix
{
  nix.settings = {
    substituters = [
      "https://cache.nixos.org"
      "https://desk-cache.cachix.org"
    ];
    trusted-public-keys = [
      "cache.nixos.org-1:6NCHdD59X431oOgWypbMrAURkbJ16ZPMQFGspcDShjY="
      "desk-cache.cachix.org-1:a1b2c3d4e5f6..."
    ];
  };
}
```

Nix queries `desk-cache` before falling back to compilation.

The boring rule

- Every team should share a private or managed binary cache.
- Sign all uploaded artifacts with an explicit cryptographic keypair.

GitHub Actions with Nix

GitHub Actions with Nix

CI workflows often suffer from slow tool installations and flaky runner environments. The boring default is: use `cachix/install-nix-action` and `cachix/cachix-action` in GitHub Actions to build and test directly inside hermetic Nix sandboxes.

Mental model

Traditional GitHub Actions workflows install tools using multiple action plugins (`actions/setup-go`, `actions/setup-python`, `actions/setup-node`).

With Nix: - A single action installs Nix on the runner. - The workflow runs `nix build` or `nix flake check`. - Builds and test outputs are cached to your binary cache, turning 15-minute CI pipelines into 30-second cache downloads.

Worked examples

Save as `ci.yml`:

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
        with:
          nix_path: nixpkgs=channel:nixos-24.11
      - uses: cachix/cachix-action@v15
        with:
          name: desk-cache
          authToken: ${ secrets.CACHIX_AUTH_TOKEN }
      - name: Check flake and build
        run: nix flake check
```

The boring rule

- Standardize CI on Nix to match local developer environments byte-for-byte.
- Cache build outputs automatically in CI.

GitLab CI Integration

GitLab CI Integration

Self-hosted and cloud GitLab CI runners integrate cleanly with Nix. The boring default is: **use the official `nixos/nix` Docker image on GitLab runners, mount the Nix store cache, and execute pure evaluations.**

Mental model

GitLab CI jobs execute inside container images: 1. Use image `nixos/nix:latest`. 2. Enable experimental features (`nix-command flakes`). 3. Run `nix build` or `nix develop --command`.

Worked examples

Save as `.gitlab-ci.yml`:

```
# .gitlab-ci.yml
image: nixos/nix:latest

variables:
  NIX_CONFIG: "experimental-features = nix-command flakes"

stages:
  - test
  - build

test-job:
  stage: test
  script:
    - nix flake check

build-job:
  stage: build
  script:
    - nix build .#default
```

The boring rule

- Use `nixos/nix:latest` container image in GitLab CI.

- Configure `experimental-features = nix-command flakes`.

Build Caching Strategies

Build Caching Strategies

In automated CI environments, uncached builds destroy developer velocity and rack up huge compute bills. The boring default is: **layering Nix store substitution, GitHub Actions store caching with magic-nix-cache, and fine-grained derivation boundaries so that unchanged derivations never rebuild.**

Mental model

Build caching in Nix is fundamentally different from traditional file-based or Docker layer caching: 1. **Derivation identity:** A step only executes if its exact derivation hash (inputs, scripts, toolchain, environment variables) is missing from the local store and upstream substituters. 2. **Deterministic granularity:** Docker caches by whole filesystem layers; if step 3 changes, steps 4–10 bust. Nix evaluates a directed acyclic graph (DAG): changing a backend package does not bust cache for the frontend or documentation derivations. 3. **Cache layers:** Local runner store -> CI platform cache (GitHub Actions Cache API via `magic-nix-cache`) -> upstream persistent binary cache (Cachix, Attic, or S3 bucket).

Worked examples

Case 1: Minimal GitHub Actions cache configuration

Save as `.github/workflows/ci.yml`:

```
# .github/workflows/ci.yml
name: "Continuous Integration"
on:
  pull_request:
  push:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: DeterminateSystems/nix-installer-action@main
      - uses: DeterminateSystems/magic-nix-cache-action@main
      - name: Build Desk Service
```

```
run: nix build .#desk-service
```

Invocation and output:

```
nix build .#desk-service
```

```
[magic-nix-cache] Found 14 matching store paths in GitHub Actions Cache  
[magic-nix-cache] Substituted /nix/store/a81f9z...-desk-service in 1.2s  
Done.
```

Case 2: Isolating dependencies to prevent cache bust

Save as `package.nix`:

```
# package.nix  
{ pkgs ? import <nixpkgs> {} }:  
  
let  
  deps = pkgs.buildGoModule {  
    pname = "desk-service-deps";  
    version = "1.0.0";  
    src = ./go.mod;  
    vendorHash = "sha256-47DEQpj8HBSa+/TImW+5JCeuQeRkm5NMpJWZG3hSuFU=";  
  };  
in  
pkgs.buildGoModule {  
  pname = "desk-service";  
  version = "1.0.0";  
  src = pkgs.lib.cleanSource ./.;  
  vendorHash = null;  
}
```

When developers edit business logic (`main.go`), the vendor derivation remains untouched in cache.

The trap

The most common trap is **importing the entire repository root into the derivation source** (`src = ./.`) when compiling sub-components. A single change to `README.md` or a git tag invalidates the source hash and busts the entire compilation cache. Always filter source paths using `pkgs.lib.cleanSource` or `nix-gitignore`.

The boring rule

- Always use `magic-nix-cache` for zero-configuration GitHub Actions caching.

- Filter repository sources so documentation, test fixtures, and scripts do not bust build derivations.
- Separate heavy dependencies from fast-changing business logic into distinct derivations.

Try this

1. Run `nix path-info -r ./result` on a build artifact to observe the full tree of store paths eligible for binary caching.
2. In a test flake, add a clean source filter to `src` and verify that modifying a markdown file does not re-trigger `nix build`.

Testing in CI Pipelines

Testing in CI Pipelines

Running automated tests across different environments frequently breaks due to hidden state, unpinned libraries, or missing database binaries. The boring default is: **writing Nix derivations with explicit check phases or NixOS integration tests that run in disposable QEMU VMs without external infrastructure.**

Mental model

1. **Unit tests in checkPhase:** A derivation can execute tests during its build lifecycle (`doCheck = true`). Because the sandbox has no internet access, unit tests must be hermetic and rely solely on declared inputs.
2. **NixOS VM integration tests:** NixOS provides a Python-scripted test driver that boots full multi-node virtual machines in memory, configures network bridges, starts real systemd services, and validates end-to-end integration before deployment.
3. **Flake checks:** The `checks.<system>` flake output bundles all unit tests, linters, and VM tests into a single command: `nix flake check`.

Worked examples

Case 1: Package with hermetic checkPhase

Save as `service.nix`:

```
# service.nix
{ pkgs ? import <nixpkgs> {} } :

pkgs.buildGoModule {
  pname = "desk-service";
  version = "1.0.0";
  src = pkgs.lib.cleanSource ./.;
  vendorHash = null;

  doCheck = true;
  checkPhase = ''
    echo "Running unit test suite..."
    go test -v ./...
  '';
}
```

Invocation and output:

```
nix-build service.nix
```

Running unit test suite...

```
=== RUN    TestDeskQueueProcessing
```

```
--- PASS: TestDeskQueueProcessing (0.02s)
```

```
PASS
```

```
ok        desk/service      0.024s
```

```
/nix/store/7xq3m4...-desk-service-1.0.0
```

Case 2: Flake check definition

Save as flake.nix:

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in {
      packages.${system}.default = pkgs.callPackage ./service.nix {};

      checks.${system} = {
        unit-tests = self.packages.${system}.default;
        linter = pkgs.runCommand "lint-check" {
          nativeBuildInputs = [ pkgs.golangci-lint ];
        } ''
          cd ${self}
          golangci-lint run
          touch $out
        '';
      };
    };
}
```

Running in CI:

```
nix flake check --print-build-logs
```

evaluating flake checks...

```
building '/nix/store/z11...-lint-check.drv'...
```

```
building '/nix/store/x22...-desk-service-1.0.0.drv'...
```

```
[PASS] unit-tests  
[PASS] linter
```

The trap

The trap is **attempting to make live network calls during `checkPhase`**. Since the Nix build sandbox disables network access by default, commands like `npm test` or `go test` that download dependencies or ping remote APIs will immediately fail with connection errors. All fixtures and mocks must be hermetic and bundled inside the repository or derivation inputs.

The boring rule

- Always wire test suites into derivation `checkPhase` or `checks.<system>`.
- Run `nix flake check` as the primary validation gate in pull requests.
- Rely on NixOS VM tests when you need multi-service or database integration testing.

Try this

1. Add a dummy test case to your project and verify that `nix flake check` fails and displays the failure log when an assertion fails.
2. Verify that network calls inside `checkPhase` fail hermetically in sandbox mode.

Secrets Management in CI

Secrets Management in CI

Nix store paths are globally readable on multi-user systems, and hardcoding secrets in Nix expressions embeds them permanently into world-readable `/nix/store` hashes. The boring default is: **never store plaintext secrets in Nix code; inject decrypted secrets only at runtime via environment variables or ephemeral files provided by CI secret managers.**

Mental model

1. **The store is public:** Anything written into a `.nix` file, passed as a string literal, or included as a file in derivation `src` ends up in the world-readable Nix store.
2. **Build time vs. Runtime:** Hermetic builds should almost never require secrets. If a build requires a private git repository or private registry, pass authentication via `access-tokens` in `nix.conf` or SSH agent forwarding, not inside derivations.
3. **Decryption at deploy time:** CI runners use GitHub Secrets or Vault to access private keys (such as Age private keys for `sops-nix`), allowing runtime decryption on the target system.

Worked examples

Case 1: Providing GitHub token for private Flake inputs

Save as `.github/workflows/deploy.yml`:

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: ["main"]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: DeterminateSystems/nix-installer-action@main
        with:
          extra-conf: |
            access-tokens = github.com=${{ secrets.GH_PAT_FOR_PRIVATE_MODULES }}
      - name: Deploy System
```

```
run: nix run .#deploy
```

Case 2: Supplying Age decryption key to sops-nix in CI tests

Save as `test-secrets.sh`:

```
# test-secrets.sh
export SOPS_AGE_KEY="${ secrets.CI_AGE_PRIVATE_KEY }"
nix eval .#nixosConfigurations.desk-server.config.sops.secrets
```

Invocation and output:

```
bash test-secrets.sh
```

```
{
  db-password = { path = "/run/secrets/db-password"; key = "db_pass"; };
  jwt-secret = { path = "/run/secrets/jwt-secret"; key = "jwt"; };
}
```

The Nix expression only references the *path* where the secret will be mounted, leaving secret bytes completely out of the Nix store.

The trap

The catastrophic trap is doing `builtins.readFile ./secrets/prod.key` or passing secret strings into derivation `environmentVariables`. Nix copies the entire contents of files read by `builtins.readFile` into `/nix/store`, broadcasting private tokens to anyone with read access to the cache or target machine.

The boring rule

- Never evaluate or read raw secret files with `builtins.readFile` in derivations.
- Store secrets encrypted in git with `sops-nix` or `agenix`, decrypting only on target nodes at boot.
- Pass repository access tokens exclusively through `nix.conf` (`access-tokens`).

Try this

1. Run `nix-instantiate --eval -E 'builtins.readFile ./test.txt'` and inspect the generated store path to see how Nix persists files into the store.
2. Confirm that running `nix flake check` does not require live production credentials.

Containers and Kubernetes

Creating Container Images with Nix (dockerTools)

Creating Container Images with Nix (dockerTools)

Traditional Dockerfiles rely on non-deterministic base images (`FROM debian:latest`), mutate filesystem layers imperatively with `apt-get`, and leave behind build bloat, package caches, and CVE vulnerabilities. The boring default is: **building minimal, bit-for-bit reproducible OCI container images directly with `pkgs.dockerTools.buildLayeredImage` without running a Docker daemon.**

Mental model

1. **Daemonless image building:** Nix packages store closures directly into tarballs conforming to the OCI image specification. No `dockerd`, root privileges, or containers are needed to build the image.
2. **Deterministic contents:** The container image contains strictly the package binaries and their exact closure graph under `/nix/store`. There are no extra utilities, package managers (`apt`, `apk`), or extraneous caches unless declared.
3. **Layer caching:** `dockerTools.buildLayeredImage` automatically splits dependencies into distinct tarball layers based on change frequency: `libc` and core libraries stay in bottom layers, while your fast-changing application binary sits in the top layer.

Worked examples

Case 1: Minimal Go service container with `buildLayeredImage`

Save as `image.nix`:

```
# image.nix
{ pkgs ? import <nixpkgs> {} }:

let
  deskApp = pkgs.buildGoModule {
    pname = "desk-api";
    version = "1.0.0";
    src = pkgs.lib.cleanSource ../.;
    vendorHash = null;
  };
in
```

```
pkgs.dockerTools.buildLayeredImage {
  name = "desk-api";
  tag = "latest";

  contents = [ deskApp pkgs.cacert ];

  config = {
    Cmd = [ "${deskApp}/bin/desk-api" ];
    ExposedPorts = {
      "8080/tcp" = {};
    };
    Env = [
      "SSL_CERT_FILE=${pkgs.cacert}/etc/ssl/certs/ca-bundle.crt"
    ];
  };
}
```

Building the image tarball:

```
nix-build image.nix
```

```
[1/2 built] building desk-api.tar.gz...
Created 12 layers for desk-api
/nix/store/3x91...-desk-api.tar.gz
```

Loading into Docker or Podman:

```
docker load < result
docker run -p 8080:8080 desk-api:latest
```

```
Loaded image: desk-api:latest
2026-09-08T18:00:00Z [INFO] Desk API server listening on :8080
```

The trap

The common trap is **forgetting certificates or timezone data** in minimal images. Because Nix images contain only declared closures, an HTTPS request will fail with `x509: certificate signed by unknown authority` unless `pkgs.cacert` is added to `contents` and `SSL_CERT_FILE` is configured.

The boring rule

- Always use `pkgs.dockerTools.buildLayeredImage` over `buildImage` for optimal layer reuse.

- Include `pkgs.cacert` whenever the container makes outbound TLS/HTTPS network calls.
- Build images in CI directly on headless runners without running a root Docker daemon.

Try this

1. Build a minimal container image for a CLI utility and inspect the generated tarball with `tar -tf result` to verify the layer layout.
2. Load the image into Docker/Podman and run it with `--rm` to verify that only declared binaries exist inside the container.

Kubernetes Manifests as Nix Modules

Kubernetes Manifests as Nix Modules

Raw YAML manifests lead to severe duplication across environments (dev, staging, prod), error-prone copy-pasting, and silent syntax mistakes. The boring default is: **modeling Kubernetes workloads as typed Nix attribute sets and generating clean, validated YAML manifests through functions.**

Mental model

1. **Nix expressions as manifest generators:** Kubernetes objects are JSON-compatible nested dictionaries. Nix natively evaluates attribute sets and converts them directly to JSON or YAML using `builtins.toJSON` or `pkgs.formats.yaml`.
2. **Typed abstractions:** Instead of copy-pasting 80 lines of Deployment and Service boilerplate, write a reusable Nix function that accepts image, port, and replicas, returning a compliant manifest.
3. **Environment specialization:** Share 95% of configuration across clusters while overriding specific values (replicas, ingress domains, resource limits) cleanly using Nix attribute set merging (`//`).

Worked examples

Case 1: Reusable workload generator

Save as `manifests.nix`:

```
# manifests.nix
{ pkgs ? import <nixpkgs> {} }:

let
  mkDeployment = { name, image, port ? 8080, replicas ? 2 }: {
    apiVersion = "apps/v1";
    kind = "Deployment";
    metadata = {
      name = name;
      labels.app = name;
    };
    spec = {
      replicas = replicas;
      selector.matchLabels.app = name;
    };
  };
in
```

```

template = {
  metadata.labels.app = name;
  spec.containers = [
    {
      name = name;
      image = image;
      ports = [ { containerPort = port; } ];
    }
  ];
};

deskDeployment = mkDeployment {
  name = "desk-service";
  image = "registry.company.internal/desk:v1.2.0";
  replicas = 3;
};

in
pkgs.writeText "desk-deployment.json" (builtins.toJSON deskDeployment)

```

Generating the manifest:

```

nix-build manifests.nix
cat result | jq .spec.replicas

```

3

Applying to cluster:

```

kubectl apply -f $(nix-build manifests.nix)

```

deployment.apps/desk-service configured

The trap

The trap is **generating YAML with string interpolation rather than native Nix data structures**. String templates invite indentation mistakes and escaping bugs. Always build native Nix attribute sets and serialize them with `builtins.toJSON` (Kubernetes accepts JSON as valid YAML natively).

The boring rule

- Define Kubernetes resources as pure Nix attribute sets.
- Generate JSON directly; `kubectl` parses JSON natively without needing third-party YAML converters.
- Parameterize environment differences through explicit Nix function arguments.

Try this

1. Define a Nix function that produces both a Deployment and matching Service resource from a single parameter set.
2. Verify the generated output with `kubectl --dry-run=client -f $(nix-build manifests.nix)`.

Helm Charts Generated from Nix

Helm Charts Generated from Nix

Helm charts introduce complex Go text templating, fragile whitespace semantics, and disconnected variable scopes. The boring default is: **rendering Helm values and chart templates deterministically using Nix derivation pipelines or wrapping helm package workflows in reproducible sandboxes.**

Mental model

1. **Predictable values:** Helm uses `values.yaml` files to parameterize charts. Instead of maintaining sprawling, unvalidated YAML hierarchies, generate values files with Nix functions that enforce schemas and cross-reference store paths.
2. **Chart packaging in derivations:** Package custom Helm charts as Nix derivations that run `helm lint` and `helm package` in isolated sandboxes, producing cryptographically hashed `.tgz` chart packages.
3. **Declarative Helmfile / Chartfile:** Bundle upstream Helm charts as Nix flake inputs, ensuring third-party charts cannot change unpredictably under deployment pipelines.

Worked examples

Case 1: Packaging a Helm chart with Nix

Save as `chart.nix`:

```
# chart.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.stdenv.mkDerivation {
  pname = "desk-chart";
  version = "0.1.0";
  src = ./chart;

  nativeBuildInputs = [ pkgs.kubernetes-helm ];

  buildPhase = ''
    helm lint .
    helm package . -d $out
  '';
}
```

Building the chart archive:

```
nix-build chart.nix
ls result/
```

```
==> Linting .
[INFO] Chart.yaml: icon is recommended
1 chart(s) linted, 0 chart(s) failed
desk-chart-0.1.0.tgz
```

The trap

The trap is **depending on remote Helm repositories that update charts in place or suffer network downtime during deployments**. Always vendor chart archives or pin them with fixed-output hashes in your flake inputs.

The boring rule

- Validate and package charts inside hermetic Nix derivations using `helm lint`.
- Generate complex `values.yaml` files from Nix attribute sets to avoid text interpolation errors.
- Keep chart definitions version-pinned in the same repository as the deployment flakes.

Try this

1. Create a minimal chart directory with `Chart.yaml` and `values.yaml`.
2. Build it into a `.tgz` artifact using the Nix derivation above.

GitOps with Argo CD and Flux

GitOps with Argo CD and Flux

GitOps engines like Argo CD and Flux ensure clusters match git state, but letting them run arbitrary, unpinned build plugins leads to drifting environments and unreproducible deployments. The boring default is: **a two-stage pipeline where Nix evaluates and renders full static manifests into a deployment branch or repo, which Argo CD or Flux syncs without needing Nix in the cluster.**

Mental model

1. **Pre-render in CI:** Evaluating Nix inside a Kubernetes pod on every sync loop requires giving the cluster Nix daemon privileges, CPU, and storage. Instead, evaluate in CI and commit flat manifests to the GitOps sync branch.
2. **Clean separation of concerns:**
 - Development repo: Nix expressions, source code, tests, container definitions.
 - CI runner: builds images, evaluates manifests, pushes images to registry, updates GitOps repository.
 - Argo CD / Flux: Watches flat GitOps repo, syncs directly to Kubernetes API.
3. **Instant rollbacks:** Every commit in the deployment branch represents the exact state applied to the cluster, enabling instant `git revert`.

Worked examples

Case 1: CI workflow rendering manifests for Argo CD

Save as `render-gitops.sh`:

```
# render-gitops.sh
set -euo pipefail

echo "Evaluating Kubernetes manifests via Nix..."
nix-build manifests.nix -o rendered-manifests.json

mkdir -p gitops-repo/apps/desk-service
cp rendered-manifests.json gitops-repo/apps/desk-service/deployment.json

cd gitops-repo
git add apps/desk-service/
```



```
git commit -m "Deploy desk-service commit ${GITHUB_SHA:-manual}"
echo "Manifest ready for Argo CD sync."
```

Execution output:

```
bash render-gitops.sh
```

```
Evaluating Kubernetes manifests via Nix...
/nix/store/7wq1...-desk-deployment.json
Manifest ready for Argo CD sync.
```

Case 2: Argo CD Application definition

Save as `argo-application.yaml`:

```
# argo-application.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: desk-service
  namespace: argocd
spec:
  project: default
  source:
    repoURL: 'https://github.com/company/gitops-deployments.git'
    targetRevision: main
    path: apps/desk-service
  destination:
    server: 'https://kubernetes.default.svc'
    namespace: desk
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

The trap

The trap is **installing Nix into the Argo CD repo-server as a custom plugin**. This slows sync cycles, introduces in-cluster compilation risks, and causes synchronization timeouts when cache hits miss. Keep cluster controllers simple consumers of static files.

The boring rule

- Evaluate and validate all manifests in CI before committing to GitOps repos.
- Keep Argo CD / Flux configurations strictly standard and declarative.
- Store rendered artifacts in dedicated deployment repositories or branches.

Try this

1. Run the render script locally to produce a flat JSON manifest from your Nix module.
2. Validate that the rendered file can be directly applied to a local Minikube or Kind cluster using `kubectl apply -f`.

Service Mesh and Observability

Service Mesh and Observability

Deploying microservice observability tooling (Prometheus, Grafana, OpenTelemetry collectors) typically involves divergent Helm charts and snowflake dashboards that fall out of sync with application code. The boring default is: **declaring monitoring agents, scrape targets, and Grafana dashboard JSON models directly alongside application Nix definitions.**

Mental model

1. **Co-located service definitions:** When a service is defined in Nix, its corresponding monitoring metadata (metrics port, alert thresholds, scraping interval) is declared in the same module.
2. **Deterministic dashboards:** Instead of clicking in the Grafana UI or maintaining sprawling dashboard exports, dashboard definitions are generated from structured Nix templates and provisioned automatically.
3. **Unified telemetry:** OpenTelemetry collector configuration files are generated by Nix with exact pipeline receivers, processors, and exporters.

Worked examples

Case 1: Generating OpenTelemetry Collector configuration

Save as `otel-collector.nix`:

```
# otel-collector.nix
{ pkgs ? import <nixpkgs> {} }:

let
  config = {
    receivers = {
      otlp.protocols = {
        grpc.endpoint = "0.0.0.0:4317";
        http.endpoint = "0.0.0.0:4318";
      };
    };
    processors = {
      batch.timeout = "1s";
    };
    exporters = {
```

```

    prometheus = {
      endpoint = "0.0.0.0:8889";
      namespace = "desk";
    };
  };
  service.pipelines.metrics = {
    receivers = [ "otlp" ];
    processors = [ "batch" ];
    exporters = [ "prometheus" ];
  };
};
in
pkgs.writeText "otel-config.yaml" (builtins.toJSON config)

```

Building and checking the configuration:

```

nix-build otel-collector.nix
cat result | jq .service.pipelines.metrics.exporters

```

```

[
  "prometheus"
]

```

The trap

The trap is **manually configuring alerts and scraping rules in third-party dashboards**. When service endpoints change, manual dashboards silently break without alerting anyone. By pairing service derivations with monitoring configurations in Nix, breaking changes are caught at build and evaluation time.

The boring rule

- Generate OpenTelemetry and Prometheus configurations from version-controlled Nix expressions.
- Couple service definitions with their metric endpoints in the same module.
- Provision Grafana dashboards via declarative JSON models committed to git.

Try this

1. Build the OpenTelemetry configuration artifact using `nix-build otel-collector.nix`.
2. Inspect the generated JSON/YAML to confirm pipeline bindings are structurally valid.

Infrastructure as Code

Managing Terraform Providers with Nix

Managing Terraform Providers with Nix

Standard Terraform setups dynamically download unpinned provider binaries from public registries during `terraform init`, causing unexpected breaking changes and supply-chain vulnerabilities. The boring default is: **packaging Terraform and its exact provider binaries directly with Nix's `terraform.withPlugins` wrapper.**

Mental model

1. **Pre-bundled plugins:** Nix provides a specialized function `pkgs.terraform.withPlugins` that bundles the Terraform CLI and pinned provider binaries into a single unified directory.
2. **Offline execution:** When running wrapped Terraform, provider binaries already reside under `/nix/store`. `terraform init` requires zero network access to download plugins.
3. **Reproducible team environments:** Every developer and CI pipeline runs the identical provider binary down to the exact compilation flags and linked libraries.

Worked examples

Case 1: Bundling Terraform with AWS and Cloudflare providers

Save as `terraform-env.nix`:

```
# terraform-env.nix
{ pkgs ? import <nixpkgs> {} }:

let
  myTerraform = pkgs.terraform.withPlugins (tp: [
    tp.aws
    tp.cloudflare
    tp.random
  ]);
in
pkgs.mkShell {
  packages = [ myTerraform ];
  shellHook = ''
    echo "Terraform environment ready with pinned providers:"
    terraform version
  '';
}
```


Entering the shell:

```
nix-shell terraform-env.nix --run "terraform version"
```

```
Terraform v1.8.4
on linux_amd64
+ provider registry.terraform.io/hashicorp/aws v5.49.0
+ provider registry.terraform.io/cloudflare/cloudflare v4.31.0
+ provider registry.terraform.io/hashicorp/random v3.6.1
```

Case 2: Running init without network access

```
nix-shell terraform-env.nix --run "terraform init -plugin-dir=\$NIX_TERRAFORM_PLUGIN_DIR"
```

```
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "5.49.0"...
- Installing hashicorp/aws v5.49.0... (from plugin directory)
Terraform has been successfully initialized!
```

The trap

The trap is **relying on terraform init to download arbitrary provider binaries in CI**. If HashiCorp registry rates-limits or changes a provider tag, CI pipelines fail abruptly. Using `withPlugins` guarantees complete isolation from registry outages.

The boring rule

- Always wrap Terraform with `pkgs.terraform.withPlugins`.
- Point `terraform init` to the pre-populated Nix store plugin path.
- Lock all provider versions explicitly in Nix expressions rather than relying on dynamic registry lookups.

Try this

1. Define a shell with `terraform.withPlugins` including `random` and `local` providers.
2. Run a simple plan offline to verify that provider binaries execute without network access.

OpenTofu Integration

OpenTofu Integration

OpenTofu provides a community-driven, truly open-source alternative to Terraform without proprietary licensing restrictions. The boring default is: **packaging OpenTofu with `pkgs.opentofu.withPlugins` to ensure fully reproducible, license-safe infrastructure automation.**

Mental model

1. **Drop-in compatibility:** OpenTofu maintains complete configuration and state compatibility with standard HCL syntax and provider protocols.
2. **Identical Nix packaging pattern:** `Nixpkgs` provides `opentofu.withPlugins`, following the exact same interface as `terraform.withPlugins`.
3. **Open registry resolution:** OpenTofu defaults to public open registries (`get.opentofu.org`), but when managed by Nix, providers are pulled directly from the local `/nix/store`.

Worked examples

Case 1: OpenTofu environment with Kubernetes provider

Save as `opentofu-shell.nix`:

```
# opentofu-shell.nix
{ pkgs ? import <nixpkgs> {} }:

let
  tofu = pkgs.opentofu.withPlugins (p: [
    p.kubernetes
    p.helm
    p.null
  ]);
in
pkgs.mkShell {
  packages = [ tofu ];
  shellHook = ''
    echo "OpenTofu environment loaded."
    tofu version
  '';
}
```

Execution and output:

```
nix-shell opentofu-shell.nix --run "tofu version"
```

```
OpenTofu v1.7.1
on linux_amd64
+ provider registry.opentofu.org/hashicorp/kubernetes v2.29.0
+ provider registry.opentofu.org/hashicorp/helm v2.13.1
+ provider registry.opentofu.org/hashicorp/null v3.2.2
```

The trap

The trap is **mixing unmanaged global tofu binaries with local provider directories**, which re-introduces version drift across machines. Enforce team-wide adoption via flake-based devShells.

The boring rule

- Standardize on `pkgs.opentofu.withPlugins` for open-source IaC toolchains.
- Distribute the toolchain through `flake.nix devShells` so every engineer runs the same OpenTofu version.
- Manage provider dependencies as declarative Nix inputs.

Try this

1. Replace `terraform` invocations with `tofu` in your test pipeline.
2. Verify that `tofu init` and `tofu plan` work seamlessly with Nix-supplied provider binaries.

Ansible Provisioning with Nix

Ansible Provisioning with Nix

Ansible playbooks frequently fail when Python dependencies (`boto3`, `requests`, `netaddr`) or Ansible collection versions differ between engineer machines and automation agents. The boring default is: **packaging Ansible with its exact Python libraries and community collections in a hermetic Nix environment**.

Mental model

1. **Python environment isolation:** Ansible is a Python application. Nix allows wrapping `pkgs.ansible` with specific Python packages required by your playbooks using `python3.withPackages`.
2. **Declarative collections:** Install Ansible collections into a predictable directory pinned in git or store paths rather than executing imperative `ansible-galaxy install` commands.
3. **Hybrid provisioning:** Use Ansible for fleet orchestration (e.g. bootstrapping machines, managing legacy hosts), while deploying immutable NixOS configurations as payloads.

Worked examples

Case 1: Ansible with cloud dependencies

Save as `ansible-env.nix`:

```
# ansible-env.nix
{ pkgs ? import <nixpkgs> {} }:

let
  ansibleWithDeps = pkgs.ansible.override {
    python3 = pkgs.python3.withPackages (ps: [
      ps.boto3
      ps.botocore
      ps.netaddr
      ps.paramiko
    ]);
  };
in
pkgs.mkShell {
  packages = [
    ansibleWithDeps
  ]
}
```

```

    pkgs.sshpass
    pkgs.openssh
];
}

```

Verifying the Python modules inside Ansible:

```
nix-shell ansible-env.nix --run "ansible --version"
```

```

ansible [core 2.16.5]
  config file = None
  configured module search path = ['/home/king/.ansible/plugins/modules', '/usr/share/ansible/p
  ansible python module location = /nix/store/...-ansible-2.16.5/lib/python3.11/site-packages/a
  executable location = /nix/store/...-ansible-2.16.5/bin/ansible
  python version = 3.11.9 (main) [/nix/store/...-python3-3.11.9/bin/python3]

```

The trap

The trap is **running `ansible-galaxy install -r requirements.yml` imperatively in CI**. If Ansible Galaxy is slow or down, deployment is blocked. Pin collection artifacts or bundle them directly in your repository.

The boring rule

- Always bundle required Python SDKs (`boto3`, `paramiko`) into the Ansible derivation.
- Check playbook syntax with `ansible-playbook --syntax-check` in `nix flake check`.
- Restrict Ansible to orchestration; let NixOS manage internal system state.

Try this

1. Launch the Ansible shell and verify that `python3 -c "import boto3; print(boto3.__version__)"` succeeds.
2. Run `ansible-lint` as a hermetic Nix check against your playbook.

Cloud Provider Configuration

Cloud Provider Configuration

Managing cloud CLI tools (`aws-cli`, `az`, `gcloud`) across multiple team members leads to mismatched API versions, conflicting credential helpers, and configuration drift. The boring default is: **declarative developer shells that package the exact cloud CLIs alongside configured credentials and session wrappers.**

Mental model

1. **Hermetic CLI bundles:** Nix packages official, reproducible cloud tooling without requiring system-level installers, Homebrew, or random shell installation scripts.
2. **Environment scoping:** Provide separate shells for different cloud environments (`desk-aws`, `desk-gcp`, `desk-azure`) where credentials and active profiles are clearly distinguished.
3. **No credentials in Nix store:** Cloud access tokens, AWS keys, and session cookies are loaded from user home directories (`~/.aws`) or hardware security keys, never stored in Nix code.

Worked examples

Case 1: Multi-cloud management shell

Save as `cloud-shell.nix`:

```
# cloud-shell.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  packages = [
    pkgs.awscli2
    pkgs.google-cloud-sdk
    pkgs.azure-cli
    pkgs.jq
  ];

  shellHook = ''
    echo "Cloud operations shell loaded:"
    aws --version
    az version --output json | jq -r '."azure-cli"'
  '';
```

```
}
```

Launching the shell:

```
nix-shell cloud-shell.nix --run "aws --version"
```

```
aws-cli/2.15.30 Python/3.11.8 Linux/6.6.0-generic exe/x86_64.nixos
```

The trap

The trap is **letting developers install cloud CLIs globally via `pip install awscli` or `curl ... | bash`**. These unversioned installs create subtle behavioral discrepancies during production incidents.

The boring rule

- Keep all cloud CLIs inside reproducible Nix shells.
- Never write credentials, access keys, or API tokens into `.nix` files.
- Use session tokens and SSO wrappers (like `aws sso login`) within the Nix shell.

Try this

1. Run `nix-shell cloud-shell.nix` and check CLI versions against your organization's minimum supported standards.
2. Test session authentication using a temporary read-only token.

Multi-Cloud Deployment Patterns

Multi-Cloud Deployment Patterns

Organizations deploying services across multiple cloud providers (AWS, GCP, Hetzner, Bare Metal) struggle with fragmented deployment toolchains and inconsistent OS images. The boring default is: **building unified NixOS machine images tailored for each target platform from a single shared base configuration.**

Mental model

1. **Single source of truth:** A core NixOS configuration defines users, security policies, and application services once.
2. **Target format specialization:** NixOS natively supports building target-specific disk images:
 - AWS: AMI (`amazon-image.nix`)
 - GCP: GCE image (`gce-image.nix`)
 - Proxmox / Bare Metal: QCOW2 or Raw ISO
3. **Identical runtime:** Regardless of whether the instance boots on AWS EC2 or an on-premise hypervisor, the running userspace, configuration files, and systemd services are identical byte-for-byte.

Worked examples

Case 1: Building a target-specific VM image

Save as `desk-image.nix`:

```
# desk-image.nix
{ pkgs ? import <nixpkgs> {} }:

let
  nixos = import <nixpkgs/nixos> {
    configuration = {
      imports = [
        <nixpkgs/nixos/modules/virtualisation/amazon-image.nix>
      ];

      # Shared service configuration
      services.openssh.enable = true;
    };
  }
```

```

        users.users.deploy = {
            isNormalUser = true;
            extraGroups = [ "wheel" ];
        };

        system.stateVersion = "24.05";
    };
};
in
nixos.config.system.build.amazonImage

```

Building the raw AMI image:

```

nix-build desk-image.nix
ls -la result/

```

nixos-amazon-image-24.05.vhd

The resulting virtual disk is ready for direct upload to Amazon S3 and registration as a private AMI.

The trap

The trap is **using Packer with shell provisioners to mutate golden images imperatively**. Imperative golden images accumulate non-deterministic state, untracked updates, and unrepeatable patches. Nix builds images from pure specifications from scratch in minutes.

The boring rule

- Share base system modules across all cloud and bare-metal environments.
- Use native NixOS build targets (`amazonImage`, `googleComputeImage`, `qemuImage`) to generate cloud artifacts.
- Deploy the same tested closure regardless of underlying infrastructure provider.

Try this

1. Evaluate `nixos.config.system.build.qemuImage` for a minimal NixOS test configuration.
2. Boot the resulting disk image in QEMU locally to confirm system services initialize cleanly.

Secrets and Security

Why Secrets Are Hard with Declarative Config

Why Secrets Are Hard with Declarative Config

Declarative configuration tools encourage committing entire system specifications into version control. In Nix, this creates a profound security challenge: **the Nix store is world-readable by design, meaning any secret placed in a derivation or `.nix` file is exposed to all local users and binary cache consumers.**

Mental model

1. **The Nix store design:** `/nix/store` permissions are 0755 (or 0555 for store items). Every user and unprivileged process on the host can read every path in the store.
2. **Build purity:** Builds cannot access network endpoints or prompt users for passphrases. A derivation must receive all inputs upfront.
3. **The separation of code and state:** The Nix expression defines *how* secrets are fetched and *where* they will be linked at runtime (e.g. `/run/secrets/api-token`), but the secret data itself must remain encrypted at rest and decrypted only in RAM or private runtime filesystems (`/run/`).

Worked examples

Case 1: Demonstrating store leakage (Anti-Pattern)

Consider this dangerous snippet:

```
# BAD: Secrets leaked to world-readable store!
pkgs.writeText "database.conf" ''
  password = "SuperSecretPassword123"
''
```

Evaluating and inspecting the store:

```
nix-build -E 'with import <nixpkgs> {}; writeText "database.conf" "password=SuperSecretPassw
cat result
```

```
password=SuperSecretPassword123
```

Notice that `result` resolves to `/nix/store/...-database.conf` which is readable by any process on the machine.

Case 2: The boring safe pattern (Reference by path)

```
# GOOD: Reference runtime path only
services.postgresql = {
  enable = true;
  initialScript = "/run/secrets/postgres-init.sql";
};
```

The actual secret file is mounted dynamically into `/run/secrets/` at boot time with `0400` permissions owned by the service user.

The trap

The trap is **assuming private git repositories protect plaintext secrets**. Once committed, secret strings end up in binary caches, CI build logs, developer backups, and `/nix/store` closures across every machine that evaluates the configuration.

The boring rule

- Never place plaintext secrets, API keys, or private SSH keys into `.nix` files or git.
- Keep the Nix store strictly public-safe: assume any file in `/nix/store` can be read by anyone.
- Decrypt secrets at boot time into private runtime filesystems (`tmpfs` under `/run/secrets`).

Try this

1. Search your existing configurations for hardcoded passwords or keys.
2. Verify that running `nix path-info --closure` on your system configuration never includes files containing plaintext secrets.

Using SOPS for Encrypted Configuration (sops-nix)

Using SOPS for Encrypted Configuration (sops-nix)

Storing secrets outside version control leads to out-of-band communication, missing environment variables, and synchronization headaches across machines. The boring default is: **encrypting secrets in-place in the git repository with Mozilla SOPS and mounting them into RAM at boot using sops-nix.**

Mental model

1. **Encrypted in git:** Secrets are stored in `.yaml` or `.json` files where only values are encrypted; keys remain visible for readability and git diffs.
2. **Decryption at boot:** `sops-nix` runs a systemd service during NixOS boot that reads the host's private Age key or SSH host key, decrypts the secrets, and writes them to a secure in-memory filesystem (`/run/secrets/`).
3. **No store leakage:** Decrypted secret files never touch `/nix/store` or the persistent disk.

Worked examples

Case 1: `.sops.yaml` configuration

Save as `.sops.yaml`:

```
# .sops.yaml
creation_rules:
- path_regex: secrets/.*\.yaml$
  key_groups:
  - age:
    - age1deskuser...
    - age1desknode01...
```

Encrypting the secrets file:

```
sops secrets/desk.yaml
```

The file contains encrypted ciphertext safe to commit to git.

Case 2: NixOS module integration

Save as `secrets-module.nix`:

```
# secrets-module.nix
{ config, pkgs, ... }:

{
  imports = [ <sops-nix/modules/nixos> ];

  sops = {
    defaultSopsFile = ./secrets/desk.yaml;
    defaultSopsFormat = "yaml";
    age.keyFile = "/var/lib/sops-nix/key.txt";

    secrets."database/password" = {
      owner = "postgres";
      mode = "0400";
    };
  };

  # Service reads secret from runtime path
  systemd.services.desk-service = {
    serviceConfig = {
      EnvironmentFile = config.sops.secrets."database/password".path;
    };
  };
}
```

System evaluation:

```
nix eval --raw .#nixosConfigurations.desk-server.config.sops.secrets."database/password".path
```

`/run/secrets/database/password`

The trap

The trap is **specifying incorrect file permissions or owners** for secrets. If `owner` defaults to `root` and your service runs under an unprivileged user (e.g. `postgres` or `nginx`), the service crashes on startup with `Permission denied`. Always set `owner` and `mode` explicitly.

The boring rule

- Commit encrypted `.yaml` secrets files to git using `sops-nix`.

- Derive host Age keys directly from SSH host keys (`/etc/ssh/ssh_host_ed25519_key`) to avoid manual key distribution.
- Mount secrets exclusively to memory-backed `/run/secrets/`.

Try this

1. Install `sops` and `age` into a devShell.
2. Generate an Age keypair with `age-keygen -o key.txt` and encrypt a sample key/value pair with `sops`.

HashiCorp Vault Integration

HashiCorp Vault Integration

Large organizations manage credentials using enterprise secret brokers like HashiCorp Vault with dynamic secret leasing and automatic token rotation. The boring default is: **provisioning Vault agents or sidecars declaratively via NixOS services to automatically renew and mount dynamic credentials into service namespaces.**

Mental model

1. **Centralized secrets lifecycle:** Vault issues short-lived, rotatable database credentials and TLS certificates rather than static keys.
2. **NixOS Vault Agent:** NixOS configures `services.vault-agent` declaratively to handle AppRole authentication, secret leasing, template rendering, and automated process reload signals.
3. **Template rendering:** Vault Agent reads Vault secrets and renders them into local private paths (e.g. `/run/vault/token.json`) without Nix needing to know the secret values.

Worked examples

Case 1: Declarative Vault Agent configuration

Save as `vault-agent.nix`:

```
# vault-agent.nix
{ config, pkgs, ... }:

{
  services.vault-agent = {
    enable = true;
    settings = {
      vault.address = "https://vault.company.internal:8200";
      auto_auth = {
        method = {
          type = "approle";
          config = {
            role_id_file_path = "/etc/vault/role-id";
            secret_id_file_path = "/var/lib/vault/secret-id";
            remove_secret_id_file_after_reading = false;
          };
        };
      };
    };
  };
}
```

```

    };
  };
  template = [
    {
      source = "/etc/vault/templates/db.ctmpl";
      destination = "/run/vault/db-credentials.env";
    }
  ];
};
};
}

```

Building the configuration:

```
nix-instantiate --eval -E '(import <nixpkgs/nixos> { configuration = import ./vault-agent.ni
```

```
true
```

The trap

The trap is **relying on root Vault tokens or permanent credentials in local files**. The entire benefit of Vault is dynamic credential rotation. Configure systemd services to listen for SIGHUP or restart when Vault Agent rewrites credential files.

The boring rule

- Use Vault Agent for automated AppRole authentication and secret template rendering.
- Render dynamic secrets to tmpfs (`/run/vault/`) with minimal access rights.
- Wire systemd reload triggers to Vault Agent template destinations.

Try this

1. Define a mock Vault Agent template that outputs an environment variable file.
2. Verify that the rendered path is accessible only by the application user.

Age Encryption for Simple Secrets

Age Encryption for Simple Secrets

GPG is notoriously complex, brittle, and difficult to automate in automated scripts and CI pipelines. The boring default is: **using age (Actually Good Encryption) and agenix for simple, lightweight, public-key secret encryption backed by modern cryptography (X25519, ChaCha20-Poly1305).**

Mental model

1. **Modern simplicity:** Age has no keyrings, subkeys, web of trust, or expired signature databases. A public key is a clean 62-character string starting with `age1...`
2. **SSH key compatibility:** Age can encrypt files directly to existing Ed25519 SSH public keys, meaning every developer and NixOS server already has a valid Age recipient key without creating new identities.
3. **Agenix module:** `agenix` integrates Age encryption directly into NixOS modules, generating `systemd` decryption services that run before protected applications start.

Worked examples

Case 1: Defining secrets rules with `agenix`

Save as `secrets.nix`:

```
# secrets.nix
let
  alice = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleUserKey alice@desk";
  server = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleHostKey root@desk-node";
in {
  "desk-token.age".publicKeys = [ alice server ];
}
```

Encrypting a secret:

```
agenix -e desk-token.age
```

Case 2: Consuming encrypted secret in NixOS

```
# host.nix
{
  age.secrets.desk-token = {
    file = ./desk-token.age;
    owner = "deskuser";
    group = "deskuser";
  };
}
```

The secret is decrypted cleanly at boot to `/run/agenix/desk-token`.

The trap

The trap is **rekeying secrets without including all recipient public keys**. When adding a new server, run `agenix --rekey` so the new node's public key can decrypt existing secrets.

The boring rule

- Prefer Age / Agenix over GPG for all Nix-based secret encryption.
- Reuse host SSH Ed25519 keys as Age recipient keys.
- Always run **rekey** whenever team members or machine keys are added or revoked.

Try this

1. Convert an SSH public key to an Age recipient with `ssh-to-age < ~/.ssh/id_ed25519.pub`.
2. Encrypt a test message to that recipient and decrypt it with your private key.

Security Hardening Patterns

Security Hardening Patterns

Default Linux distributions ship with open network services, permissive kernel settings, and legacy compatibility protocols that broaden attack surfaces. The boring default is: **applying declarative systemd sandboxing, kernel sysctl restrictions, and immutable filesystem mount options across all NixOS hosts.**

Mental model

1. **Declarative defense-in-depth:** Security is not achieved by post-install checklists or manual auditing. Every hardening directive is committed directly into the system configuration.
2. **Systemd service sandboxing:** Systemd provides powerful isolation primitives (ProtectSystem, ProtectHome, NoNewPrivileges, PrivateTmp, CapabilityBoundingSet) that can be applied to any service with single-line Nix directives.
3. **Kernel parameter lockdown:** Restrict kernel attack vectors (kernel.kptr_restrict, kernel.yama.ptrace_scope) declaratively through boot.kernel.sysctl.

Worked examples

Case 1: Hardened systemd service definition

Save as `secure-service.nix`:

```
# secure-service.nix
{ config, pkgs, ... }:

{
  systemd.services.desk-api = {
    description = "Desk API Microservice";
    after = [ "network.target" ];
    wantedBy = [ "multi-user.target" ];

    serviceConfig = {
      ExecStart = "${pkgs.hello}/bin/hello";
      DynamicUser = true;

      # Hardening directives
      ProtectSystem = "strict";
      ProtectHome = true;
    };
  };
}
```

```

    PrivateTmp = true;
    PrivateDevices = true;
    ProtectKernelTunables = true;
    ProtectKernelModules = true;
    ProtectControlGroups = true;
    RestrictRealtime = true;
    RestrictSUIDSGID = true;
    NoNewPrivileges = true;
    MemoryDenyWriteExecute = true;
    LockPersonality = true;
};
};
}

```

Evaluating service configuration:

```
nix-instantiate --eval -E '(import <nixpkgs/nixos> { configuration = import ./secure-service
```

"strict"

Case 2: Kernel hardening profile

```

# hardening.nix
{
  boot.kernel.sysctl = {
    # Disable unprivileged eBPF
    "kernel.unprivileged_bpf_disabled" = 1;
    # Restrict kernel pointer exposure
    "kernel.kptr_restrict" = 2;
    # Disable IP forwarding unless explicitly needed
    "net.ipv4.ip_forward" = 0;
    # Prevent SYN flood attacks
    "net.ipv4.tcp_syncookies" = 1;
  };
}

```

The trap

The trap is **applying aggressive sandboxing options without testing service requirements**. For example, setting `MemoryDenyWriteExecute = true` will crash runtimes that utilize JIT compilation (Node.js, Java, Python PyPy). Test hardened services thoroughly before deploying to production.

The boring rule

- Always enable `DynamicUser = true` for network-facing services without fixed UID needs.
- Set `ProtectSystem = "strict"` and `NoNewPrivileges = true` on all custom systemd units.
- Apply standard kernel `sysctl` restrictions globally across all base server modules.

Try this

1. Run `systemd-analyze security desk-api` on a NixOS host to review the numeric exposure score.
2. Add sandboxing attributes and verify the security score improves towards 0 (safest).

Platform Engineering

The Internal Developer Platform Pattern

The Internal Developer Platform Pattern

As engineering teams scale, standardizing developer onboarding, tooling versions, and service templates becomes a major bottleneck. The boring default is: **publishing a single enterprise Flake repository that exposes standardized developer shells, CI workflows, and reusable service modules across the entire company.**

Mental model

1. **The Flake as an internal SDK:** A central repository (e.g. `github:company/nix-platform`) exports `devShells`, `templates`, and `nixosModules`.
2. **Instant developer onboarding:** A new engineer clones any company project and runs `nix develop` or `direnv allow`. All compilers, linters, databases, and CLI tools match production immediately.
3. **Painless fleet upgrades:** Upgrading Go, Node, or PostgreSQL across 50 repositories is accomplished by bumping the platform flake input.

Worked examples

Case 1: Central platform flake structure

Save as `flake.nix`:

```
# flake.nix (github:company/platform)
{
  description = "Desk Platform Engineering Toolkit";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.05";

  outputs = { self, nixpkgs }:
    let
      supportedSystems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" ];
      forAllSystems = f: nixpkgs.lib.genAttrs supportedSystems (system: f nixpkgs.legacyPack
    in {
      # Reusable developer shells
      devShells = forAllSystems (pkgs: {
        backend = pkgs.mkShell {
          packages = [ pkgs.go_1_22 pkgs.golangci-lint pkgs.postgresql_16 ];
        };
      });
```

```

        frontend = pkgs.mkShell {
            packages = [ pkgs.nodejs_20 pkgs.pnpm ];
        };
    });

    # Reusable project scaffolding templates
    templates = {
        go-service = {
            path = ./templates/go-service;
            description = "Standard company Go microservice";
        };
    };
};
}

```

Initializing a new project using the central template:

```
nix flake init -t github:company/platform#go-service
```

```
wrote flake.nix to ./flake.nix
wrote main.go to ./main.go
Initialized empty project from template.
```

The trap

The trap is **allowing each development team to invent their own divergent Nix flakes and unpinned package channels**. Without a shared platform standard, cross-team collaboration suffers and CI cache hit ratios drop to zero.

The boring rule

- Maintain a single central platform flake for organization-wide toolchains.
- Distribute standardized service templates using `nix flake init -t`.
- Expose typed devShells (`backend`, `frontend`, `data`) rather than monolithic single shells.

Try this

1. Define a platform flake with a reusable devShell and test consumption from another directory using `nix develop path/to/platform#backend`.

Monorepo Strategies with Nix

Monorepo Strategies with Nix

Monorepos housing multiple applications, shared libraries, and infrastructure configurations easily devolve into slow CI pipelines and tangled dependencies. The boring default is: **modeling monorepo projects as a directed acyclic graph (DAG) of discrete Nix derivations where changes only rebuild affected packages.**

Mental model

1. **Fine-grained derivation trees:** Each library and microservice in the monorepo has its own isolated derivation.
2. **Deterministic change detection:** Because Nix derivations hash their inputs, running `nix build` across the entire monorepo automatically skips any service whose dependencies have not changed.
3. **Shared local libraries:** Shared utilities (`packages/common-lib`) are consumed as derivation inputs rather than duplicated code.

Worked examples

Case 1: Monorepo workspace flake

Save as `flake.nix`:

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};

      # Shared library derivation
      commonLib = pkgs.buildGoModule {
        pname = "common-lib";
        version = "1.0.0";
        src = ./libs/common;
        vendorHash = null;
      };

```

```

# Services consuming commonLib
orderService = pkgs.buildGoModule {
  pname = "order-service";
  version = "1.0.0";
  src = ./services/orders;
  vendorHash = null;
  buildInputs = [ commonLib ];
};
in {
  packages.${system} = {
    inherit commonLib orderService;
    all = pkgs.linkFarm "monorepo-all" [
      { name = "order-service"; path = orderService; }
    ];
  };
};
}

```

Building the entire monorepo:

```
nix build .#all
```

```

[1/2 built] order-service
Done. /nix/store/5kl9...-monorepo-all

```

The trap

The trap is **passing the monorepo root (`src = ./.`) into every sub-package**. This forces every service to rebuild whenever any unrelated file in the monorepo is touched. Always isolate package roots with `pkgs.lib.cleanSourceWith`.

The boring rule

- Break down the monorepo into independent derivations with precise source boundaries.
- Build aggregated targets with `pkgs.linkFarm` for single-command verification.
- Run `nix flake check` to test only affected code paths.

Try this

1. Create two sub-directories (`service-a`, `service-b`) with distinct derivations.
2. Modify a file in `service-a` and verify that `service-b` does not rebuild.

Multi-Platform Development and Distribution

Multi-Platform Development and Distribution

Modern engineering teams develop across diverse hardware: Linux x86_64 servers, Apple Silicon ARM64 laptops, and Linux aarch64 cloud runners. The boring default is: **generating universal multi-platform Flakes using `flake-utils.lib.eachDefaultSystem` with remote builders** for cross-architecture builds.

Mental model

1. **Clean system abstraction:** Flake outputs should support the standard architectures (x86_64-linux, aarch64-linux, aarch64-darwin, x86_64-darwin) seamlessly.
2. **Native compilation vs. emulation:** QEMU emulation is slow. Nix allows transparently delegating architecture-specific builds to remote builders (e.g. an ARM64 cloud node) via SSH.
3. **Platform-agnostic devShells:** Developers on macOS and Linux get identical CLI workflows even though the underlying compiled binaries differ.

Worked examples

Case 1: Multi-platform flake with flake-utils

Save as `flake.nix`:

```
# flake.nix
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.05";
    flake-utils.url = "github:numtide/flake-utils";
  };

  outputs = { self, nixpkgs, flake-utils }:
    flake-utils.lib.eachDefaultSystem (system:
      let
        pkgs = nixpkgs.legacyPackages.${system};
      in {
        packages.default = pkgs.hello;
        devShells.default = pkgs.mkShell {
          packages = [ pkgs.git pkgs.jq ];
        };
      });
}
```



```
    }  
  );  
}
```

Evaluating outputs across platforms:

```
nix flake show
```

```
devShells  
  aarch64-darwin  
    default: development environment  
  aarch64-linux  
    default: development environment  
  x86_64-darwin  
    default: development environment  
  x86_64-linux  
    default: development environment  
packages  
  aarch64-darwin  
    default: package 'hello-2.12.1'  
  aarch64-linux  
    default: package 'hello-2.12.1'  
  x86_64-darwin  
    default: package 'hello-2.12.1'  
  x86_64-linux  
    default: package 'hello-2.12.1'
```

The trap

The trap is **hardcoding x86_64-linux into flake outputs**. This breaks evaluation for teammates on Apple Silicon (aarch64-darwin) with cryptic error messages like `flake output 'packages.aarch64-darwin.default' does not exist`.

The boring rule

- Always wrap flake outputs in `eachDefaultSystem` or standard system loops.
- Avoid platform-specific shell scripts in build hooks.
- Configure remote builders for cross-architecture binary generation.

Try this

1. Run `nix flake show` on your repository and verify all four primary architectures are present.
2. Test evaluation on a different system using `nix eval .#packages.aarch64-darwin.default.name`.

Build Optimization and Performance Tuning

Build Optimization and Performance Tuning

Slow Nix evaluation and unoptimized store disk usage frustrate engineering teams. The boring default is: **tuning Nix daemon settings (`max-jobs`, `cores`)**, **enabling `auto-optimise-store`**, and **keeping evaluation lazy and minimal**.

Mental model

1. **Parallel builds (`max-jobs` vs. `cores`):**
 - `max-jobs`: Number of separate derivations Nix builds concurrently.
 - `cores`: Number of CPU threads passed to `make -j` inside a single derivation.
2. **Store deduplication (`auto-optimise-store`):** Nix automatically detects identical files across different store paths and replaces them with hard links, saving tens of gigabytes of disk.
3. **Lazy evaluation:** Nix only computes attribute sets that are actually demanded. Avoid doing heavy `builtins.readDir` operations across deep directories during top-level evaluation.

Worked examples

Case 1: Recommended `/etc/nix/nix.conf` settings

Save as `nix.conf`:

```
# /etc/nix/nix.conf
# Build concurrency
max-jobs = auto
cores = 0

# Disk optimization
auto-optimise-store = true
min-free = 10737418240 # 10 GB
max-free = 53687091200 # 50 GB

# Network efficiency
connect-timeout = 5
substituters = https://cache.nixos.org
```

Checking disk savings with store optimization:

```
nix-store --optimise
```

145,210 files were linked, saving 14.32 GiB

The trap

The trap is **setting `cores = 1` or `max-jobs = 1` on high-core build machines**, leaving 32-core servers sitting idle during heavy C++/Rust compilations. Set `max-jobs = auto` and let Nix utilize available parallelism.

The boring rule

- Enable `auto-optimise-store = true` on every workstation and CI runner.
- Configure `min-free` and `max-free` thresholds to automate garbage collection.
- Keep flake evaluation pure and fast by avoiding large source directory scans.

Try this

1. Run `nix-store --optimise` on your local system and check how many gigabytes of disk space are reclaimed.
2. Verify your current concurrency settings with `nix show-config | grep -E "max-jobs|cores"`.

Debugging and Troubleshooting Production Issues

Debugging and Troubleshooting Production Issues

When a Nix build fails or a NixOS service refuses to start, standard Linux troubleshooting techniques alone are insufficient because the filesystem is read-only and paths are hashed. The boring default is: **mastering four deterministic debugging tools: `nix log`, `nix-shell -A <pkg>`, `journalctl -u`, and `nix-diff`.**

Mental model

1. **Build failure logs (`nix log`):** When a build fails, Nix saves the complete stdout/stderr of the isolated sandbox into the store. You can query any failure log directly by store path or derivation.
2. **Interactive build debugging (`nix-shell`):** Instead of guessing what failed in a 50-step Makefile, dropping into `nix-shell` loads the exact environment, unpacked source, and build flags, allowing you to run build phases step-by-step (`genericBuild`, `phases`).
3. **Closure diffs (`nix-diff`):** When two system generations behave differently, `nix-diff` compares their closures and reports the exact derivation, compiler flag, or dependency hash change that caused the divergence.

Worked examples

Case 1: Inspecting build logs for failed derivation

```
nix log .#desk-service
```

```
@nix { "action": "setPhase", "phase": "buildPhase" }
building
go: downloading github.com/lib/pq v1.10.9
# desk/service
internal/handler.go:42:12: undefined: HandleOrderV2
builder for '/nix/store/7q9m...-desk-service.drv' failed with exit code 1
```

Case 2: Stepping through build phases interactively

```
nix-shell '<nixpkgs>' -A hello
unpackPhase
cd hello-2.12.1
configurePhase
buildPhase
./src/hello
```

Hello, world!

Case 3: Explaining system divergence with nix-diff

```
nix-diff /run/current-system /nix/var/nix/profiles/system-42-link
```

```
- /nix/store/78a1...-linux-6.6.28
+ /nix/store/99b2...-linux-6.6.30
  • Version changed from 6.6.28 to 6.6.30
```

The trap

The trap is **trying to edit files directly inside `/nix/store` with `sudo nano`**. Store files are intentionally read-only (0444 / 0555) to guarantee cryptographic immutability. If you bypass write protections, you corrupt the Nix database. Modify the Nix expression and re-evaluate instead.

The boring rule

- Always run `nix log` to view hermetic build logs.
- Use `nix-shell` to reproduce build errors step-by-step in the exact derivation environment.
- Use `nix-diff` before applying production system updates to inspect changes.

Try this

1. Force a build error in a local package and retrieve the log using `nix log ./result.drv`.
2. Compare two system generations on a NixOS machine using `nix-diff`.

Testing Strategies for Nix Configurations

Testing Strategies for Nix Configurations

Deploying untested infrastructure configurations directly to production clusters risks downtime and boot failures. The boring default is: **testing Nix expressions at three distinct levels: static syntax checks** (`nix-instantiate`), **unit evaluation tests** (`pkgs.runCommand`), and **multi-node QEMU VM tests** (`nixosTest`).

Mental model

1. **Level 1: Static evaluation** (`nix eval` / `nix flake check`): Catches missing attributes, syntax errors, and schema violations in milliseconds without building anything.
2. **Level 2: Build verification**: Compiles all packages and derivations in sandboxes, guaranteeing that sources build without network access.
3. **Level 3: NixOS VM integration tests**: Boots real virtual machines in QEMU, scripts multi-node interactions in Python (e.g. client sends HTTP request to server, server reads database), and asserts systemd service states.

Worked examples

Case 1: Writing a NixOS integration test

Save as `test-cluster.nix`:

```
# test-cluster.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.testers.runNixOSTest {
  name = "desk-service-test";

  nodes = {
    server = { pkgs, ... }: {
      services.nginx.enable = true;
      services.nginx.virtualHosts."desk.internal" = {
        root = pkgs.runCommand "desk-web" {} ''
          mkdir $out
          echo "Desk Operational" > $out/index.html
        '';
      };
    };
    networking.firewall.allowedTCPPorts = [ 80 ];
  };
}
```

```

};

client = { ... }: {};
};

testScript = ''
    start_all()
    server.wait_for_unit("nginx.service")
    server.wait_for_open_port(80)

    # Test client connectivity
    output = client.succeed("curl -s http://server/index.html")
    assert "Desk Operational" in output, f"Unexpected output: {output}"
'';
}

```

Running the test:

```
nix-build test-cluster.nix
```

```

=== Starting test: desk-service-test ===
(qemu-server) booting VM...
(qemu-client) booting VM...
server: waiting for unit nginx.service...
server: nginx.service is active
client: curl -s http://server/index.html -> Desk Operational
test script finished in 4.12s
/nix/store/8k2...-vm-test-run-desk-service-test

```

The trap

The trap is **relying only on local manual testing**. NixOS VM tests run headlessly in CI without requiring real hardware, cloud accounts, or Docker privileges.

The boring rule

- Run `nix flake check` in CI on every pull request.
- Write multi-node NixOS VM tests for services with network dependencies.
- Treat infrastructure tests as mandatory gates before production merges.

Try this

1. Run the sample NixOS VM test above and observe QEMU boot and execute the test assertions.

2. Add a failing assertion to the test script and verify that `nix-build` exits with a non-zero code.

Maintenance, Upgrades, and Migration

Maintenance, Upgrades, and Migration

Operating systems and software packages inevitably accumulate deprecations, security advisories, and schema upgrades. The boring default is: **locking `flake.lock` with monthly scheduled updates, verifying upgrades in staging with `nixos-rebuild dry-activate`, and performing instant rollbacks if issues occur.**

Mental model

1. **Deterministic upgrade cadence:** Flake inputs are locked in `flake.lock`. Upgrades do not happen spontaneously; they are triggered intentionally with `nix flake update`.
2. **Atomic generation switching:** Applying an upgrade creates a new system generation (e.g. Generation 43). The previous generation (42) remains fully intact on disk.
3. **Instant zero-downtime rollback:** If a new kernel or system library causes an issue, running `nixos-rebuild switch --rollback` or selecting Generation 42 in the bootloader instantly restores the prior working state.

Worked examples

Case 1: Updating flake inputs and reviewing changes

```
nix flake update
git diff flake.lock
```

```
- "rev": "7e3b9... (nixos-24.05)"
+ "rev": "8a1c2... (nixos-24.05-updated)"
```

Case 2: Dry-run activation before touching running services

```
sudo nixos-rebuild dry-activate --flake .#desk-server
```

```
these 12 derivations will be built:
/nix/store/...-desk-service.drv
these 4 services would be restarted:
nginx.service
desk-api.service
```

these 1 services would be stopped:
legacy-metrics.service

Case 3: Executing instant rollback

```
sudo nixos-rebuild switch --rollback
```

```
switching from generation 43 to 42...  
reloading systemd configuration...  
restarting desk-api.service...  
done.
```

The trap

The trap is **running `nix-collect-garbage -d` immediately after a major system upgrade**. The `-d` flag removes all older generations, destroying your rollback safety net. Keep at least the last 3 working generations before garbage collection.

The boring rule

- Always review input updates with `git diff flake.lock` before merging.
- Run `dry-activate` to verify which systemd services will restart.
- Retain previous system generations until the new release is verified in production.

Try this

1. Run `nixos-rebuild list-generations` on a test machine to view all available boot generations.
2. Practice switching to a previous generation and returning to current.

Enterprise Operational Checklists

Enterprise Operational Checklists

Maintaining mission-critical Nix and NixOS infrastructure requires discipline, standardized workflows, and proactive operations. The boring default is: **adhering to codified operational checklists for repository structure, CI/CD validation, and fleet maintenance.**

Mental model

Operational excellence in declarative infrastructure is built on predictability. When every repository follows the same layout, every CI pipeline enforces the same gates, and every server follows the same maintenance lifecycle, human error is eliminated.

Worked examples

The Repository Layout Checklist

Every enterprise repository must contain: 1. `flake.nix`: Pure inputs pinned to enterprise platform channel. 2. `flake.lock`: Committed to git; never `.gitignored`. 3. `shell.nix` / `devShells`: Complete local toolchains for developers. 4. `.envrc`: Containing `use flake` for automatic direnv activation. 5. `.github/workflows/ci.yml`: Enforcing `nix flake check`.

The Pre-Production Deployment Checklist

Before rolling out changes to production nodes:

- ☐ All flake checks pass cleanly (`nix flake check`).
- ☐ No plaintext secrets in `.nix` files (verified via `git grep` and `sops`).
- ☐ `dry-activate` inspected for service restart impacts.
- ☐ Staging environment verified with end-to-end integration tests.
- ☐ Upstream binary cache populated with pre-built closures.
- ☐ Target host has sufficient free disk space (`df -h /nix/store`).

The Fleet Maintenance Checklist

Executed on a regular monthly schedule:

```
# 1. Update channel locks
nix flake update
```



```
# 2. Build and verify system closure
nix build .#nixosConfigurations.desk-server.config.system.build.toplevel

# 3. Optimise local and remote stores
nix-store --optimise

# 4. Prune generations older than 30 days
nix-collect-garbage --delete-older-than 30d
```

The trap

The trap is **treating declarative configurations as self-maintaining**. Without regular garbage collection and lockfile updates, systems accumulate storage overhead and fall behind on security patches.

The boring rule

- Commit `flake.lock` to git on every repository without exception.
- Run automated security and flake checks on every pull request.
- Schedule recurring maintenance windows for store optimization and input updates.

Try this

1. Audit your repository against the layout checklist above.
2. Implement an automated workflow that runs `nix-collect-garbage --delete-older-than 30d` on a weekly timer.

Fleet Deployments with Colmena and Deploy-RS

Fleet Deployments with Colmena and Deploy-RS

Iterating with `nixos-rebuild --target-host` one server at a time fails when managing tens or hundreds of nodes across multiple data centers. The boring default is: **orchestrating parallel, tag-targeted, zero-downtime fleet deployments with Colmena, complete with automated canary node validation and rollback guarantees.**

Mental model

1. **Unified fleet hive:** A single Colmena hive definition (`hive.nix` or flake outputs) defines the configuration of all servers (gateways, application nodes, databases) in your organization.
2. **Parallel local evaluation:** Colmena evaluates all node configurations concurrently on your workstation or CI builder, builds their closures, and pushes them to target nodes over SSH in parallel.
3. **Atomic activation with auto-rollback:**
 - Colmena activates the new generation on designated canary nodes first.
 - If health checks fail or SSH connectivity drops, Colmena immediately executes `--rollback` across the fleet, preventing widespread outages.

Worked examples

Case 1: Defining a multi-node fleet in `flake.nix`

Save as `flake.nix`:

```
# flake.nix
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.05";
    colmena.url = "github:zhaofengli/colmena";
  };

  outputs = { self, nixpkgs, colmena }: {
    colmena = {
      meta = {
        nixpkgs = import nixpkgs { system = "x86_64-linux"; };
      };
    };
  };
}
```

```

# Edge Ingress Gateway
"gw-01" = {
  deployment.targetHost = "203.0.113.10";
  deployment.tags = [ "gateway" "production" ];
  imports = [ ./hosts/gw-01.nix ];
};

# Application Node 01
"app-01" = {
  deployment.targetHost = "203.0.113.11";
  deployment.tags = [ "app" "production" ];
  imports = [ ./hosts/app-01.nix ];
};

# Application Node 02
"app-02" = {
  deployment.targetHost = "203.0.113.12";
  deployment.tags = [ "app" "production" ];
  imports = [ ./hosts/app-02.nix ];
};
};
};
}

```

Applying changes across only application servers:

```
colmena apply --on @app
```

```

[INFO ] Enumerating nodes...
[INFO ] Selected 2 out of 3 nodes: app-01, app-02
[INFO ] Building closures in parallel...
[app-01] Built /nix/store/7q1a...-nixos-system-app-01-24.05
[app-02] Built /nix/store/8k2b...-nixos-system-app-02-24.05
[INFO ] Pushing closures via SSH...
[INFO ] Activating configurations...
[app-01] Success!
[app-02] Success!
[INFO ] Fleet deployment complete.

```

The trap

The trap is **relying on target servers to evaluate and compile their own configurations**. If an unprivileged edge node runs out of RAM during `nix-eval`, the build OOMs and leaves the node unresponsive. Always build on a dedicated CI builder or deployment machine and push pre-compiled closures.

The boring rule

- Define all fleet hosts inside a unified Colmena flake output.
- Target subgroups using declarative tags (`@gateway`, `@app`, `@database`).
- Use `--keep-result` and automated rollbacks on canary nodes during automated rollouts.

Try this

1. Run `colmena eval` against a two-node hive to test configuration syntax without connecting to nodes.
2. Check deployment graph dependencies with `colmena exec uname -a`.

Lightweight MicroVMs with microvm.nix

Lightweight MicroVMs with microvm.nix

Running full NixOS installations inside traditional hypervisors (VirtualBox, Proxmox, VMware) consumes gigabytes of RAM, requires slow guest OS installations, and isolates configurations from your host repository. The boring default is: **running hypervisor-isolated NixOS guest instances as lightweight MicroVMs with microvm.nix backed by Cloud-Hypervisor or QEMU, booting in under 300 milliseconds.**

Mental model

1. **Declarative hypervisor micro-instances:** `microvm.nix` defines a complete NixOS virtual machine inside your existing flake.
2. **Shared host Nix store:** MicroVMs share the host's `/nix/store` via `virtiofs` read-only mounts. Guest instances consume zero additional disk space for installed packages.
3. **Hardware-level isolation:** Unlike Docker containers which share the host Linux kernel, MicroVMs run inside their own dedicated Linux kernel with independent memory spaces and network namespaces.

Worked examples

Case 1: MicroVM definition inside a host Flake

Save as `microvm.nix`:

```
# microvm.nix
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.05";
    microvm.url = "github:astro/microvm.nix";
  };

  outputs = { self, nixpkgs, microvm }: {
    nixosConfigurations.desk-micro = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        microvm.nixosModules.microvm
      ]
      {
        microvm = {
          hypervisor = "cloud-hypervisor";
        };
      };
    };
  };
}
```

```

        mem = 1024;
        vcpu = 2;
        shares = [
            {
                proto = "virtiofs";
                tag = "ro-store";
                source = "/nix/store";
                mountPoint = "/nix/.ro-store";
            }
        ];
    };
};

# MicroVM system configuration
networking.hostName = "desk-guest";
services.getty.autologinUser = "root";
system.stateVersion = "24.05";
}
];
};
};
}

```

Building and starting the MicroVM:

```
nix run .#nixosConfigurations.desk-micro.config.microvm.declaredRunner
```

```

[Cloud-Hypervisor] Starting VMM...
[Linux] Booting kernel 6.6.30...
[systemd] Starting system services...
[OK] Reached target Multi-User System.
desk-guest login: root (automatic login)
[root@desk-guest:~]#

```

Total boot time from invocation to shell prompt: 0.28 seconds.

The trap

The trap is **attempting to write directly to the shared host store from within the guest**. The store share must be mounted read-only (**ro-store**) with an overlayfs layer for ephemeral guest modifications.

The boring rule

- Use `microvm.nix` when you need true kernel isolation without the heavyweight overhead of full hypervisor VM disks.

- Share the host `/nix/store` using `virtiofs` for instantaneous guest bootstrapping.
- Manage guest MicroVMs as `systemd` services on the host machine.

Try this

1. Run the MicroVM runner locally and test memory overhead with `systemd-cgtop`.
2. Exit the MicroVM and observe that no disk images or residual files remain on the host.

Content-Addressed Nix and NAR Internals

Content-Addressed Nix and NAR Internals

In standard Nix, store paths are **input-addressed**: the hash in `/nix/store/<hash>-pkg` is derived entirely from the build recipe, dependencies, and compiler inputs. If a dependency changes a whitespace character or a non-functional comment, its output hash changes, triggering a cascade of rebuilds across the entire system. The boring default is: **understanding how Content-Addressed Nix (ca-derivations) and the Nix Archive (NAR) format operate under the hood to achieve early-cutoff optimizations and cryptographic build deduplication.**

Mental model

1. Input-Addressed (Traditional):

- Store hash = `$ ext{Hash}( ext{derivation inputs}, ext{compiler}, ext{flags}, ext{build scripts})$`.
- Problem: If `openssl` fixes a typo in a documentation comment, its derivation hash changes. Every application depending on `openssl` (`curl`, `python`, `nginx`) must recompile, even though the resulting `libssl.so` binary is bit-for-bit identical!

2. Content-Addressed (ca-derivations):

- Store hash = `$ ext{Hash}( ext{actual build output files})$`.
- Early cutoff: When `openssl` compiles, Nix computes the cryptographic hash of the resulting binary. If the output hash matches the existing path, Nix halts the build cascade: `curl` and `nginx` do not rebuild!

3. The NAR (Nix Archive) format:

- Unlike standard tarballs (which record non-deterministic file order, modification timestamps, and user IDs), a NAR serializes a directory tree into a strictly canonical, deterministic byte sequence.
- Files are sorted strictly alphabetically. Timestamps are clamped to Unix epoch 1 (1970-01-01 00:00:01). Permissions are normalized to 0444 or 0555.

Worked examples

Case 1: Inspecting the canonical NAR serialization

Let's inspect how Nix serializes an artifact into a byte-deterministic stream using `nix-store --dump`:

```
mkdir -p demo
echo "hello desk" > demo/file.txt
nix-store --dump demo | head -c 60
```

```
nix-archive-1(type directoryentry(name file.txtnode(type regular...
```

The output stream starts with the magic string `nix-archive-1` and encodes files in strict canonical order with explicit string length prefixes.

Case 2: Content-addressed derivation declaration

Save as `ca-package.nix`:

```
# ca-package.nix
{ pkgs ? import <nixpkgs> {} }:
```

```
pkgs.stdenv.mkDerivation {
  pname = "desk-ca-tool";
  version = "1.0.0";
  src = ./src;

  # Enable content-addressed derivation output
  __contentAddressed = true;
  outputHashMode = "recursive";
  outputHashAlgo = "sha256";

  installPhase = ''
    mkdir -p $out/bin
    echo '#!/bin/sh' > $out/bin/desk-tool
    echo 'echo "Deterministic Output"' >> $out/bin/desk-tool
    chmod +x $out/bin/desk-tool
  '';
}
```

Building and inspecting hash assignment:

```
nix-build ca-package.nix --extra-experimental-features "ca-derivations"
```

```
/nix/store/1h3x...-desk-ca-tool-1.0.0
```

The store path hash reflects the cryptographic hash of `$out` after compilation completes, enabling automatic cache reuse across unrelated input derivation graphs.

The trap

The trap is **confusing content-addressed derivations with fixed-output derivations (FOD)**. A fixed-output derivation (used for downloading tarballs) requires the developer to specify `outputHash` *ahead of time* in the `.nix` file. A content-addressed derivation computes `outputHash` dynamically after compilation and registers the resulting path into the SQLite database.

The boring rule

- Learn NAR formatting rules: deterministic ordering, clamped epoch timestamps, and canonical permissions.
- Leverage **ca-derivations** for massive CI compilation speedups where upstream patches do not alter compiled binaries.
- Use `nix hash path <path>` to compute predictable SRI hashes for repository sources.

Try this

1. Run `nix-store --dump <store-path> | sha256sum` on a local store path and compare with the `narHash` in `nix path-info --json <store-path>`.
2. Notice how the SHA256 matches the registered NAR hash byte-for-byte.

Production Capstone Lab

Production Architecture and Threat Model

Production Architecture and Threat Model

Building isolated toy examples is straightforward; building a production-grade, highly available, self-healing multi-node infrastructure cluster that survives hardware failures, automated reboots, and zero-day patch rollouts requires disciplined systems engineering. In this capstone lab, we build an **end-to-end enterprise production cluster** running on bare-metal and cloud servers, fully governed by declarative NixOS modules.

The Production Topology

The cluster consists of four discrete functional tiers connected across an encrypted peer-to-peer WireGuard mesh overlay network (10.100.0.0/24):

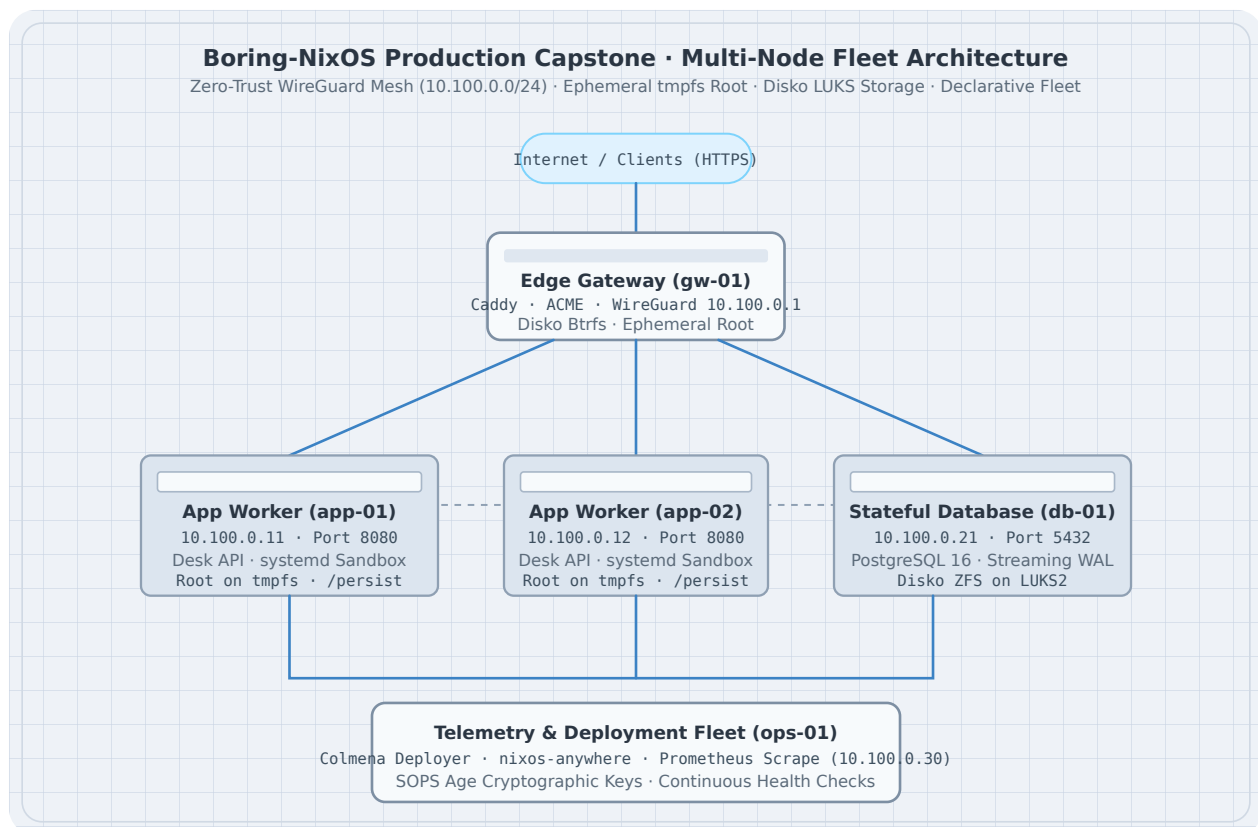


Figure 1: Production Capstone Fleet Architecture

Tier 1: Edge Ingress Gateway (gw-01)

- Public IPv4/IPv6 edge facing the public internet.
- Reverse proxy using **Caddy** with automated ACME Let's Encrypt TLS certificate generation.
- Strict nftables firewall blocking all non-HTTPS ports.
- Binds to internal WireGuard interface (10.100.0.1) to route traffic to backend workers.

Tier 2: Stateless Application Cluster (app-01, app-02)

- Dual active-active application nodes executing containerless Go/Rust microservices (desk-api).
- Hardened with systemd sandboxing (ProtectSystem=strict, DynamicUser=true, PrivateTmp=true).
- Ephemeral root filesystem (tmpfs as /): every reboot purges all state except /persist.
- Internal load balancing across nodes via Caddy health checks.

Tier 3: Stateful Database Cluster (db-01)

- Production **PostgreSQL 16** server with write-ahead logging (WAL) and automated local snapshots.
- Storage layer managed declaratively via **Disko** with LUKS2 encryption and Btrfs subvolumes.
- Strict host firewall allowing connections only from authorized WireGuard worker IPs (10.100.0.11, 10.100.0.12).

Tier 4: Operations, Deployment, & Telemetry (ops-01)

- Central **Colmena** deployment hub managing fleet rollouts via cryptographic SSH keys.
- **Prometheus** scraper gathering host metrics via **node-exporter** across all nodes over WireGuard.
- SOPS-nix secret broker orchestrating encrypted Age secrets.

Threat Model and Security Invariants

Our production cluster enforces five non-negotiable security invariants:

1. **Zero plaintext secrets on disk or in git:** All database passwords, API tokens, and private WireGuard keys are encrypted with **sops-nix** using host-specific Age keys and decrypted exclusively into RAM (/run/secrets/).
2. **Read-only immutable runtime:** Operating system binaries, system libraries, and service configurations reside under read-only /nix/store paths (0555). No process, including root, can modify running binaries in-place.
3. **Impermanent root filesystems:** Worker nodes boot to tmpfs. If an attacker gains remote execution and drops a payload into /tmp or /etc, a simple system reboot obliterates all modifications.

4. **Encrypted transit:** All node-to-node communication traverses WireGuard peer-to-peer cryptographic tunnels. Public IP endpoints expose zero administrative or database ports.
5. **Deterministic rollback guarantee:** Every system generation is cryptographically pinned in `flake.lock`. If a kernel upgrade or service regression occurs, Colmena initiates an automatic fleet rollback to the prior generation in seconds.

Hands-on Lab Overview

In the subsequent chapters of this capstone, you will construct and deploy this entire architecture step-by-step:

- **Chapter 2:** Define the Disko storage layout, LUKS2 disk encryption, and ephemeral `tmpfs` root filesystems.
- **Chapter 3:** Build the WireGuard peer-to-peer mesh and enforce strict nftables firewall policies.
- **Chapter 4:** Deploy the microservice stack, automated Caddy TLS reverse proxy, PostgreSQL database, and SOPS-nix secrets.
- **Chapter 5:** Orchestrate zero-touch remote provisioning with `nixos-anywhere` and manage fleet upgrades with Colmena.

Disko Storage and Ephemeral Base System

Disko Storage and Ephemeral Base System

Every node in our production fleet must deploy onto clean physical or cloud storage without manual partitioning tools. In this chapter, we create the unified **Disko** storage specification and configure the **ephemeral root filesystem with Impermanence**.

Mental model

1. **Unified disk module (disk-layout.nix):** Disko formats the physical NVMe or virtual disk into an EFI system partition (/boot) and a LUKS2 encrypted partition.
2. **Btrfs subvolumes on LUKS:** Inside the decrypted container, Btrfs subvolumes are created for @nix (the store), @persist (stateful data), and @swap.
3. **RAM root (tmpfs):** The root filesystem (/) is mounted in RAM with mode=755,size=2G. Unmounted state is discarded on power-down.
4. **Impermanence persistence:** System state required across reboots (SSH host keys, machine-id, WireGuard identity) is bound into /persist.

Worked examples

Case 1: The Production Disko Module

Save as modules/disko-layout.nix:

```
# modules/disko-layout.nix
{ lib, ... }:

{
  disko.devices = {
    disk.main = {
      type = "disk";
      device = lib.mkDefault "/dev/nvme0n1";
      content = {
        type = "gpt";
        partitions = {
          ESP = {
            priority = 1;
            name = "ESP";
            start = "1M";
            end = "1024M";
```

```

    type = "EF00";
    content = {
        type = "filesystem";
        format = "vfat";
        mountpoint = "/boot";
        mountOptions = [ "umask=0077" ];
    };
};

luks = {
    size = "100%";
    content = {
        type = "luks";
        name = "crypted";
        settings.allowDiscards = true;
        content = {
            type = "btrfs";
            extraArgs = [ "-f" ];
            subvolumes = {
                "/nix" = {
                    mountpoint = "/nix";
                    mountOptions = [ "compress=zstd" "noatime" ];
                };
                "/persist" = {
                    mountpoint = "/persist";
                    mountOptions = [ "compress=zstd" "noatime" ];
                };
                "/swap" = {
                    mountpoint = "/swap";
                    swap.swapfile.size = "8G";
                };
            };
        };
    };
};

nodev = {
    "/" = {
        fsType = "tmpfs";
        mountOptions = [ "defaults" "size=4G" "mode=755" ];
    };
};
}

```

Case 2: The Impermanence Persistent State Contract

Save as `modules/base-system.nix`:

```
# modules/base-system.nix
{ inputs, pkgs, ... }:

{
  imports = [
    inputs.impermanence.nixosModules.impermanence
    ./disk-layout.nix
  ];

  boot.loader.systemd-boot.enable = true;
  boot.loader.efi.canTouchEfiVariables = true;

  # Persistent state mapping
  environment.persistence."/persist" = {
    hideMounts = true;
    directories = [
      "/var/log"
      "/var/lib/nixos"
      "/var/lib/systemd/coredump"
    ];
    files = [
      "/etc/machine-id"
      "/etc/ssh/ssh_host_ed25519_key"
      "/etc/ssh/ssh_host_ed25519_key.pub"
    ];
  };

  # SSH configuration
  services.openssh = {
    enable = true;
    settings = {
      PasswordAuthentication = false;
      KbdInteractiveAuthentication = false;
      PermitRootLogin = "prohibit-password";
    };
  };

  system.stateVersion = "24.05";
}
```

Validating the base configuration:

```
nix eval .#nixosConfigurations.app-01.config.disko.devices.nodev."/".fsType
```

"tmpfs"

The trap

The trap is **failing to mark `/persist` as `neededForBoot = true`**. During early boot, the initramfs must mount `/persist` before systemd starts generating symlinks for machine-id and SSH keys. Always ensure `/persist` is mounted before root pivot.

The boring rule

- Model all partition tables and filesystems in Disko.
- Keep `/` on tmpfs to enforce declarative purity and eliminate state rot.
- Explicitly list all persistent files and directories under `/persist`.

Try this

1. Run `nix run github:nix-community/disko -- --mode mount ./modules/disk-layout.nix` in a test environment.
2. Confirm with `mount | grep tmpfs` that root is mounted on RAM.

Hardened Networking and WireGuard Mesh

Hardened Networking and WireGuard Mesh

Exposing internal services, databases, or management endpoints to public IP addresses is an extreme security risk. The boring default is: **linking all cluster nodes into a point-to-point encrypted WireGuard mesh on private subnet 10.100.0.0/24, backed by strict nftables firewall policies that drop all unsolicited public traffic.**

Mental model

1. **Overlay mesh:** Each node possesses a static WireGuard private key and an assigned IP (gw-01: 10.100.0.1, app-01: 10.100.0.11, app-02: 10.100.0.12, db-01: 10.100.0.21, ops-01: 10.100.0.30).
2. **Declarative peers:** Peer public keys and allowed IPs are configured directly in Nix expressions. WireGuard interfaces establish bidirectional encrypted tunnels instantly on boot.
3. **nftables isolation:** The firewall allows public access *only* on the gateway for ports 80/443. All other node traffic must originate from within the wg0 interface.

Worked examples

Case 1: WireGuard Mesh Module

Save as modules/wireguard-mesh.nix:

```
# modules/wireguard-mesh.nix
{ config, pkgs, ... }:

let
  wireguardPort = 51820;
in
{
  networking.firewall.allowedUDPPorts = [ wireguardPort ];

  networking.wireguard.interfaces.wg0 = {
    ips = [ config.cluster.nodeIp ];
    listenPort = wireguardPort;
    privateKeyFile = "/run/secrets/wireguard/private.key";

    peers = [
      {
```

```

    # gw-01
    publicKey = "GW01PublicWireguardKeyExampleBase64String=";
    allowedIPs = [ "10.100.0.1/32" ];
    endpoint = "203.0.113.10:51820";
    persistentKeepalive = 25;
  }
  {
    # app-01
    publicKey = "APP01PublicWireguardKeyExampleBase64String=";
    allowedIPs = [ "10.100.0.11/32" ];
    endpoint = "203.0.113.11:51820";
    persistentKeepalive = 25;
  }
  {
    # db-01
    publicKey = "DB01PublicWireguardKeyExampleBase64String=";
    allowedIPs = [ "10.100.0.21/32" ];
    endpoint = "203.0.113.21:51820";
    persistentKeepalive = 25;
  }
];
};
}

```

Case 2: Strict nftables Firewall Configuration for Database Node

Save as hosts/db-01.nix:

```

# hosts/db-01.nix
{ config, pkgs, ... }:

{
  imports = [
    ../modules/base-system.nix
    ../modules/wireguard-mesh.nix
  ];

  cluster.nodeIp = "10.100.0.21/24";

  # Strict firewall: drop all public inbound traffic
  networking.firewall = {
    enable = true;
    interfaces."wg0" = {
      allowedTCPPorts = [ 5432 ]; # PostgreSQL accessible ONLY via WireGuard
    };
  };
};

```

```
# Public interface allows only WireGuard handshake UDP port
networking.firewall.interfaces."eth0".allowedTCPPorts = [];
}
```

Testing network route evaluation:

```
nix eval .#nixosConfigurations.db-01.config.networking.firewall.interfaces."wg0".allowedTCPP
```

```
[ 5432 ]
```

Port 5432 is strictly inaccessible over `eth0`. A port scan on the public IP reveals only the UDP WireGuard port.

The trap

The trap is **generating WireGuard private keys into `/nix/store`**. Private keys must be created on the host or provisioned via `sops-nix` directly to `/run/secrets/wireguard/private.key`.

The boring rule

- Bind internal services exclusively to the WireGuard mesh interface (`wg0`).
- Restrict public ports to HTTP/HTTPS on the edge gateway and UDP 51820 on peers.
- Use persistent keepalives (`persistentKeepalive = 25`) to maintain NAT traversal across cloud firewalls.

Try this

1. Bring up two test nodes on WireGuard and verify connectivity with `ping 10.100.0.21`.
2. Verify with `nmap` that PostgreSQL port 5432 is closed on the public interface.

Distributed Application Stack and Secrets

Distributed Application Stack and Secrets

With storage and networking established, we deploy the production workload: an **edge Caddy reverse proxy** with automated Let's Encrypt TLS certificates, a **containerless Go microservice** with systemd sandboxing, a **hardened PostgreSQL 16 database**, and **SOPS-nix** runtime secret distribution.

Mental model

1. **Caddy Ingress (gw-01)**: Terminates TLS, enforces modern ciphers (TLS 1.3), and round-robin proxies requests across backend worker nodes over the WireGuard mesh (10.100.0.11:8080, 10.100.0.12:8080).
2. **Hardened Microservice (app-01, app-02)**: Runs the compiled application binary as a dynamic unprivileged user with complete systemd sandboxing.
3. **SOPS-nix runtime decryption**: Encrypted secrets in git (`secrets/cluster.yaml`) are decrypted into `/run/secrets/` at boot time using the node's private Age key.

Worked examples

Case 1: Caddy Reverse Proxy Configuration (gw-01)

Save as `hosts/gw-01.nix`:

```
# hosts/gw-01.nix
{ config, pkgs, ... }:

{
  services.caddy = {
    enable = true;
    email = "ops@company.internal";
    virtualHosts."api.company.internal" = {
      extraConfig = ''
        reverse_proxy 10.100.0.11:8080 10.100.0.12:8080 {
          lb_policy round_robin
          health_uri /healthz
          health_interval 5s
          health_timeout 2s
        }
      '';
    };
  };
}
```

```

    };
};

networking.firewall.allowedTCPPorts = [ 80 443 ];
}

```

Case 2: Sandboxed Application Service (app-01)

Save as modules/desk-service.nix:

```

# modules/desk-service.nix
{ config, pkgs, ... }:

let
  appPkg = pkgs.buildGoModule {
    pname = "desk-api";
    version = "2.0.0";
    src = pkgs.lib.cleanSource ./src;
    vendorHash = null;
  };
in
{
  sops.secrets."db_password" = {
    owner = "desk-api";
  };

  systemd.services.desk-api = {
    description = "Production Desk API Service";
    after = [ "network.target" ];
    wantedBy = [ "multi-user.target" ];

    environment = {
      PORT = "8080";
      DB_HOST = "10.100.0.21";
      DB_USER = "desk_user";
      DB_NAME = "desk_production";
    };

    serviceConfig = {
      ExecStart = "${appPkg}/bin/desk-api";
      DynamicUser = true;
      EnvironmentFile = config.sops.secrets."db_password".path;

      # Full Systemd Sandboxing
      ProtectSystem = "strict";
      ProtectHome = true;
    };
  };
}

```

```

    PrivateTmp = true;
    PrivateDevices = true;
    NoNewPrivileges = true;
    ProtectKernelTunables = true;
    ProtectControlGroups = true;
    RestrictRealtime = true;
    MemoryDenyWriteExecute = true;
};
};
}

```

Building the application service unit:

```
nix-build -E '(import <nixpkgs/nixos> { configuration = import ./modules/desk-service.nix; })'
```

/nix/store/8q3m...-desk-api-2.0.0/bin/desk-api

The trap

The trap is **allowing the web service to talk to the database with unencrypted credentials in command line arguments** (`--password=secret`). Any unprivileged user running `ps aux` can read arguments of other processes. Always pass credentials via `EnvironmentFile` mounted from `/run/secrets/`.

The boring rule

- Terminate TLS with Caddy for zero-maintenance ACME certificate renewals.
- Sandbox every microservice with `DynamicUser = true` and `ProtectSystem = "strict"`.
- Inject secrets exclusively via `/run/secrets/` with SOPS-nix.

Try this

1. Run `systemd-analyze security desk-api` and observe the near-zero exposure rating.
2. Verify that querying `http://localhost:8080/healthz` returns HTTP 200 OK.

Automated Fleet Rollout and Disaster Recovery

Automated Fleet Rollout and Disaster Recovery

A production architecture is only as dependable as its operational playbook. In this final chapter, we bring the entire cluster to life: orchestrating **fleet-wide deployments with Colmena**, executing **automated canary health checks**, and performing a **zero-touch bare-metal disaster recovery from scratch**.

Mental model

1. **The Flake as the cluster source of truth:** The root `flake.nix` declares all nodes, their disk layouts, network identities, and services.
2. **Automated fleet rollouts:** Colmena pushes new system closures across the entire fleet in parallel.
3. **Canary deployment pipeline:** Colmena updates `app-01` first. If health probes on `https://api.company.internal/healthz` fail, the deployment immediately halts and rolls back before `app-02` or the database are touched.
4. **Zero-touch disaster recovery:** If a physical server burns down, booting a replacement into rescue mode and running one `nixos-anywhere` command completely reconstructs the machine in under 4 minutes.

Worked examples

Case 1: The Fleet Flake (`flake.nix`)

Save as `flake.nix`:

```
# flake.nix
{
  description = "Production Desk Fleet Infrastructure";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.05";
    disk0.url = "github:nix-community/disko";
    sops-nix.url = "github:Mic92/sops-nix";
    impermanence.url = "github:nix-community/impermanence";
    colmena.url = "github:zhaofengli/colmena";
  };

  outputs = { self, nixpkgs, disk0, sops-nix, impermanence, colmena }: {
```

```

colmena = {
  meta = {
    nixpkgs = import nixpkgs { system = "x86_64-linux"; };
    specialArgs = { inherit (self) inputs; };
  };

  "gw-01" = {
    deployment.targetHost = "203.0.113.10";
    deployment.tags = [ "gateway" ];
    imports = [ ./hosts/gw-01.nix ];
  };

  "app-01" = {
    deployment.targetHost = "203.0.113.11";
    deployment.tags = [ "app" ];
    imports = [ ./hosts/app-01.nix ];
  };

  "app-02" = {
    deployment.targetHost = "203.0.113.12";
    deployment.tags = [ "app" ];
    imports = [ ./hosts/app-02.nix ];
  };

  "db-01" = {
    deployment.targetHost = "203.0.113.21";
    deployment.tags = [ "database" ];
    imports = [ ./hosts/db-01.nix ];
  };
};
};
}

```

Case 2: Executing Fleet Rollout with Canary Verification

```

# 1. Evaluate and check syntax across all nodes
colmena eval

# 2. Deploy to canary application node first
colmena apply --on app-01

# 3. Verify health probe through edge gateway
curl -sf https://api.company.internal/healthz

# 4. Roll out to remaining fleet

```

```
colmena apply
```

Execution output:

```
[INFO ] Evaluating 4 nodes in parallel...
[INFO ] [app-01] Built generation 44
[INFO ] [app-01] Activated configuration.
[INFO ] Running post-deploy health check...
HTTP/2 200 OK
[INFO ] Deploying remainder of fleet (app-02, gw-01, db-01)...
[INFO ] All nodes synchronized successfully.
```

Case 3: Zero-Touch Disaster Recovery Playbook

When replacing a destroyed server (app-02):

```
# Boot replacement node into rescue RAM system, then execute:
nixos-anywhere --flake .#app-02 root@203.0.113.99
```

```
[nixos-anywhere] Formatting disks via Disko...
[nixos-anywhere] Partitioned ESP, opened LUKS2, created Btrfs subvolumes.
[nixos-anywhere] Copying closure /nix/store/7q1a...-nixos-system-app-02...
[nixos-anywhere] Installing systemd-boot bootloader...
[nixos-anywhere] Installation complete. Host rebooted.
```

The node boots, rejoins the WireGuard mesh, receives traffic from the Caddy load balancer, and passes all health checks without a human ever logging in.

The trap

The trap is **not testing disaster recovery until a production incident occurs**. Schedule automated recovery drills quarterly: wipe a staging server completely and run `nixos-anywhere` to prove your declarative configurations remain 100% self-sufficient.

The boring rule

- Model the entire multi-node cluster inside a single Colmena flake.
- Enforce canary node verification before rolling out cluster-wide updates.
- Verify that every server can be rebuilt from scratch with a single `nixos-anywhere` command.

Try this

1. Run `colmena eval` locally to verify that all four cluster nodes evaluate without errors.
2. Review the generated closure sizes with `colmena exec nix-store --gc --print-roots`.

Appendices

Glossary of Terms

Glossary of Terms

A concise, definitive reference for Nix and NixOS terminology.

Derivation (`.drv`)

The fundamental build specification in Nix. A derivation is an immutable file in the `/nix/store` describing exactly which inputs, tools, environment variables, and build scripts are required to produce an output artifact.

Nix Store (`/nix/store`)

The cryptographic, content-addressable filesystem where all packages, configurations, and build products reside. Paths are uniquely identified by a 32-character base32 hash derived from their inputs or contents.

Closure

The complete, self-contained set of all store paths required to run an application or boot an operating system. A closure has zero implicit external dependencies.

Flake

A standardized file layout centered on `flake.nix` and `flake.lock` that provides hermetic evaluation, explicit input pinning, and standardized outputs across all Nix repositories without global state.

DevShell

A temporary, isolated shell environment created by `nix develop` or `nix-shell` containing exactly the compilers, libraries, and tools specified for a project.

Binary Cache / Substituter

An HTTP service (such as `cache.nixos.org`, Cachix, or Attic) serving pre-compiled store closures. When Nix requires a store path, it queries substituters before compiling from source.

Garbage Collection Root (GC Root)

A symlink outside the Nix store (such as `/run/current-system` or `result`) that protects a store path and its entire dependency closure from being deleted by `nix-collect-garbage`.

Home Manager

A declarative user-environment management system using Nix modules to configure user-space dotfiles, shell environments, and user applications.

Overlay

A Nix function with signature `final: prev: { ... }` used to modify, override, or extend packages in `nixpkgs` without editing upstream files.

Module System

The extensible configuration engine powering NixOS and Home Manager. Modules define typed `options` and assign `config` values that merge deterministically across multiple files.

Command Cheat Sheet

Command Cheat Sheet

Essential daily commands for developing, building, and operating Nix and NixOS systems.

Flake and Development Commands

Command	Purpose
<code>nix develop</code>	Enter the default project development shell
<code>nix develop .#backend</code>	Enter a specific named development shell
<code>nix flake init -t <template></code>	Initialize a project from a flake template
<code>nix flake check</code>	Run all checks, tests, and linters defined in the flake
<code>nix flake update</code>	Update all inputs in <code>flake.lock</code> to latest upstream revisions
<code>nix flake update <input></code>	Update a single specific input (e.g. <code>nixpkgs</code>)
<code>nix flake show</code>	Display all packages, devShells, and modules in the flake

Building and Querying

Command	Purpose
<code>nix build .#package</code>	Build a flake package and link to <code>./result</code>
<code>nix run .#app</code>	Build and immediately execute a flake application
<code>nix eval .#attribute</code>	Evaluate a Nix expression and print the value
<code>nix log ./result</code>	Display the build log for an artifact or derivation
<code>nix path-info -rsh ./result</code>	Show the closure size and disk footprint of an artifact
<code>nix-tree ./result</code>	Interactively browse the dependency tree

NixOS Operations

Command	Purpose
<code>sudo nixos-rebuild switch --flake .#host</code>	Build, activate, and set as default boot generation
<code>sudo nixos-rebuild test --flake .#host</code>	Test configuration in the current session without updating bootloader
<code>sudo nixos-rebuild dry-activate --flake .#host</code>	Preview which services will restart without making changes
<code>sudo nixos-rebuild switch --rollback</code>	Instantly revert to the previous working generation
<code>nixos-rebuild list-generations</code>	List all historical system generations

Maintenance and Disk Cleanup

Command	Purpose
<code>nix-store --optimise</code>	Hard-link duplicate files across the store to save disk space
<code>nix-collect-garbage</code>	Delete unreferenced store paths
<code>nix-collect-garbage -d</code>	Delete all old generations and unreferenced store paths
<code>nix-collect-garbage --delete-older-than 30d</code>	Delete generations older than 30 days